Projector-Resolved Gibbs Reconstruction

A step-by-step study of a discontinuity, Fourier ringing, packet separation, finite recovery, and reconstruction

We follow one explicitly defined periodic signal from its elementary pieces through Fourier analysis, the Gibbs limit, packet separation, exact synthesis, interval recovery, perturbation tests, and noise experiments. Every calculation, plot, and numerical check comes from one Python file.

Komorebi P.

Worked mathematical and computational edition

Abstract

The whole argument follows one signal. On the circle

$$\mathbb{T} = \mathbb{R}/(2\pi\mathbb{Z}),$$

define

$$s(x) = \frac{1}{4} + \frac{2}{5}\cos(2x) - \frac{1}{3}\sin(3x) + \frac{1}{5}\cos(5x), \qquad j(x) = \mathbf{1}_{[-\pi/3,\,\pi/3)}(x),$$

and

$$f(x) = s(x) + j(x).$$

The smooth packet is a degree-five trigonometric polynomial. The edge packet is a unit rectangular discontinuity. Global Fourier truncation reproduces the smooth part exactly once the order is at least five, but it approximates the discontinuity by a Dirichlet-kernel convolution. At the right edge, after scaling

$$x = a - \frac{z}{N + 1/2}, \qquad a = \frac{\pi}{3},$$

the jump partial sums converge to

$$\frac{1}{2} + \frac{1}{\pi}\,\mathrm{Si}(z).$$

The first interior maximum occurs at $z = \pi$, producing the classical overshoot

$$G = \frac{\mathrm{Si}(\pi)}{\pi} - \frac{1}{2} = 0.089489872236\ldots.$$

The oscillatory region narrows, but the normalized first overshoot does not vanish.

The model space is the direct sum

$$\mathcal{M}_5 = \mathcal{T}_5 \oplus \mathrm{span}\{j\},$$

where $\mathcal{T}_5$ is the real degree-five trigonometric space. The associated projectors P_s and P_j give the exact analysis

$$f \longmapsto (P_\mathrm{s} f, P_\mathrm{j} f) = (s, j).$$

Instead of applying the same global Fourier truncation to both packets, define

$$\boxed{\mathcal{R}_N = S_N P_\mathrm{s} + P_\mathrm{j}.}$$

For every $N \geq 5$,

$$\mathcal{R}_N f = S_N s + j = s + j = f.$$

The discontinuity is not smoothed, filtered, or represented by an increasingly long Fourier tail. It is retained exactly as a geometric packet. In this finite model, the jump-generated Gibbs term never enters the reconstructed signal.

The second half removes the oracle assumption from the finite sampled example. On a 96-point grid, the smooth background is represented by its complete degree-five design matrix and every admissible positive cyclic interval packet of length at most half the grid is enumerated. After projecting both the data and each candidate packet off the smooth subspace, the best amplitude is obtained in closed form and the residual is minimized exactly. The true interval $(\mathrm{start}, \mathrm{length}) = (32, 32)$ is the unique minimizer. Its objective residual is zero to floating-point representation, while the next candidate has residual $0.7531607493\ldots$. The reconstructed data have maximum residual 5.55×10^{-16}.

The finite exact theorem is separated from numerical robustness experiments. Noise, off-grid edges, filtering comparisons, and runtime scaling are reported as computations. Universal edge discovery, arbitrary multidimensional singularities, and unrestricted Gibbs removal are not claimed.

The precise claim

What is proved. The raw Fourier partial sums of the chosen jump packet have the universal Gibbs edge profile. The smooth-edge model has complementary projectors. The operator $\mathcal{R}_N = S_N P_{\mathrm{s}} + P_{\mathrm{j}}$ is the identity on the whole model space for $N \geq 5$. On the displayed noise-free grid, profile least squares over the finite interval library recovers the unique true packet.

What is measured. The supplied program runs fixed numerical experiments on noise recovery, off-grid localization, filtering, transition width, runtime, and residual localization.

What lies outside this example. Ordinary Fourier partial sums still ring. The argument does not recover every discontinuity from arbitrary noisy data, and it does not claim historical priority for singularity subtraction or adapted representations. It gives one fully worked realization of the calculus.

How to read the book

The argument runs in this order:

seed → exact coefficients → Dirichlet kernel → Gibbs limit
→ model packets → resolved operator → finite sample projector
→ interval search → exact reconstruction → residual diagnostics
→ residual check.

We first define the signal and derive the Gibbs mechanism. We then build the packet reconstruction, recover the interval from samples, compare it with standard filters, and study what happens when the support or amplitude is wrong. The last part gives the numerical checks and the full source listing.

Contents

Abstract i

The precise claim ii

How to read the book iii

Part I: The single signal 1

1 The primitive seed 1
- 1.1 Ambient space and periodic coordinate . 1
- 1.2 The smooth seed 1
- 1.3 The geometric edge packet 2
- 1.4 The complete signal 2

2 The first exact packet decomposition 4
- 2.1 The model class 4

Part II: Why global Fourier truncation rings 6

3 Exact Fourier coefficients 6
- 3.1 The edge coefficient calculation 6
- 3.2 Six residue packets in frequency 6
- 3.3 The full coefficient formula 6

4 The Dirichlet-kernel mechanism 9
- 4.1 Partial sums as convolution 9
- 4.2 Why sharpening produces oscillation . . 9

5 The scaled Gibbs limit 11
- 5.1 Right-edge scaling 11
- 5.2 The first maximum 11

Part III: The packet reconstruction 18

6 The resolved operator 18
- 6.1 Why the global operator is the wrong block assignment 18
- 6.2 Block matrix 18

7 Extension beyond finite-band smooth packets 21
- 7.1 Analytic smooth background 21

8 Why filtering is different 22
- 8.1 Fejer averaging 22
- 8.2 Lanczos and exponential filters 22
- 8.3 Packet retention changes the approximation problem 22

Part IV: Recovering the packet from samples 26

9 The finite observation model 26
- 9.1 The sample grid 26
- 9.2 The smooth design matrix 26

10 Profile least squares over interval packets 29
- 10.1 Eliminating the smooth coefficients . . . 29
- 10.2 The finite identifiability gap 29

11 Reconstruction after packet recovery 32
- 11.1 Recovered smooth packet 32
- 11.2 Recovered coefficients 32

12 The single executable algorithm 34

Part V: Perturbation and residual-driven refinement 35

13 What happens when the packet is wrong 35
- 13.1 One-sample mislocation 35
- 13.2 Amplitude mismatch 35

14 Noise experiments 39
- 14.1 Status of the experiment 39
- 14.2 Observed behavior 39

15 Off-grid edges 42
- 15.1 Sampling resolution 42
- 15.2 Refinement experiment 42

Part VI: Exact reconstruction and approximation 44

16 Error ledger 44
- 16.1 Metrics 44
- 16.2 Exact versus approximate operations . . 44

17 Computational complexity and alternatives 46
- 17.1 Reference implementation cost 46
- 17.2 Accelerations consistent with the proof . 46

18 How the proof fits together 47
- 18.1 Two independent chains 47

Part VII: Verification and reproducibility 49

19 Verification table 49
- 19.1 Independent checks 49

20 One-command reproduction **51**

21 Final theorem and conclusion **52**

Part VIII: Appendices **53**

A Exact formula sheet **53**
A.1 Signal 53
A.2 Fourier coefficients 53
A.3 Partial sum 53
A.4 Gibbs profile 53
A.5 Resolved operator 53
A.6 Finite profile objective 53

B Selected numerical tables **54**
B.1 Exact recovery record 54
B.2 Machine-readable tables 54

C The full Python implementation **55**

D File map **80**

E What this example establishes **81**

List of Figures

1.1 The smooth seed, geometric edge packet, and complete signal. The vertical discontinuities occur at $x = \pm\pi/3$. 2
1.2 Primitive atoms from which the complete signal is assembled. 3
1.3 The edge packet as a geometric object with two oriented boundaries. 3

2.1 The calculation from the signal seed to the final residual. 5
2.2 The exact decomposition $f = P_s f + P_j f$. . . . 5

3.1 Exact Fourier magnitudes of the smooth and edge packets. 7
3.2 The exact six-phase pattern of $\sin(n\pi/3)$. . . 7
3.3 The algebraic edge-coefficient tail and its $1/(\pi n)$ envelope. 8

4.1 Dirichlet kernels at four orders. 10
4.2 The local convolution mechanism at the right edge. 10

5.1 Raw global Fourier reconstruction at $N = 5$. . 12
5.2 Raw global Fourier reconstruction at $N = 10$. 12
5.3 Raw global Fourier reconstruction at $N = 20$. 13
5.4 Raw global Fourier reconstruction at $N = 40$. 13
5.5 Right-edge detail at $N = 10$. 14
5.6 Right-edge detail at $N = 20$. 14
5.7 Right-edge detail at $N = 40$. 15
5.8 Collapse of finite edge profiles onto the sine-integral limit. 15
5.9 Convergence of the first scaled overshoot to G. 16
5.10 Overshoot for raw, filtered, and packet-resolved methods. 16
5.11 Global RMSE as the spectral order grows. . . 17
5.12 Boundary-band RMSE. 17

6.1 Analysis, packet-specific operation, and synthesis. 19
6.2 Block matrix of the resolved reconstruction operator. 19
6.3 Exact resolved reconstruction of the continuous worked signal. 20
6.4 Pointwise residual of the resolved reconstruction. 20

8.1 Fejer reconstruction of the jump packet. . . . 22
8.2 Lanczos-filtered jump reconstruction. 23
8.3 Exponentially filtered jump reconstruction. . 23
8.4 Full-signal comparison at $N = 40$. 24
8.5 Right-edge comparison at $N = 40$. 24
8.6 Four complementary error measures at $N = 40$. 25
8.7 Boundary error versus transition width. . . . 25

9.1 The continuous signal and its 96 observed values. 27
9.2 The complete degree-five sampled trigonometric design. 27
9.3 Residualizing the data and the true mask against the smooth subspace. 28

10.1 The ranked profile objective values. 30
10.2 The complete start-length objective surface. . 30
10.3 Best residual, second-best residual, and their gap. 31

11.1 Data, recovered smooth packet, and recovered edge packet. 32
11.2 True and recovered smooth coefficients. . . . 33
11.3 Sample-space synthesis and residual. 33

13.1 A one-sample edge shift and its residual. . . . 36
13.2 Residuals for several amplitude errors. 36
13.3 Exact support-error formula for shifted masks. 37
13.4 Absolute residual under a wrong interval packet. 37
13.5 Residual-driven local refinement. 38

14.1 Three representative noisy data sets and their recovered packet models. 39
14.2 Exact support-recovery frequency in the deterministic Monte Carlo suite. 40
14.3 Mean absolute start-index error. 40
14.4 Mean absolute amplitude error. 41
14.5 Mean reconstruction error in a seven-sample band around each boundary. 41

15.1 One off-grid sampled example and its best grid packet. 42
15.2 Endpoint error and grid spacing under refinement. 43

16.1 Residue-class refinement of the edge coefficient sequence. 45
16.2 The final claim boundary. 45

17.1 Measured library-build and profile-search times for the reference code. 46

18.1 Dependency map of the failure theorem and the reconstruction theorem. 47
18.2 The calculation from start to finish. 48

19.1 Verification results generated by the program. 50

1 The primitive seed

Part I: The single signal
We begin with the signal itself. Its smooth and discontinuous pieces are written down before any approximation is applied.

1.1 Ambient space and periodic coordinate

The ambient continuous coordinate is the circle

$$\mathbb{T} = \mathbb{R}/(2\pi\mathbb{Z}).$$

Representatives are chosen in $[-\pi, \pi)$. Addition is always interpreted modulo 2π. The complex Fourier characters are

$$e_n(x) = \mathrm{e}^{\mathrm{i}nx}, \qquad n \in \mathbb{Z}.$$

They satisfy

$$\frac{1}{2\pi}\int_{-\pi}^{\pi} e_n(x)\overline{e_m(x)}\,dx = \delta_{nm}.$$

The Fourier coefficient convention is

$$\widehat{g}(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} g(x)\mathrm{e}^{-\mathrm{i}nx}\,dx.$$

The order-N partial-sum operator is

$$S_N g(x) = \sum_{|n|\le N} \widehat{g}(n)\mathrm{e}^{\mathrm{i}nx}.$$

1.2 The smooth seed

Define

$$\boxed{s(x) = \frac{1}{4} + \frac{2}{5}\cos(2x) - \frac{1}{3}\sin(3x) + \frac{1}{5}\cos(5x).}$$

The smooth part is not an unspecified remainder. It is the following degree-five trigonometric polynomial. Its nonzero complex coefficients are

$$\widehat{s}(0) = \frac{1}{4}, \qquad \widehat{s}(\pm 2) = \frac{1}{5}, \qquad \widehat{s}(3) = \frac{\mathrm{i}}{6}, \qquad \widehat{s}(-3) = -\frac{\mathrm{i}}{6}, \qquad \widehat{s}(\pm 5) = \frac{1}{10}.$$

Therefore

$$S_N s = s \qquad (N \ge 5).$$

The smooth packet is deliberately finite so that every remaining error can be attributed exactly to the treatment of the discontinuity.

1.3 The geometric edge packet

Let

$$a = \frac{\pi}{3}$$

and define the periodic half-open interval packet

$$\boxed{j(x) = \mathbf{1}_{[-a,a)}(x).}$$

Its one-sided values at the right edge are

$$j(a^-) = 1, \qquad j(a^+) = 0.$$

Its jump amplitude is exactly one. At the left edge the orientation is reversed.

1.4 The complete signal

The one signal used throughout the book is

$$\boxed{f = s + j.}$$

It has no hidden random component. Every coefficient, edge, sample, and reconstruction is generated from this formula.

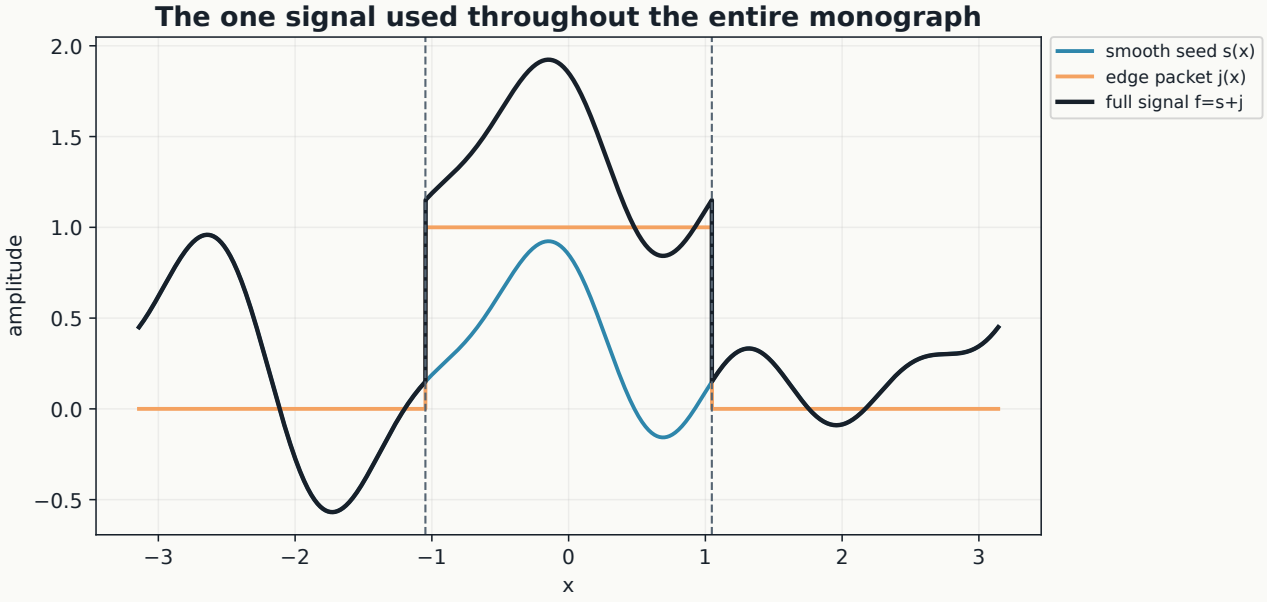


Figure 1.1: The smooth seed, geometric edge packet, and complete signal. The vertical discontinuities occur at $x = \pm\pi/3$.

What the figure shows. The key distinction is already visible. The smooth packet is global and oscillatory but finite-band. The edge packet is spatially simple and spectrally long-tailed. Treating them with the same approximation operator is a coordinate choice, not a necessity.

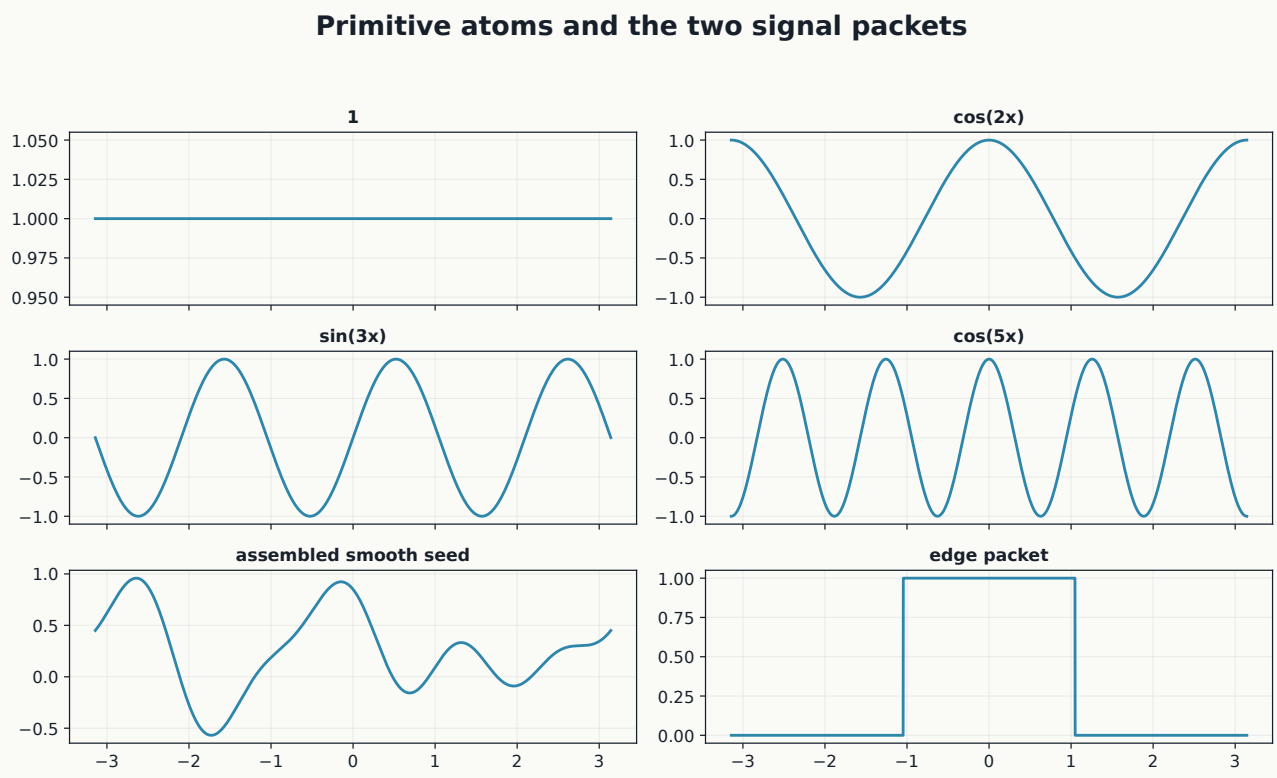


Figure 1.2: Primitive atoms from which the complete signal is assembled.

What the figure shows. The signal is built from a constant, three harmonic atoms, scalar multiplication, addition, and one interval indicator. Nothing new is added after the target signal has been chosen.

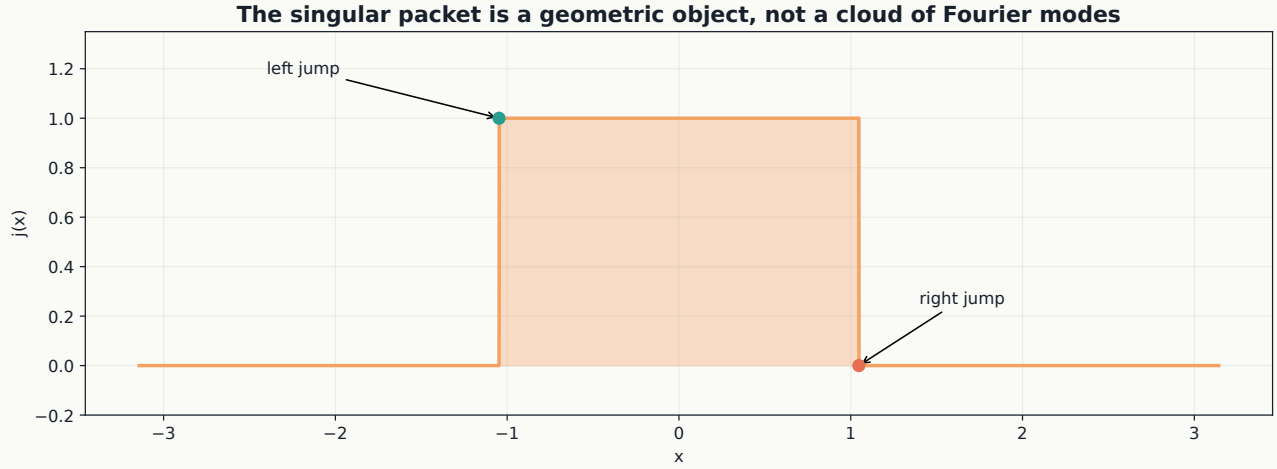


Figure 1.3: The edge packet as a geometric object with two oriented boundaries.

What the figure shows. The right edge is the location at which the detailed Gibbs calculation will be performed. The left edge behaves by symmetry.

2 The first exact packet decomposition

2.1 The model class

Let $\mathcal{T}_5$ be the real trigonometric polynomials of degree at most five:

$$\mathcal{T}_5 = \operatorname{span}_{\mathbb{R}}\{1, \cos x, \sin x, \ldots, \cos 5x, \sin 5x\}.$$

Define

$$\mathcal{J} = \operatorname{span}_{\mathbb{R}}\{j\}.$$

Since every member of $\mathcal{T}_5$ is continuous and every nonzero member of $\mathcal{J}$ has a jump, the intersection is trivial:

$$\mathcal{T}_5 \cap \mathcal{J} = \{0\}.$$

Hence

$$\boxed{\mathcal{M}_5 = \mathcal{T}_5 \oplus \mathcal{J}}$$

is a direct sum.

Definition 2.1 (Smooth and edge projectors)**.** *For $g = t + cj \in \mathcal{M}_5$, define*

$$P_{\mathrm{s}} g = t, \qquad P_{\mathrm{j}} g = cj.$$

Theorem 2.2 (Exact two-packet calculus)**.** *On $\mathcal{M}_5$,*

$$P_{\mathrm{s}}^2 = P_{\mathrm{s}}, \qquad P_{\mathrm{j}}^2 = P_{\mathrm{j}}, \qquad P_{\mathrm{s}} P_{\mathrm{j}} = P_{\mathrm{j}} P_{\mathrm{s}} = 0, \qquad P_{\mathrm{s}} + P_{\mathrm{j}} = I.$$

Consequently every $g \in \mathcal{M}_5$ has the exact analysis-synthesis identity

$$\boxed{g = P_{\mathrm{s}} g + P_{\mathrm{j}} g.}$$

Proof. Write $g = t + cj$. Then $P_{\mathrm{s}} g = t$ and $P_{\mathrm{j}} g = cj$. Reapplying either map does not change its selected component, so both are idempotent. Applying P_{s} to cj gives zero, and applying P_{j} to t gives zero. Finally $P_{\mathrm{s}} g + P_{\mathrm{j}} g = t + cj = g$. □

Remark 2.3. These are algebraic projectors on the chosen direct sum. They need not be orthogonal in the ambient L^2 inner product. Exact reconstruction requires the two ranges to be complementary and complete. Orthogonality is a separate geometric question.

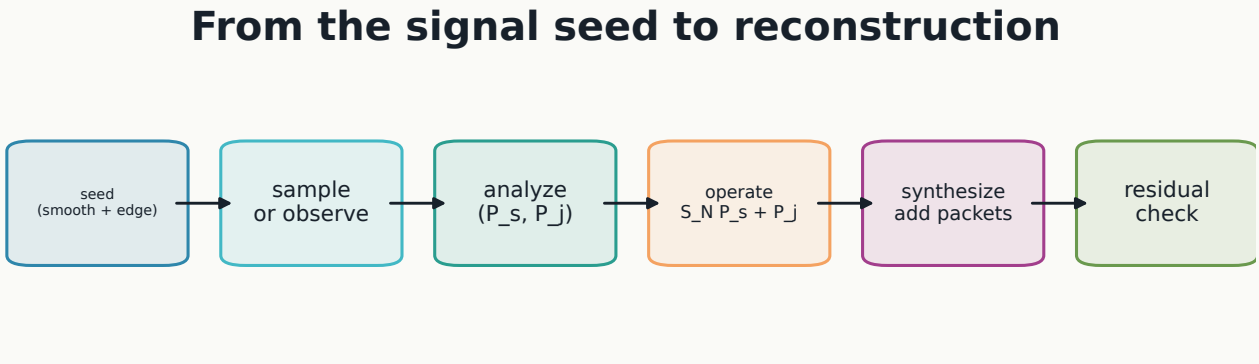


Figure 2.1: The calculation from the signal seed to the final residual.

What the figure shows. The smooth packet enters the Fourier block. The edge packet passes through the identity block. The two outputs are then added.

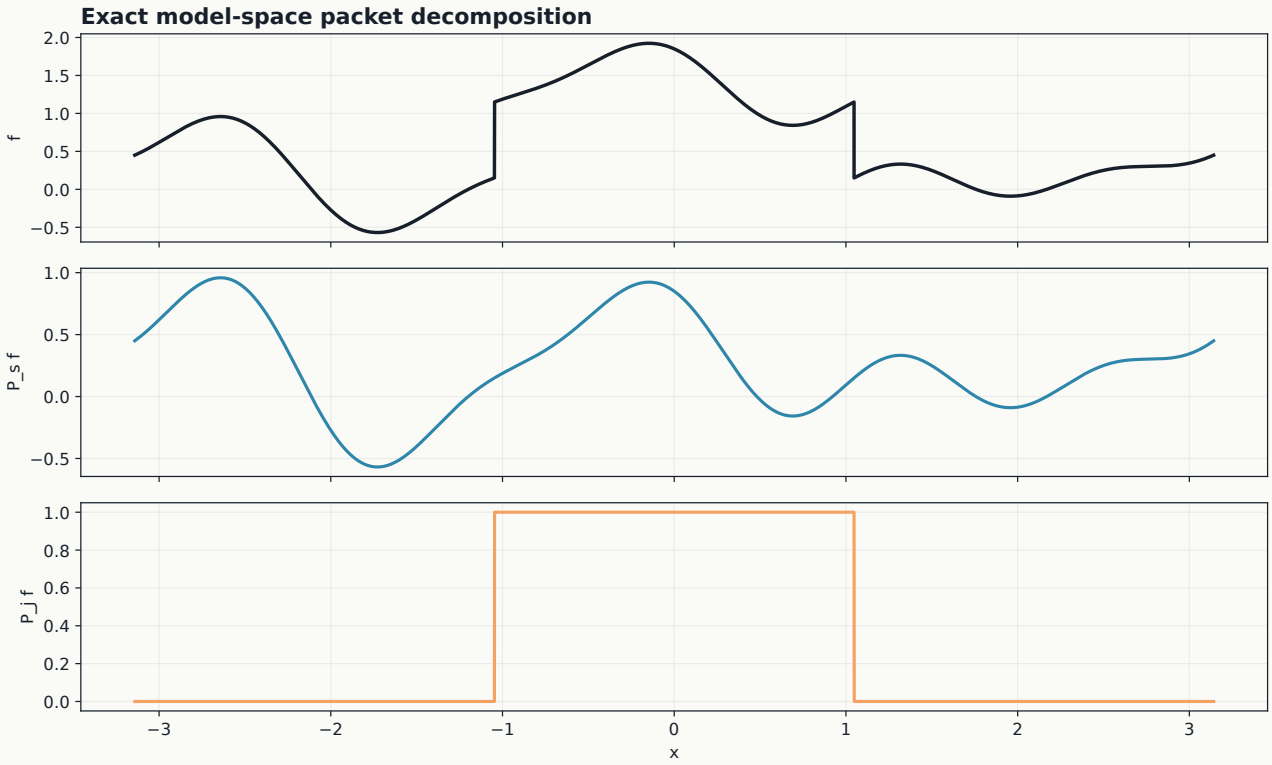


Figure 2.2: The exact decomposition $f = P_{\mathrm{s}} f + P_{\mathrm{j}} f$.

What the figure shows. This is the structural step on which the entire reconstruction rests. Gibbs ringing will be traced to applying S_N to $P_{\mathrm{j}} f$ rather than preserving that packet.

3 Exact Fourier coefficients

Part II: Why global Fourier truncation rings
This part derives the coefficient tail, the Dirichlet-kernel convolution, and the scaled sine-integral profile. The overshoot is traced to a precise local limit near the jump.

3.1 The edge coefficient calculation

For $n \neq 0$,

$$\begin{aligned}\widehat{j}(n) &= \frac{1}{2\pi}\int_{-a}^{a} \mathrm{e}^{-\mathrm{i}nx}\,dx \\ &= \frac{1}{2\pi}\left[\frac{\mathrm{e}^{-\mathrm{i}nx}}{-\mathrm{i}n}\right]_{-a}^{a} \\ &= \frac{\mathrm{e}^{\mathrm{i}na}-\mathrm{e}^{-\mathrm{i}na}}{2\pi\mathrm{i}n} \\ &= \boxed{\frac{\sin(na)}{\pi n}}.\end{aligned}$$

For $n = 0$,

$$\widehat{j}(0) = \frac{1}{2\pi}\int_{-a}^{a} 1\,dx = \frac{a}{\pi} = \frac{1}{3}.$$

Thus

$$\boxed{\widehat{j}(n) = \begin{cases} 1/3, & n = 0, \\ \sin(n\pi/3)/(\pi n), & n \neq 0. \end{cases}}$$

3.2 Six residue packets in frequency

The factor $\sin(n\pi/3)$ has the exact period-six pattern

$$\sin\left(\frac{n\pi}{3}\right) = \begin{cases} 0, & n \equiv 0, 3 \pmod 6, \\ \sqrt{3}/2, & n \equiv 1, 2 \pmod 6, \\ -\sqrt{3}/2, & n \equiv 4, 5 \pmod 6. \end{cases}$$

Therefore the jump's Fourier sequence already carries a polyphase decomposition. Two residue packets vanish, two are positive, and two are negative. The factor $1/n$ remains and prevents finite spectral closure.

3.3 The full coefficient formula

Combining the smooth and jump packets gives

$$\widehat{f}(n) = \widehat{s}(n) + \widehat{j}(n).$$

For $|n| > 5$,

$$\widehat{f}(n) = \widehat{j}(n) = \frac{\sin(n\pi/3)}{\pi n}.$$

The asymptotic decay is only $O(1/|n|)$ along the four nonzero residue classes. This is the spectral signature of the discontinuity.

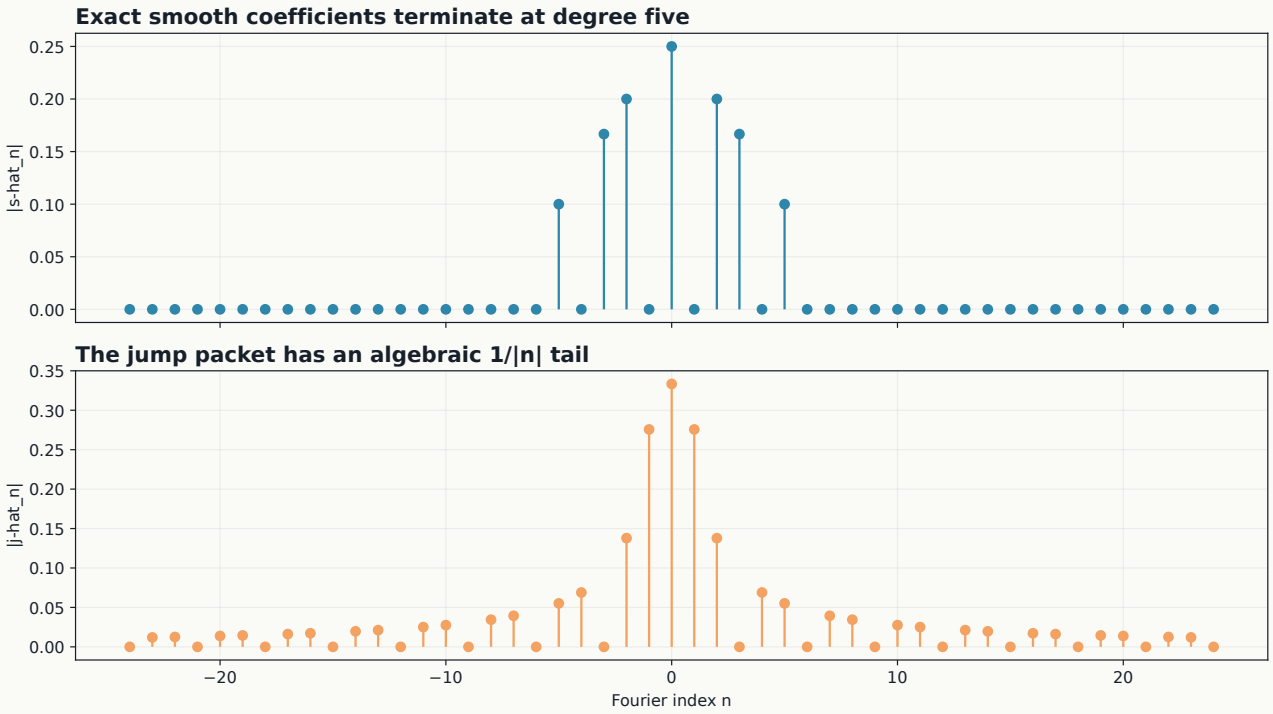


Figure 3.1: Exact Fourier magnitudes of the smooth and edge packets.

What the figure shows. The smooth spectrum terminates. The edge spectrum does not. Global truncation therefore spends all additional resolution on approximating the discontinuity.

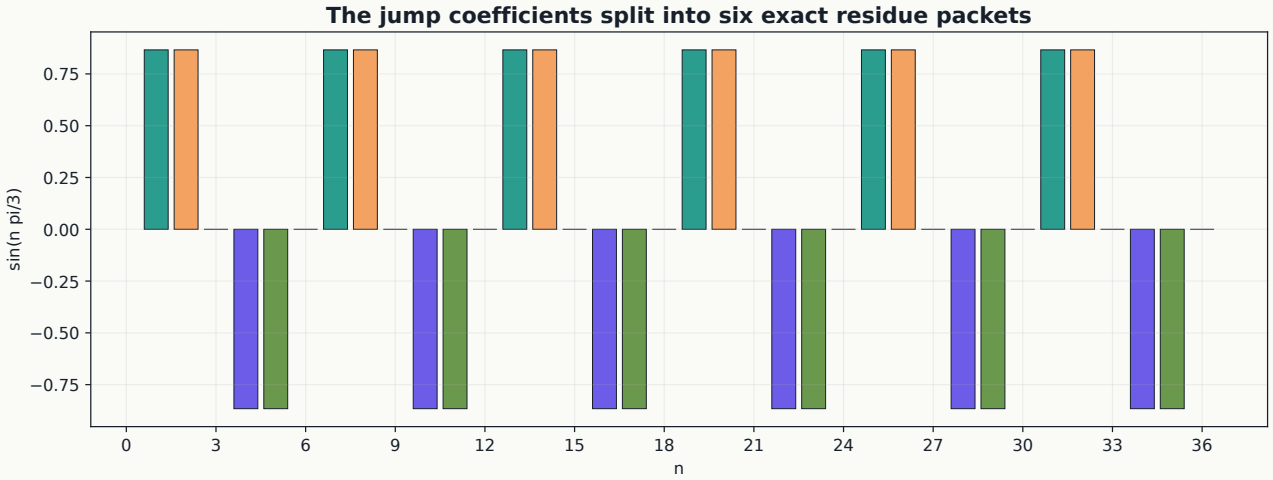


Figure 3.2: The exact six-phase pattern of $\sin(n\pi/3)$.

What the figure shows. The coefficients split into six residue classes. This does not remove the $1/n$ tail. It makes the arithmetic pattern visible.

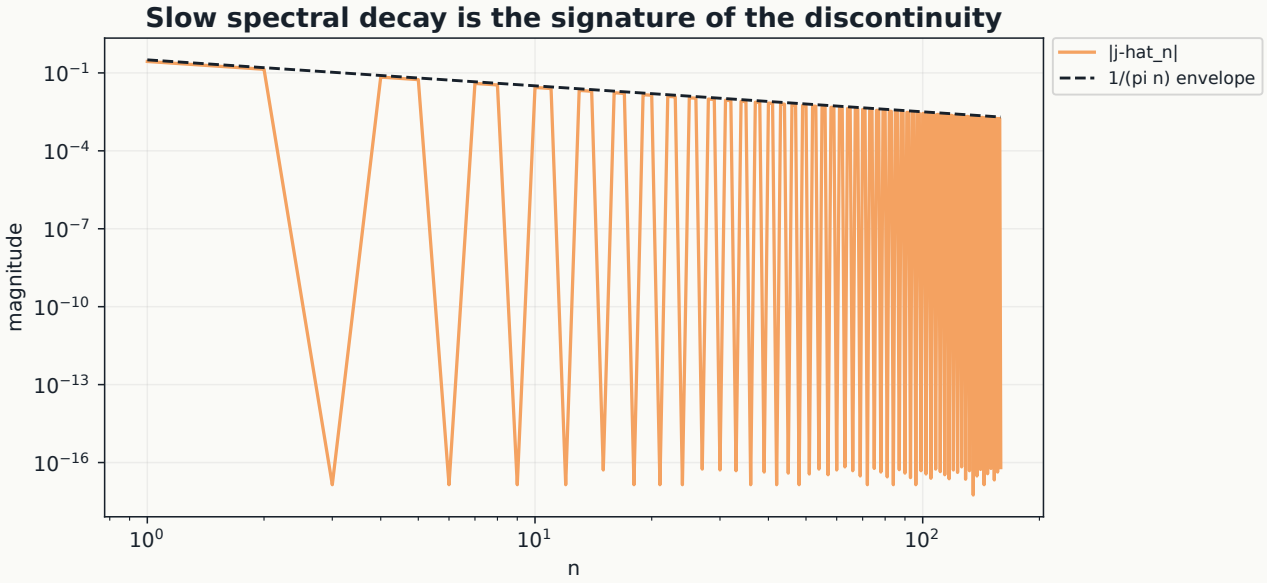


Figure 3.3: The algebraic edge-coefficient tail and its $1/(\pi n)$ envelope.

What the figure shows. No amount of increasing N turns the jump packet into a finite Fourier object.

4 The Dirichlet-kernel mechanism

4.1 Partial sums as convolution

The order-N Dirichlet kernel is

$$D_N(u) = \sum_{n=-N}^{N} \mathrm{e}^{inu} = \frac{\sin((N+1/2)u)}{\sin(u/2)}.$$

For every integrable periodic function,

$$S_N g(x) = \frac{1}{2\pi} \int_{-\pi}^{\pi} D_N(x-t) g(t)\, dt.$$

For the interval packet,

$$\boxed{S_N j(x) = \frac{1}{2\pi} \int_{-a}^{a} D_N(x-t)\, dt.}$$

Equivalently, direct coefficient substitution gives

$$\boxed{S_N j(x) = \frac{1}{3} + \frac{2}{\pi} \sum_{n=1}^{N} \frac{\sin(n\pi/3)}{n} \cos(nx).}$$

4.2 Why sharpening produces oscillation

The mass of D_N concentrates near the origin as N grows, but the kernel is not nonnegative. It has alternating sidelobes whose amplitudes are large enough to survive the edge scaling. Convolution at a discontinuity necessarily samples both one-sided values. The width of the resulting transition decreases, while the normalized first oscillation approaches a nonzero constant.

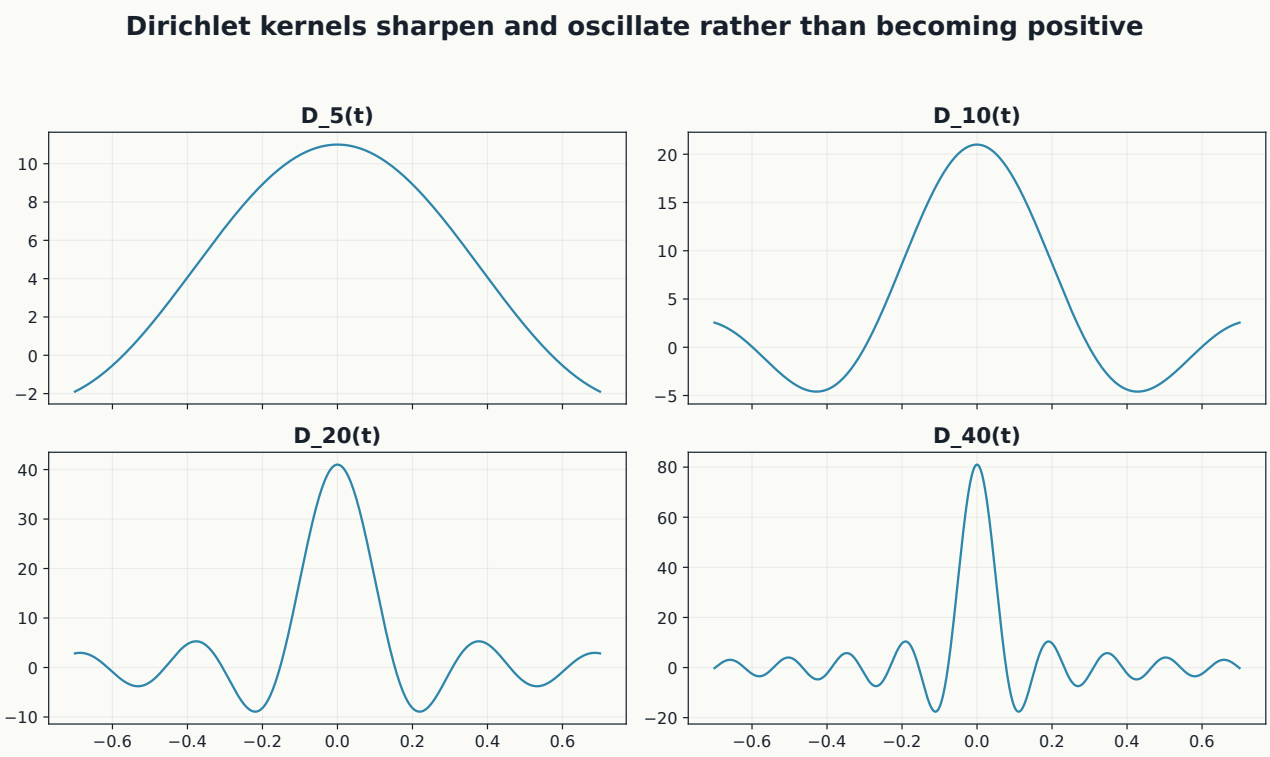


Figure 4.1: Dirichlet kernels at four orders.

What the figure shows. The central lobe becomes narrower, but the kernel retains alternating positive and negative sidelobes.

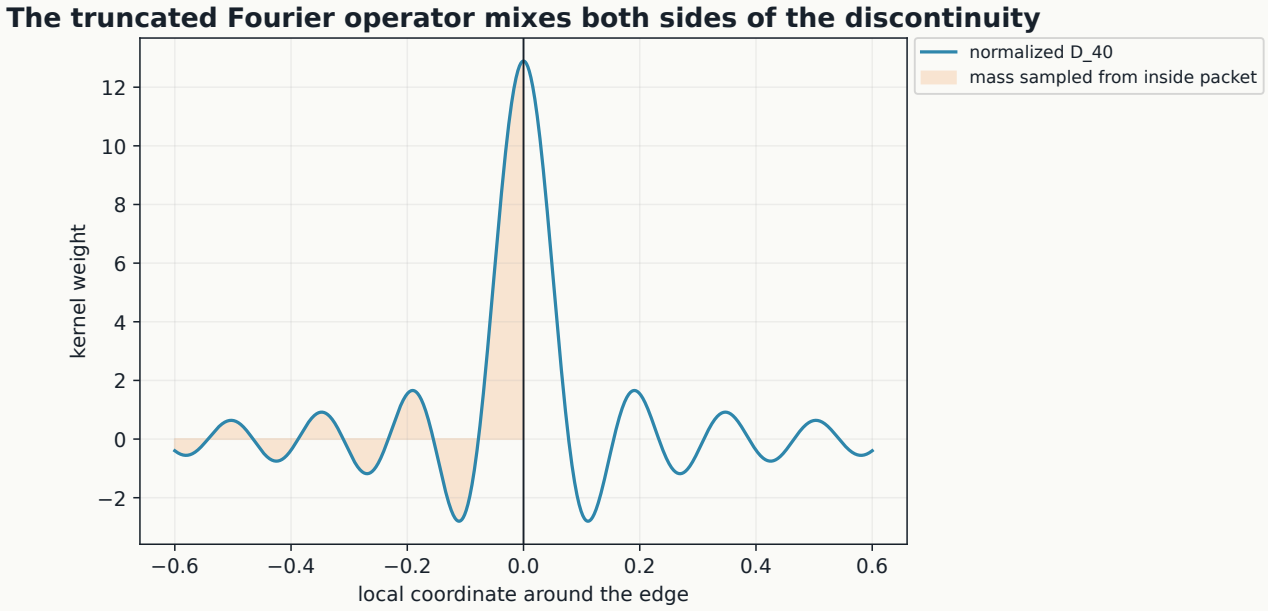


Figure 4.2: The local convolution mechanism at the right edge.

What the figure shows. The global truncation operator cannot know that the left and right samples belong to different geometric packets. It mixes both sides through the same kernel.

5 The scaled Gibbs limit

5.1 Right-edge scaling

Focus on the right edge $x = a$. Approach it from the interior with

$$x_N(z) = a - \frac{z}{N + 1/2}, \qquad z \in \mathbb{R}.$$

Positive z lies inside the interval. The local scale is exactly the reciprocal width of the main Dirichlet lobe.

Theorem 5.1 (Universal edge profile)**.** *For each fixed $z \in \mathbb{R}$,*

$$\boxed{\lim_{N\to\infty} S_N j\left(a - \frac{z}{N + 1/2}\right) = \frac{1}{2} + \frac{1}{\pi}\,\mathrm{Si}(z),}$$

where

$$\mathrm{Si}(z) = \int_0^z \frac{\sin u}{u}\,du.$$

Proof. Write the convolution integral in the local variable $u = (N + 1/2)(t - a)$. Only the endpoint $t = a$ contributes a nonuniform term. The opposite endpoint stays a fixed distance away. In the endpoint window,

$$\sin\left(\frac{x - t}{2}\right) \sim \frac{x - t}{2},$$

and the rescaled Dirichlet kernel becomes the sinc kernel. The midpoint contribution gives $1/2$, while integrating the local sinc profile from 0 to z gives $\mathrm{Si}(z)/\pi$. Contributions away from the edge converge uniformly and supply the correct one-sided baseline. Combining the terms yields the stated limit. □

5.2 The first maximum

Differentiate the limiting profile:

$$\frac{d}{dz}\left(\frac{1}{2} + \frac{1}{\pi}\,\mathrm{Si}(z)\right) = \frac{1}{\pi}\frac{\sin z}{z}.$$

The first positive critical point is $z = \pi$. It is a maximum. Therefore

$$\frac{1}{2} + \frac{\mathrm{Si}(\pi)}{\pi} = 1 + G,$$

where

$$\boxed{G = \frac{\mathrm{Si}(\pi)}{\pi} - \frac{1}{2} = 0.089489872236\ldots.}$$

The normalized overshoot is about 8.949% of the jump amplitude.

Corollary 5.2 (Persistence on the complete signal)**.** *For every $N \geq 5$, the smooth packet is reproduced exactly by S_N. Hence the local overshoot of $S_N f$ relative to the interior branch $s+1$ is exactly the overshoot of $S_N j$ relative to* 1*.*

Boundary 5.3. The theorem states that the first scaled overshoot persists. It does not say that the oscillatory region keeps a fixed physical width. Its width is $O(1/N)$.

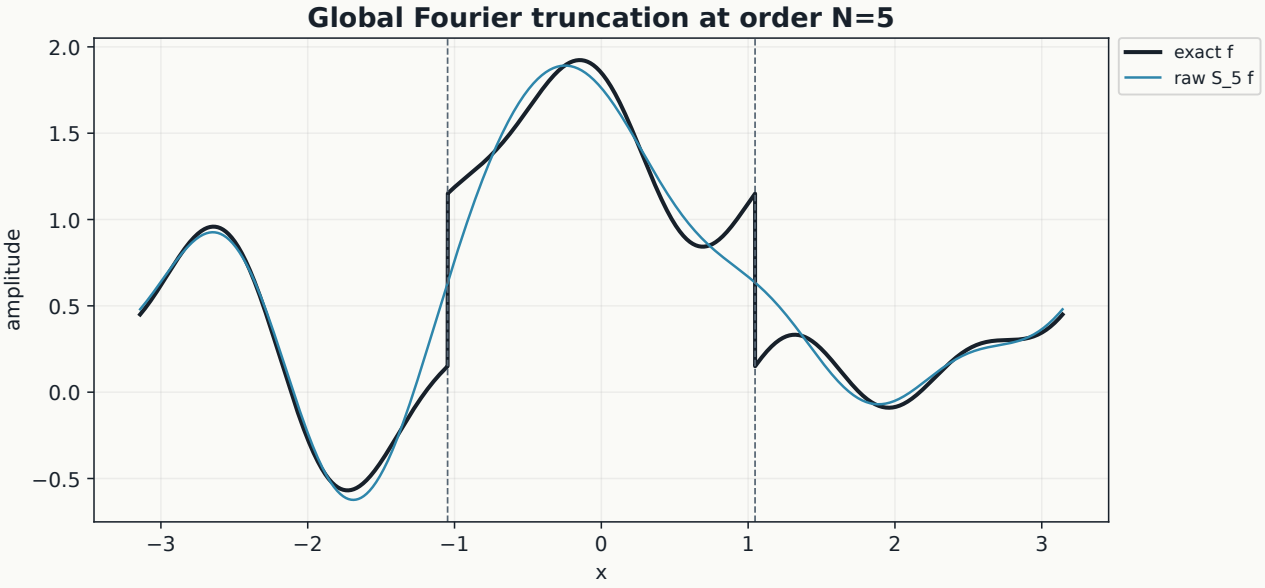


Figure 5.1: Raw global Fourier reconstruction at $N = 5$.

What the figure shows. At the first order that exactly contains the smooth seed, all visible error is generated by the jump packet.

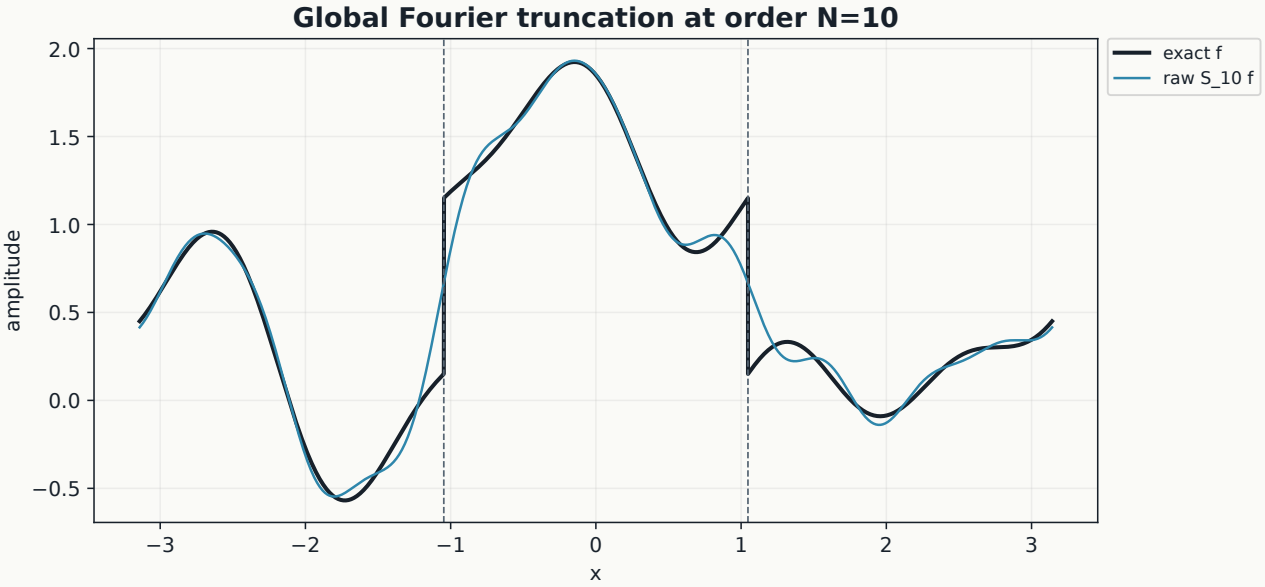


Figure 5.2: Raw global Fourier reconstruction at $N = 10$.

What the figure shows. Additional modes narrow the ringing region but do not eliminate its edge mechanism.

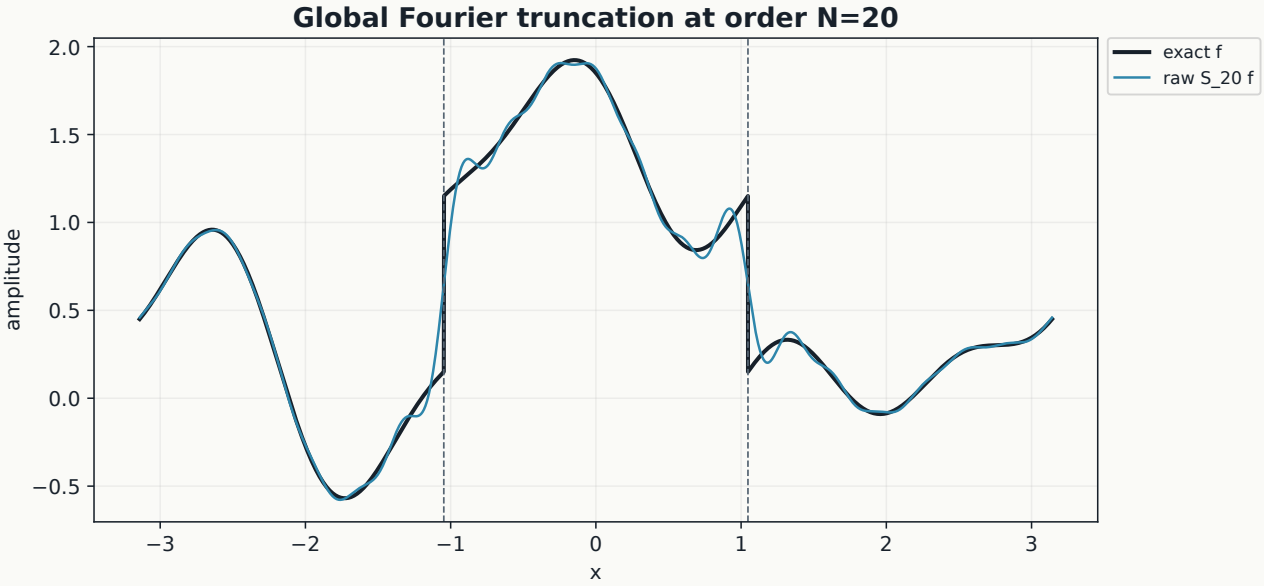


Figure 5.3: Raw global Fourier reconstruction at $N = 20$.

What the figure shows. The global error decreases while the first edge overshoot remains comparable to the jump amplitude.

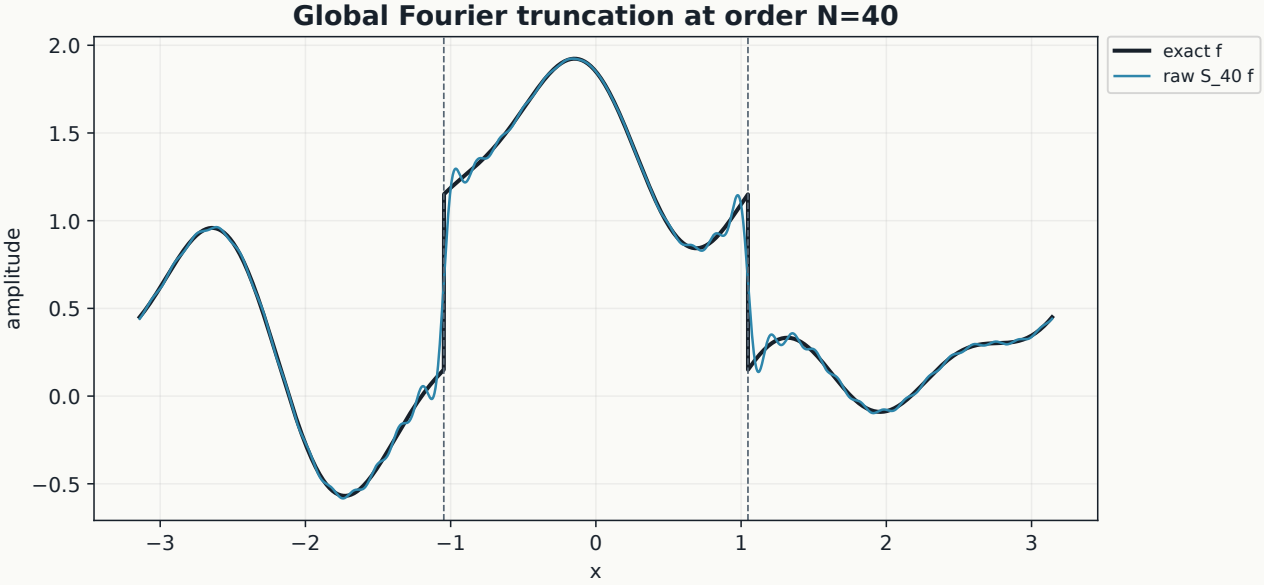


Figure 5.4: Raw global Fourier reconstruction at $N = 40$.

What the figure shows. The oscillation is localized more tightly around the two boundaries.

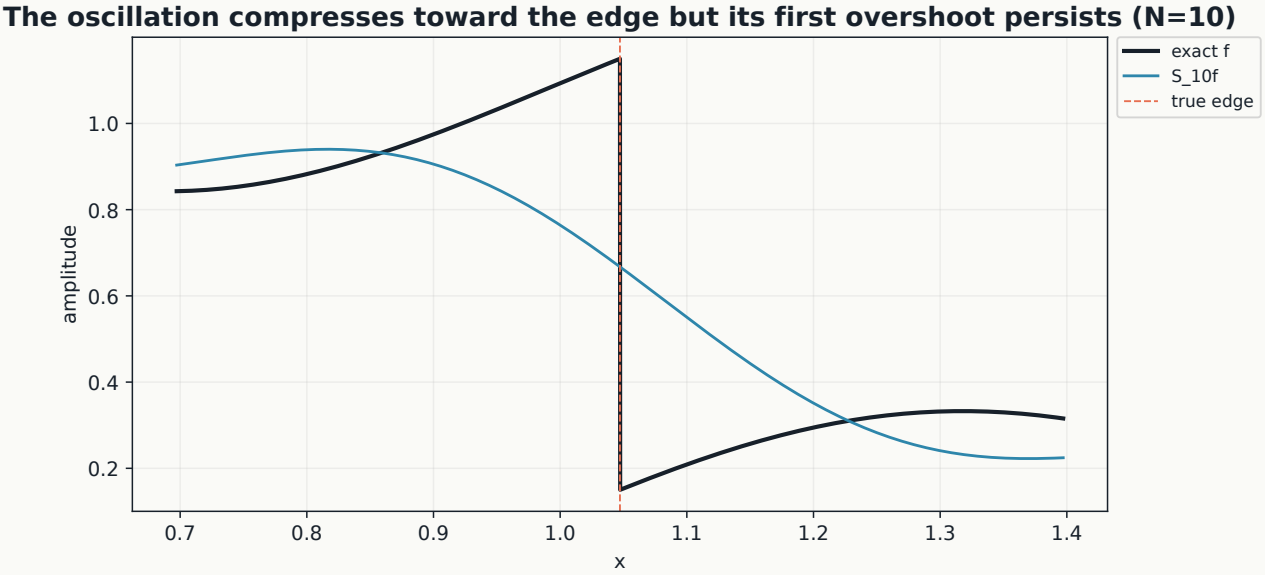


Figure 5.5: Right-edge detail at $N = 10$.

What the figure shows. The approximation crosses the exact one-sided levels and oscillates around them.

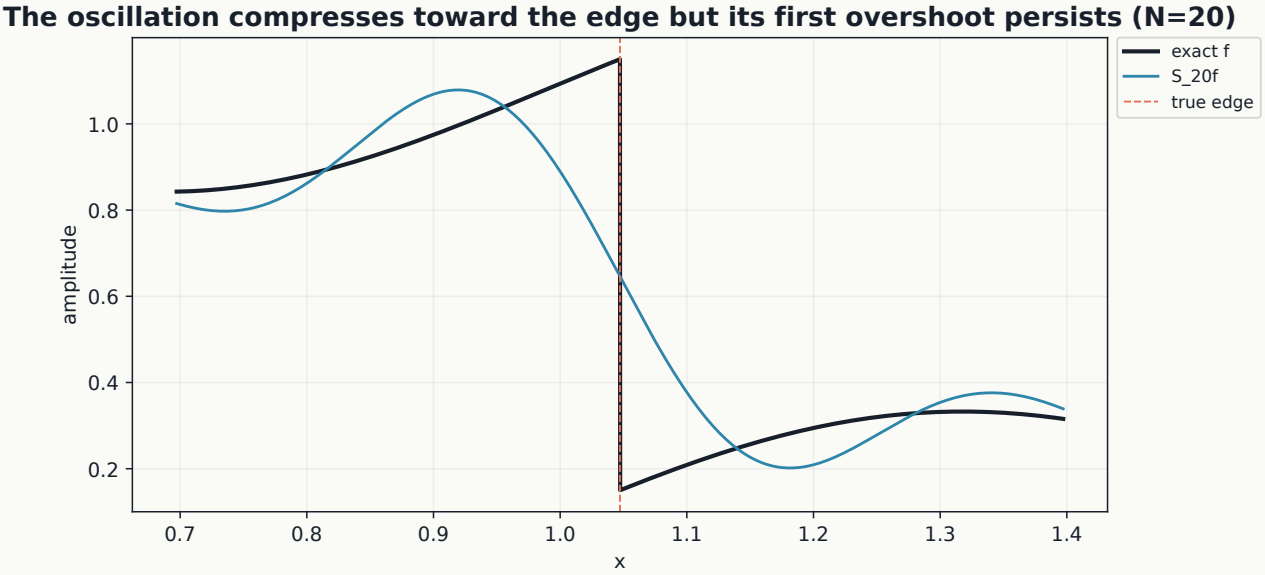


Figure 5.6: Right-edge detail at $N = 20$.

What the figure shows. The width contracts. The first overshoot becomes more clearly separated.

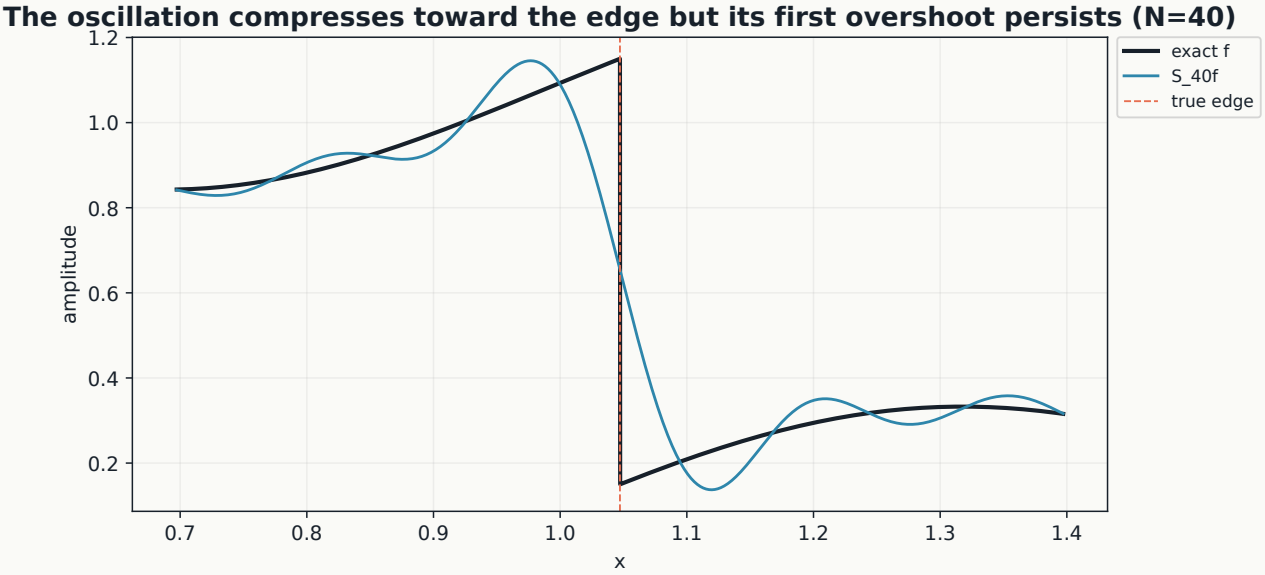


Figure 5.7: Right-edge detail at $N = 40$.

What the figure shows. The maximum is close to the universal normalized value.

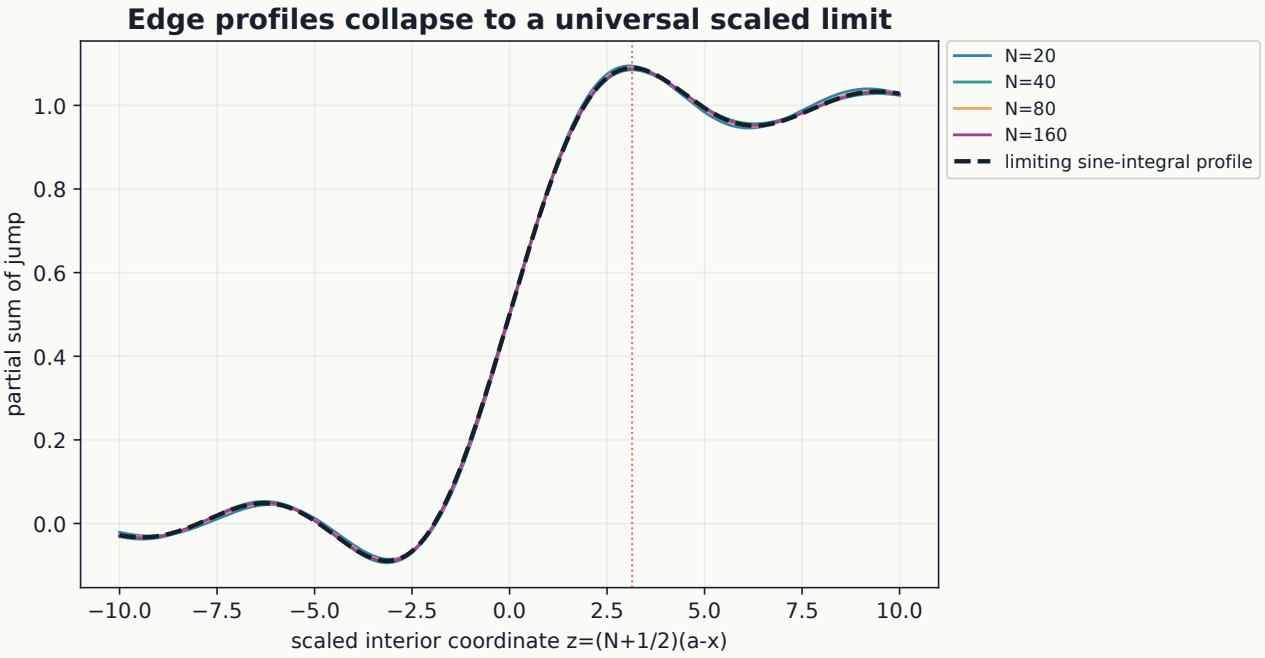


Figure 5.8: Collapse of finite edge profiles onto the sine-integral limit.

What the figure shows. The correct comparison variable is $z = (N + 1/2)(a - x)$, not the unscaled spatial coordinate.

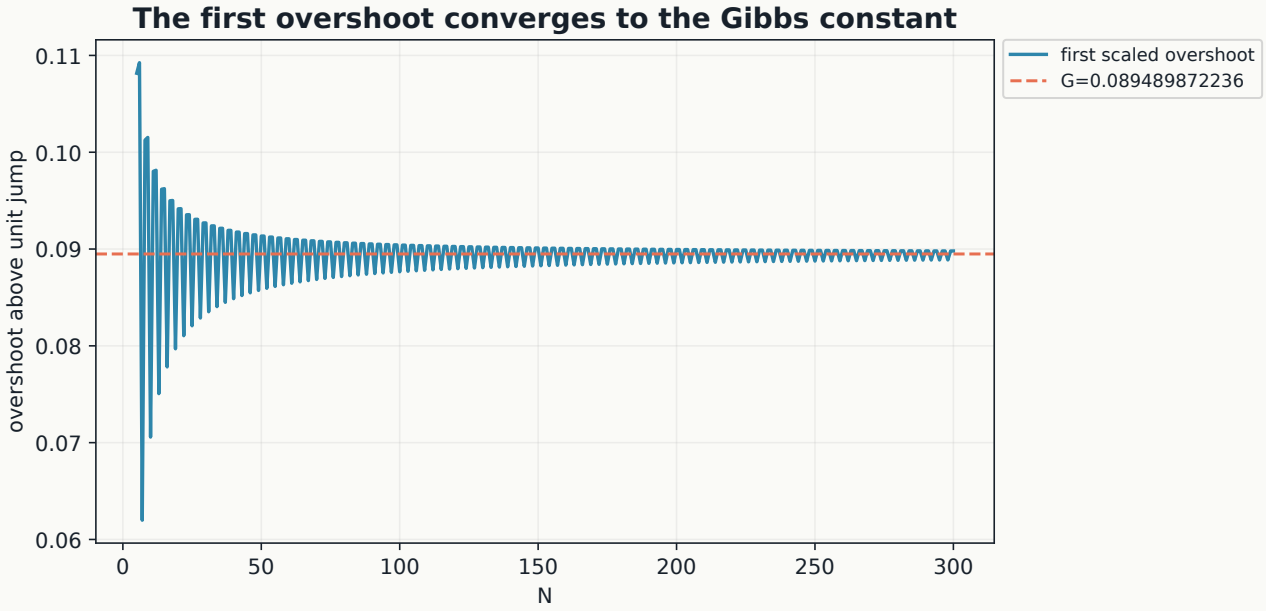


Figure 5.9: Convergence of the first scaled overshoot to G.

What the figure shows. The plotted value is evaluated at the theoretically selected point $x = a - \pi/(N + 1/2)$.

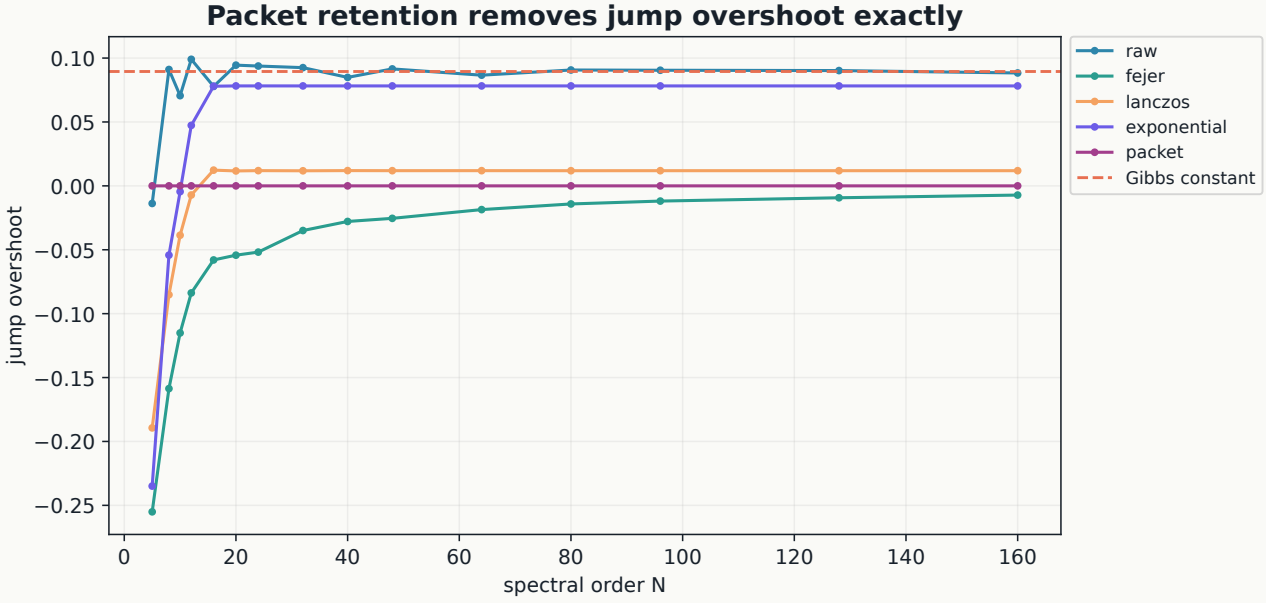


Figure 5.10: Overshoot for raw, filtered, and packet-resolved methods.

What the figure shows. The packet curve is identically zero because the edge packet is never truncated.

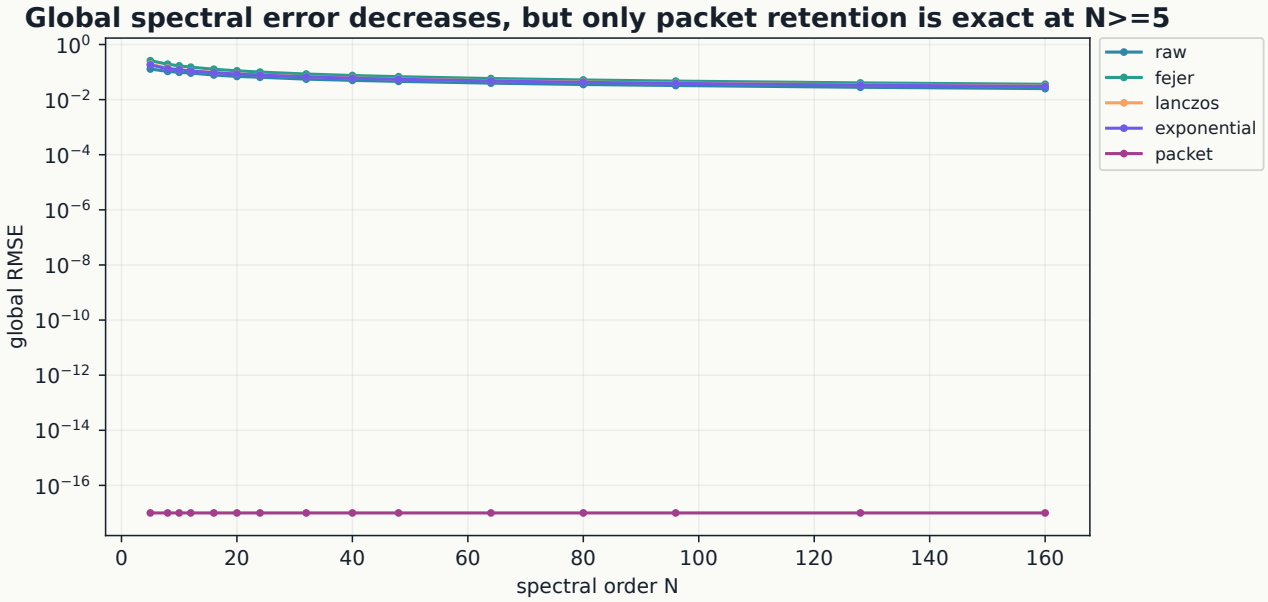


Figure 5.11: Global RMSE as the spectral order grows.

What the figure shows. For this finite-band background, the packet reconstruction reaches floating-point zero at $N = 5$.

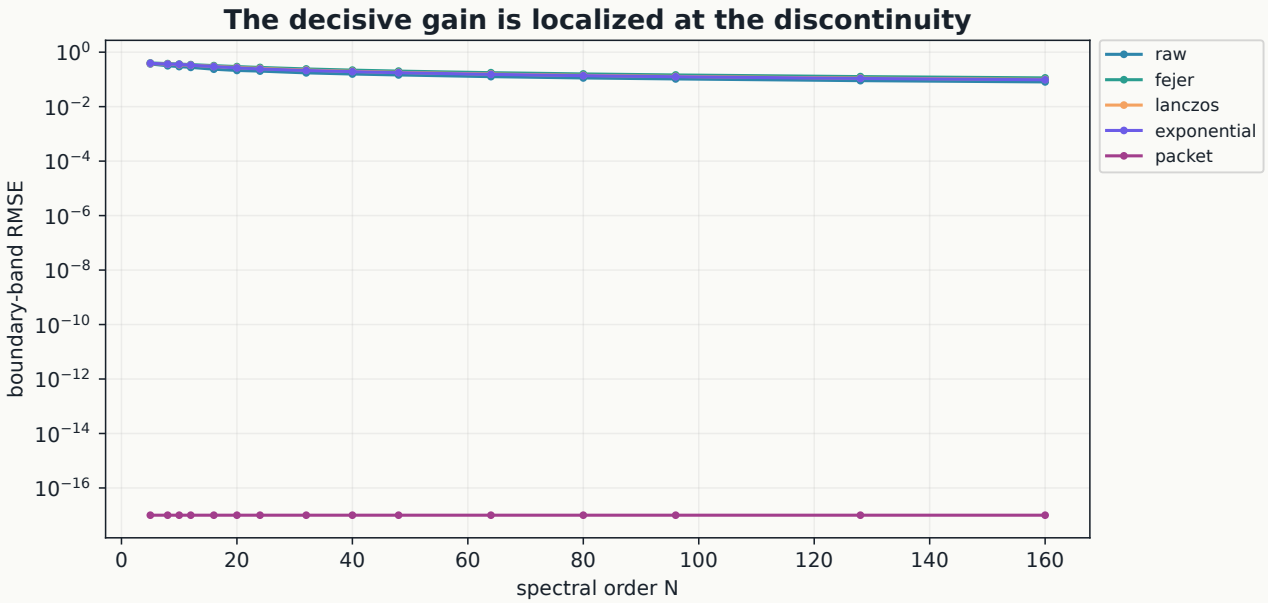


Figure 5.12: Boundary-band RMSE.

What the figure shows. This metric isolates the computational region in which the coordinate choice matters most.

6 The resolved operator

Part III: The packet reconstruction
The singular piece now receives its own coordinate. Fourier truncation acts on the smooth packet, while the edge packet passes through unchanged.

6.1 Why the global operator is the wrong block assignment

Global truncation applies S_N to the complete signal:

$$S_N f = S_N P_{\mathrm{s}} f + S_N P_{\mathrm{j}} f.$$

The first block is exact for $N \geq 5$. The second block is the Gibbs-producing approximation of the discontinuity.

The calculus permits different transfer rules on different packets. Define

$$\boxed{\mathcal{R}_N = S_N P_{\mathrm{s}} + P_{\mathrm{j}}.}$$

The smooth packet is handled spectrally. The edge packet is retained in its native geometric coordinate.

Theorem 6.1 (Exact Gibbs-bypassing reconstruction)**.** *For every $g \in \mathcal{M}_5$ and every $N \geq 5$,*

$$\boxed{\mathcal{R}_N g = g.}$$

Proof. Write $g = t + cj$ with $t \in \mathcal{T}_5$. Then

$$\mathcal{R}_N g = S_N P_{\mathrm{s}}(t + cj) + P_{\mathrm{j}}(t + cj) = S_N t + cj.$$

Since t has degree at most five and $N \geq 5$, $S_N t = t$. Hence $\mathcal{R}_N g = t + cj = g$. □

Corollary 6.2 (Exact one-sided values)**.** *For the worked signal and $N \geq 5$,*

$$(\mathcal{R}_N f)(a^-) = s(a) + 1, \qquad (\mathcal{R}_N f)(a^+) = s(a).$$

No jump-induced overshoot, undershoot, or transition blur is introduced.

6.2 Block matrix

Relative to

$$\mathcal{M}_5 = \mathcal{T}_5 \oplus \mathcal{J},$$

the operator is

$$\mathcal{R}_N = \begin{pmatrix} S_N|_{\mathcal{T}_5} & 0 \\ 0 & I_{\mathcal{J}} \end{pmatrix}.$$

For $N \geq 5$ this becomes

$$\mathcal{R}_N = \begin{pmatrix} I_{\mathcal{T}_5} & 0 \\ 0 & I_{\mathcal{J}} \end{pmatrix} = I_{\mathcal{M}_5}.$$

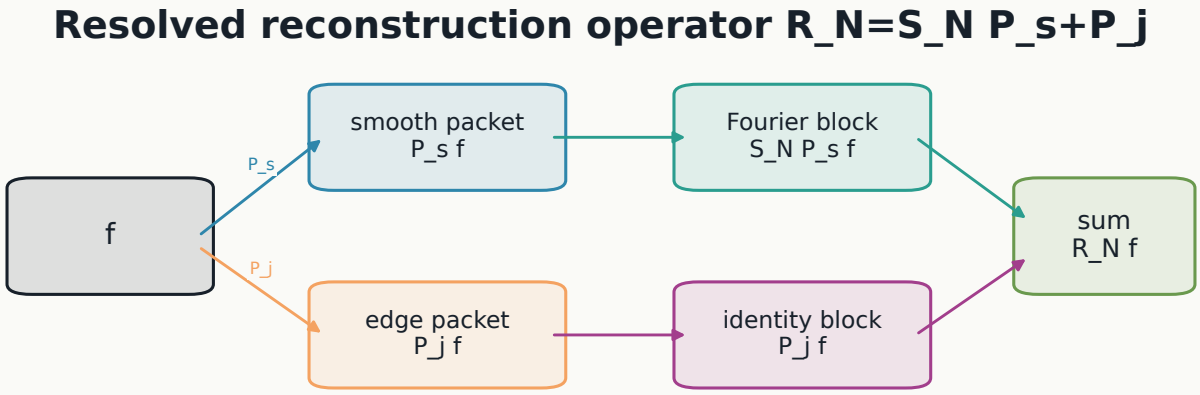


Figure 6.1: Analysis, packet-specific operation, and synthesis.

What the figure shows. The change is not a stronger Fourier filter. The two packets are given different operators because they have different structure.

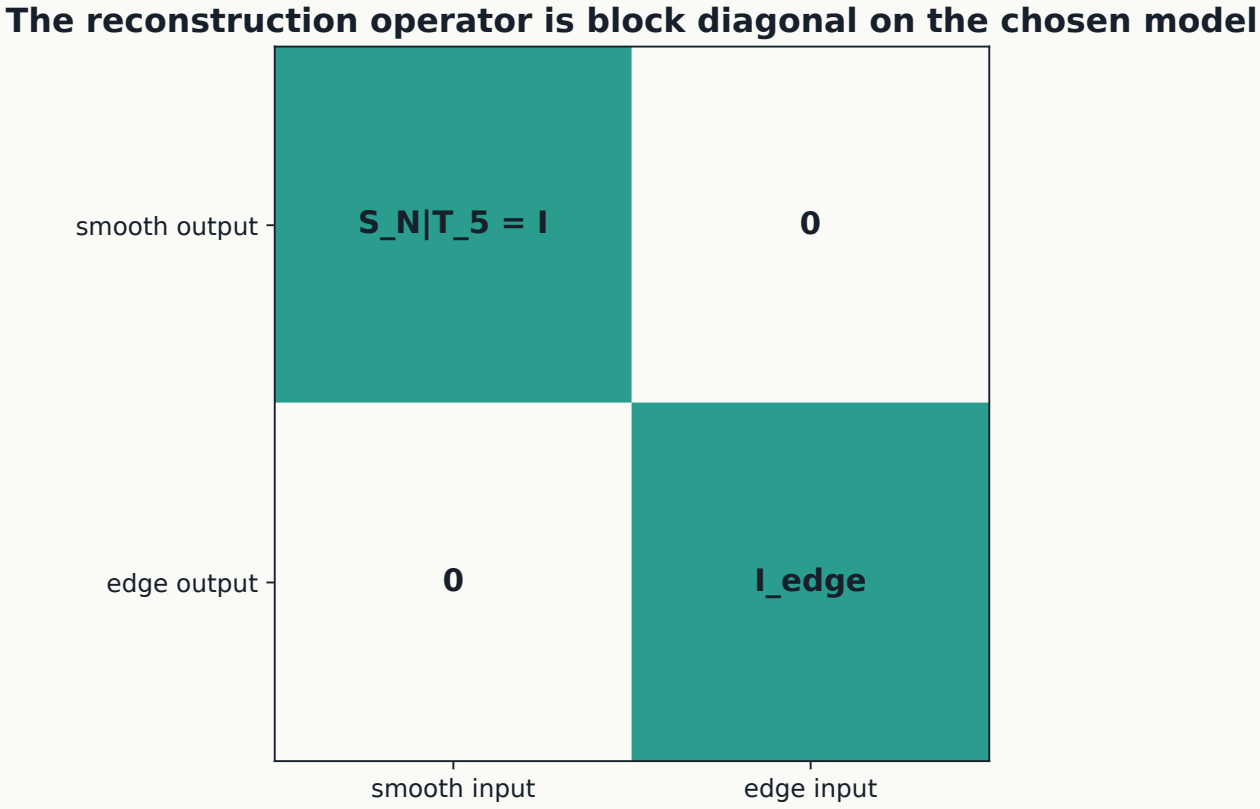


Figure 6.2: Block matrix of the resolved reconstruction operator.

What the figure shows. At sufficient smooth resolution, both diagonal blocks are identities and both cross blocks vanish.

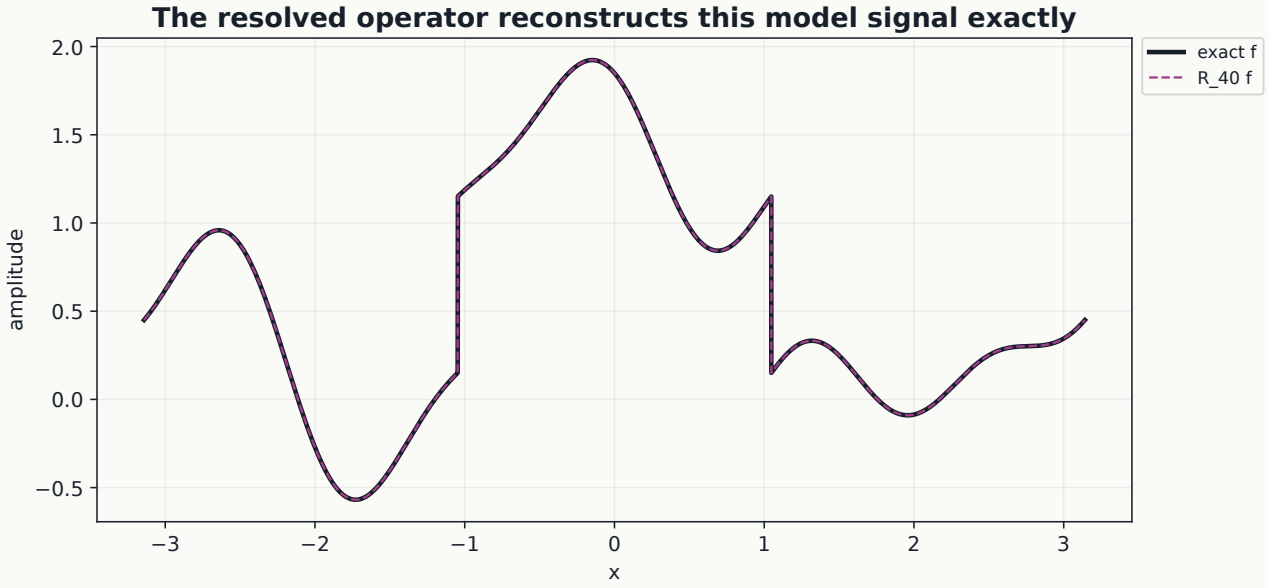


Figure 6.3: Exact resolved reconstruction of the continuous worked signal.

What the figure shows. The dashed reconstruction is hidden beneath the exact curve.

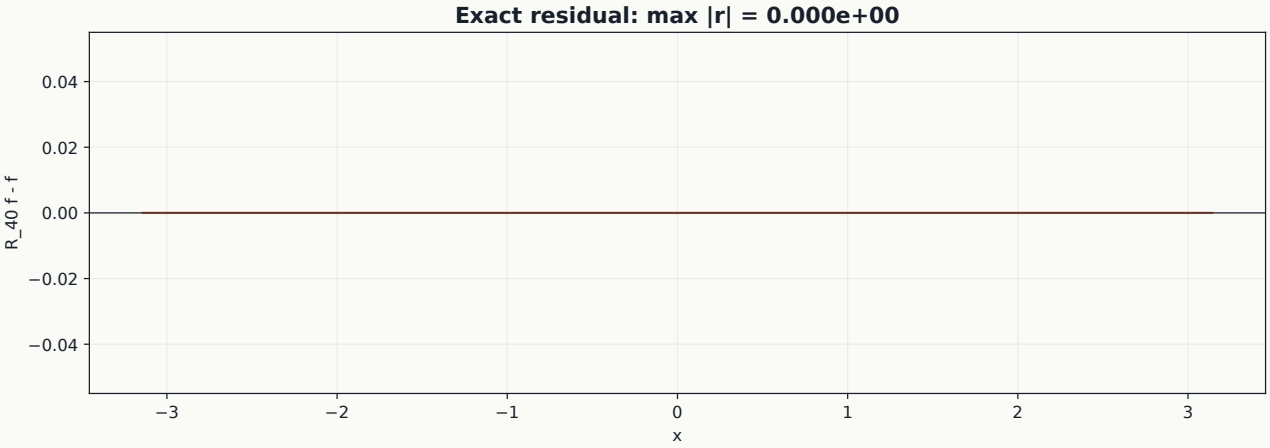


Figure 6.4: Pointwise residual of the resolved reconstruction.

What the figure shows. The nonzero scale is only floating-point evaluation noise.

7 Extension beyond finite-band smooth packets

7.1 Analytic smooth background

The smooth part of the worked signal is finite-band, so the identity appears at a finite order. The same argument applies to any smooth packet s for which a Fourier error estimate is available. Let

$$f = s + cj, \qquad \mathcal{R}_N f = S_N s + cj.$$

Then the exact identity is

$$\boxed{f - \mathcal{R}_N f = s - S_N s.}$$

The jump contributes no approximation error at all.

Proposition 7.1 (Uniform analytic-tail bound)**.** *Suppose*

$$|\widehat{s}(n)| \le Ce^{-\rho|n|}$$

for constants $C, \rho > 0$. Then

$$\|f - \mathcal{R}_N f\|_\infty \le 2C \sum_{n>N} e^{-\rho n} = \boxed{\frac{2Ce^{-\rho(N+1)}}{1 - e^{-\rho}}}.$$

Proof. The edge packet cancels exactly. Apply the triangle inequality to the omitted Fourier tail of s. □

Remark 7.2. The bound is uniform across the edge. The smooth approximation remains uniformly convergent because the discontinuity is no longer part of the packet being truncated.

8 Why filtering is different

8.1 Fejer averaging

The Fejer sum is

$$\sigma_N g = \frac{1}{N+1} \sum_{m=0}^{N} S_m g.$$

Its kernel is nonnegative and integrates to 2π. Hence for $0 \le j \le 1$,

$$0 \le \sigma_N j \le 1.$$

It eliminates overshoot of the jump component, but it replaces the exact discontinuity by a transition layer.

8.2 Lanczos and exponential filters

Lanczos and exponential spectral weights suppress high-frequency coefficients. They reduce sidelobes but also modify low-order information unless designed to preserve selected modes. The essential tradeoff is between oscillation and edge width.

8.3 Packet retention changes the approximation problem

Filtering asks how to approximate the whole signal more gently in the same global Fourier coordinate. Packet retention asks which components should be approximated in Fourier coordinates at all. The second question can remove the singular packet from the truncation error instead of redistributing its error.

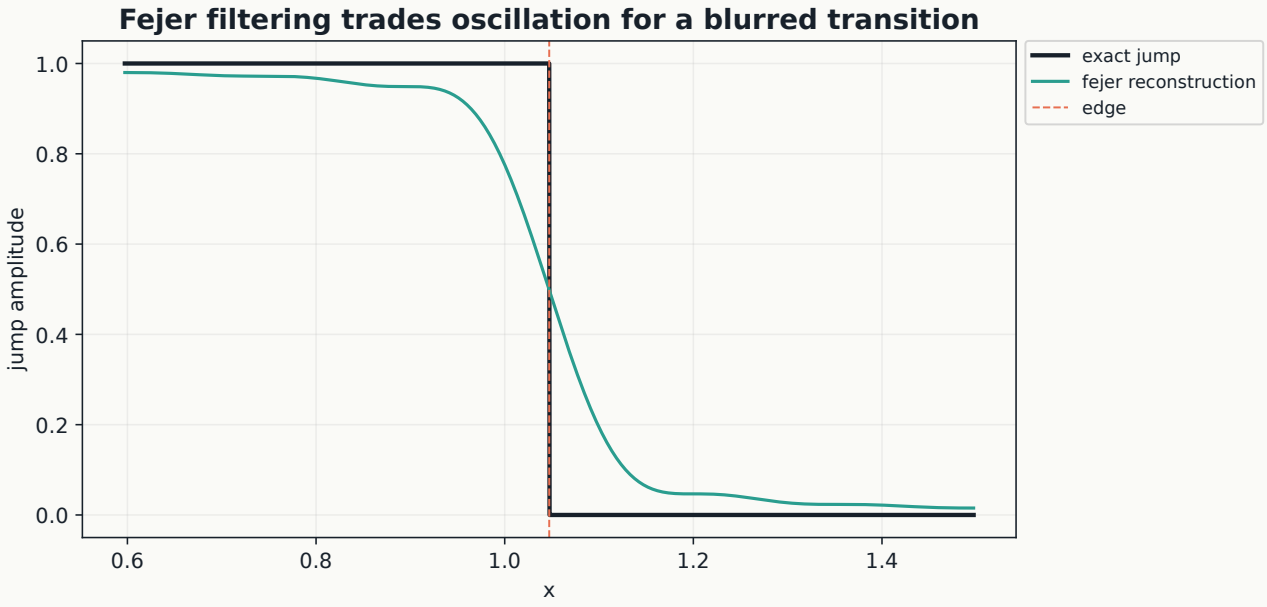


Figure 8.1: Fejer reconstruction of the jump packet.

What the figure shows. Positivity prevents overshoot, but the discontinuity is replaced by a finite transition width.

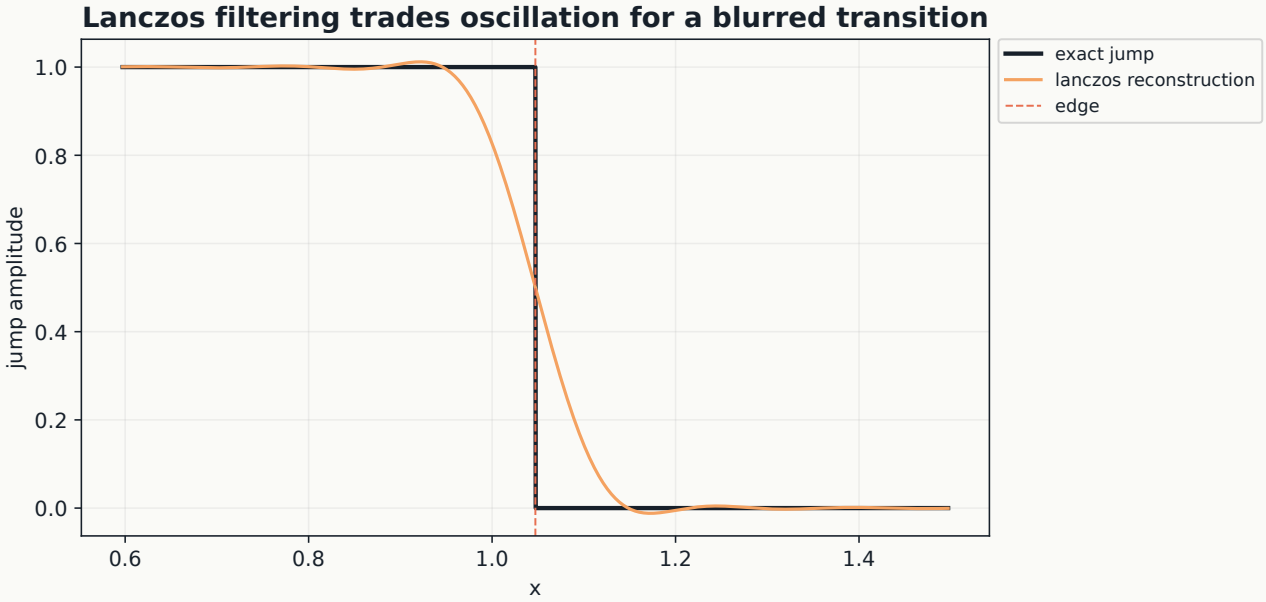


Figure 8.2: Lanczos-filtered jump reconstruction.

What the figure shows. Ringing is reduced but not removed exactly, and the edge remains blurred.

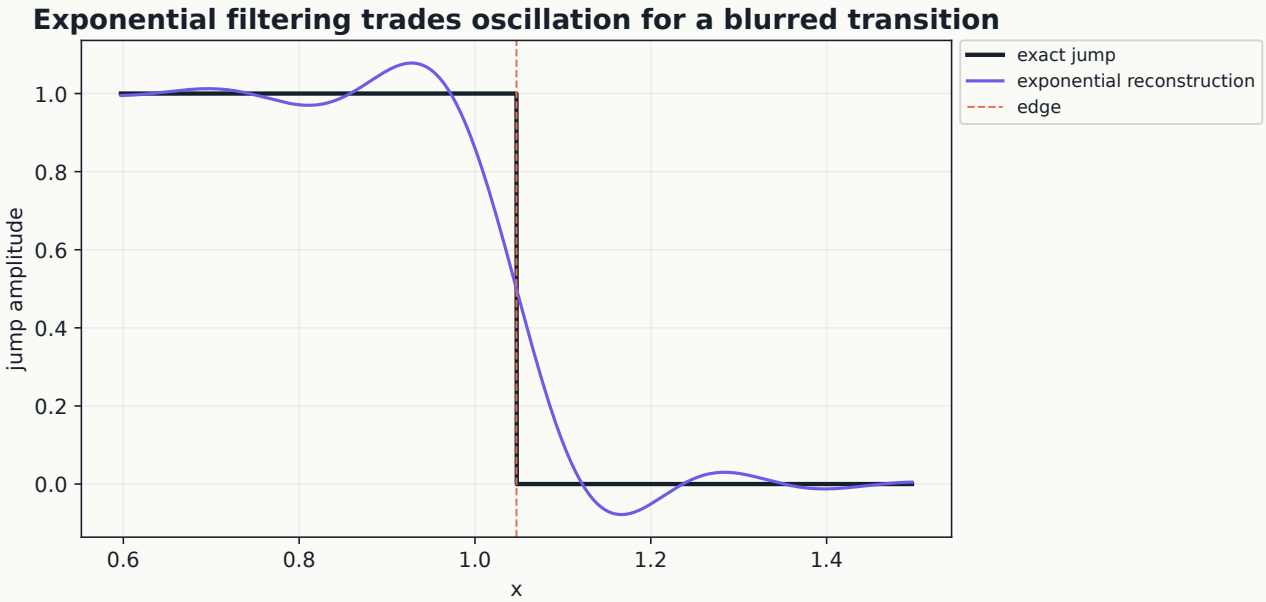


Figure 8.3: Exponentially filtered jump reconstruction.

What the figure shows. Strong high-mode damping controls oscillations by suppressing the very coefficients needed to sharpen the edge.

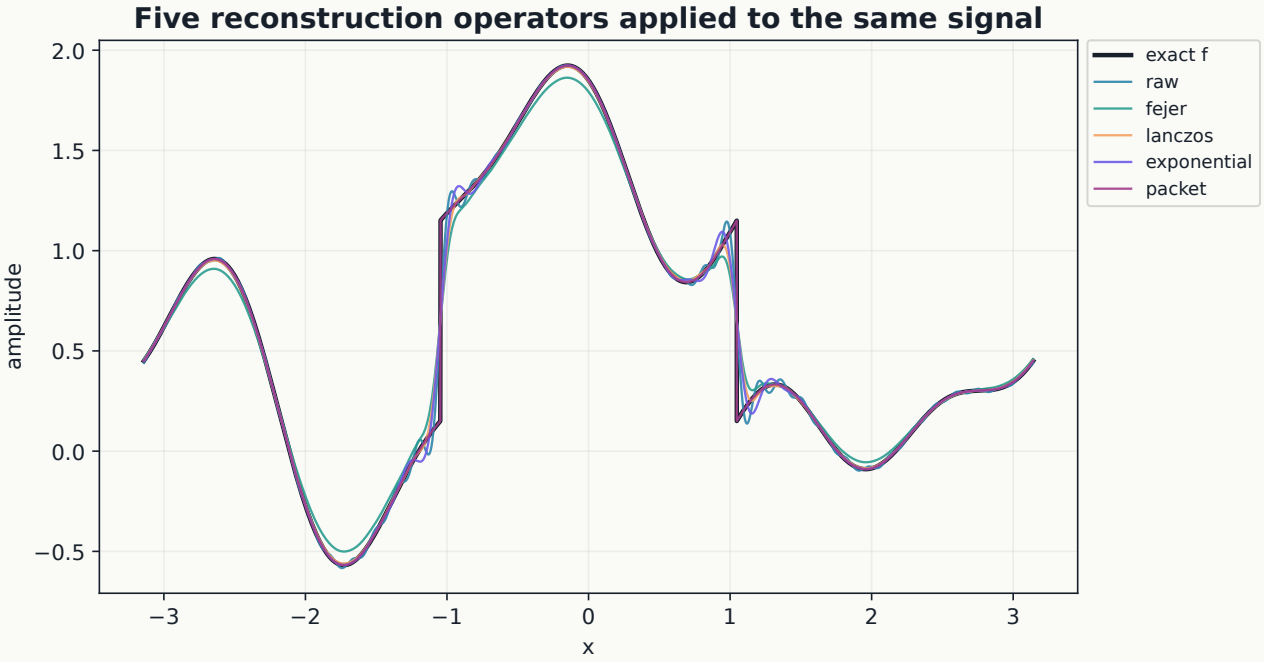


Figure 8.4: Full-signal comparison at $N = 40$.

What the figure shows. Away from the edge all methods are visually similar. The decisive differences occur in a narrow boundary region.

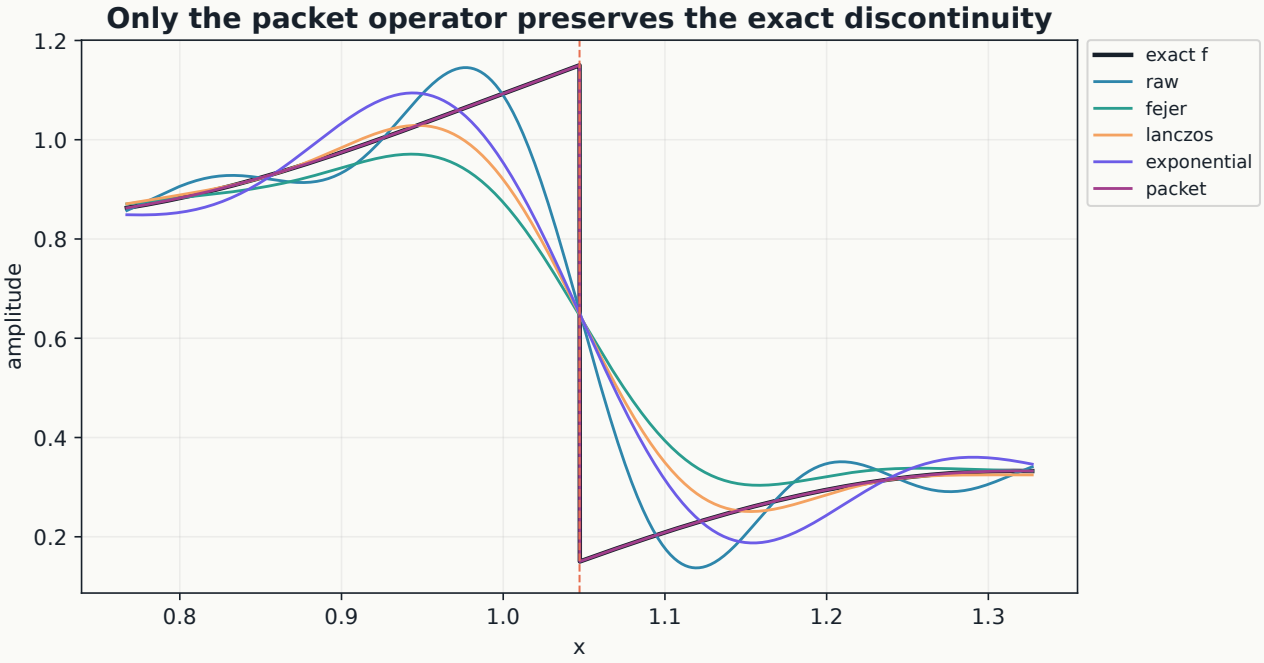


Figure 8.5: Right-edge comparison at $N = 40$.

What the figure shows. The packet method preserves both one-sided values and the exact edge location.

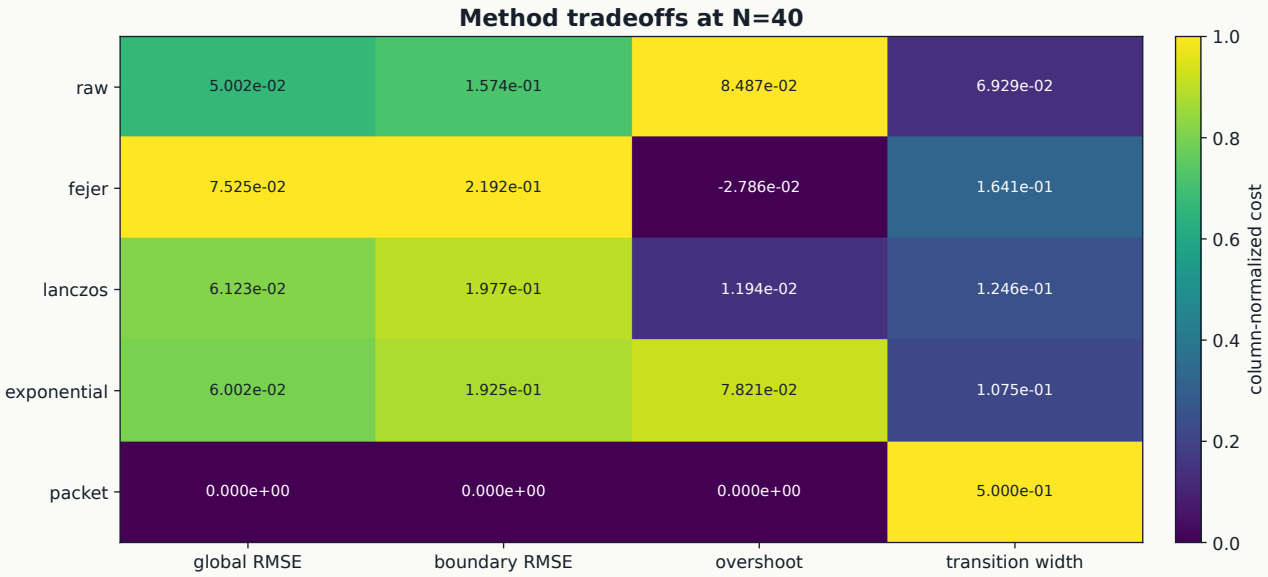


Figure 8.6: Four complementary error measures at $N = 40$.

What the figure shows. No scalar metric alone captures overshoot, blur, global error, and boundary error.

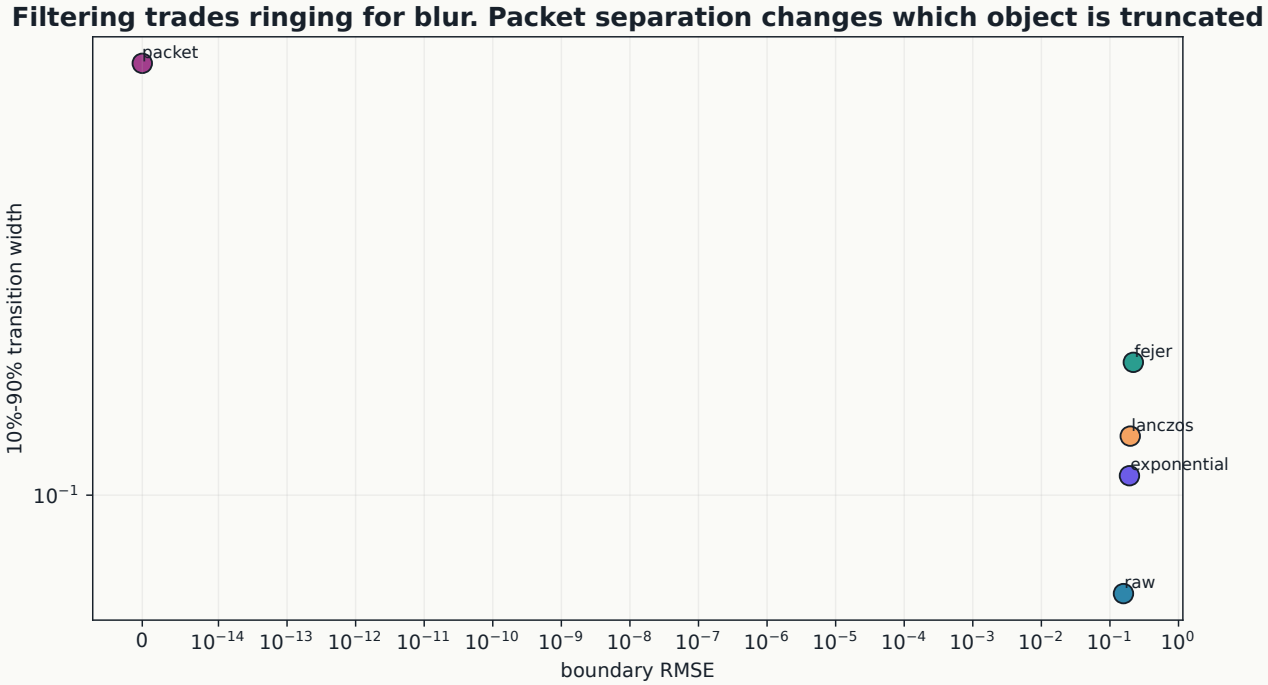


Figure 8.7: Boundary error versus transition width.

What the figure shows. Filtering moves along a blur-error curve. Exact packet retention changes the coordinate model and occupies the origin for this worked class.

9 The finite observation model

Part IV: Recovering the packet from samples
The edge is no longer assumed known. It is recovered from a finite grid by searching a stated interval library and fitting the smooth background in closed form.

9.1 The sample grid

Take

$$M = 96, \qquad x_m = -\pi + \frac{2\pi m}{M}, \qquad 0 \le m < M.$$

Because M is divisible by three, the edges $\pm\pi/3$ align with the sample grid. The interval begins at sample

$$r_* = 32$$

and has length

$$\ell_* = 32.$$

Let

$$q_{r,\ell}[m] = \mathbf{1}_{\{r,r+1,\dots,r+\ell-1\} \bmod M}(m)$$

be the cyclic interval mask. The true mask is

$$q_* = q_{32,32}.$$

The observed vector is

$$y = B\theta_* + q_*.$$

9.2 The smooth design matrix

Let $B \in \mathbb{R}^{M\times 11}$ contain the sampled basis

$$1, \cos x, \sin x, \dots, \cos 5x, \sin 5x.$$

Let Q be any orthonormal basis for $\mathrm{Ran}(B)$ and set

$$P_B = QQ^{\mathsf{T}}.$$

The residual projector is

$$P_B^{\perp} = I - P_B.$$

Since the smooth seed lies in $\mathrm{Ran}(B)$,

$$P_B^{\perp} y = P_B^{\perp} q_*.$$

This removes every unknown smooth coefficient before interval selection.

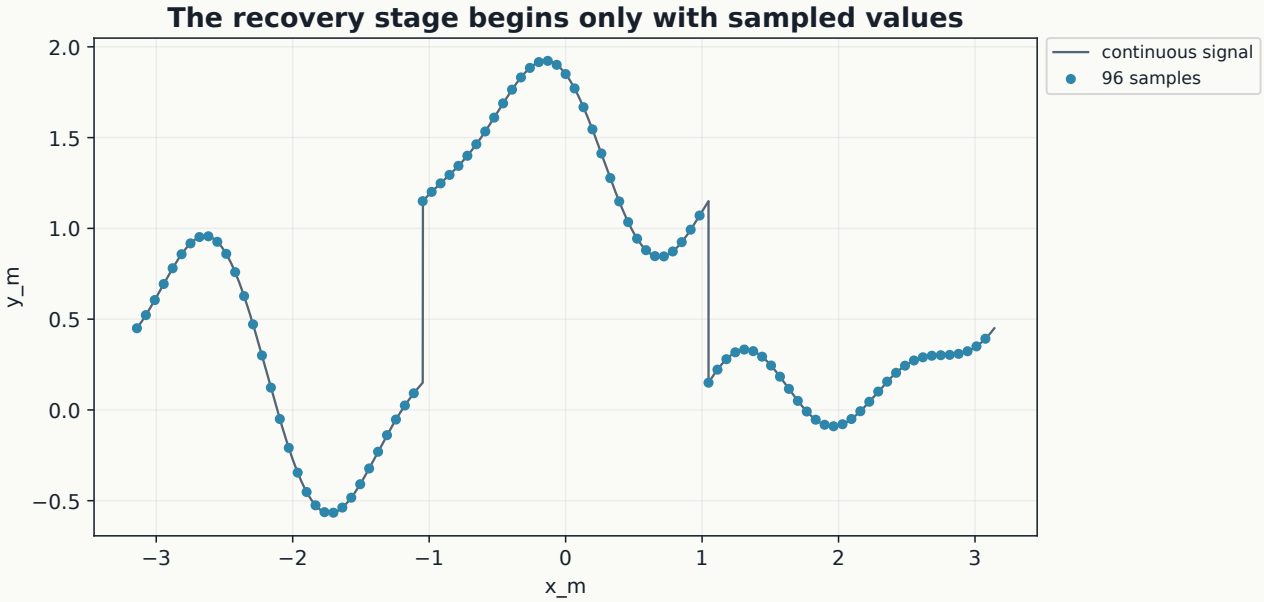


Figure 9.1: The continuous signal and its 96 observed values.

What the figure shows. The recovery algorithm receives the samples, the smooth degree, and the allowed interval library. It does not receive the true start or length.

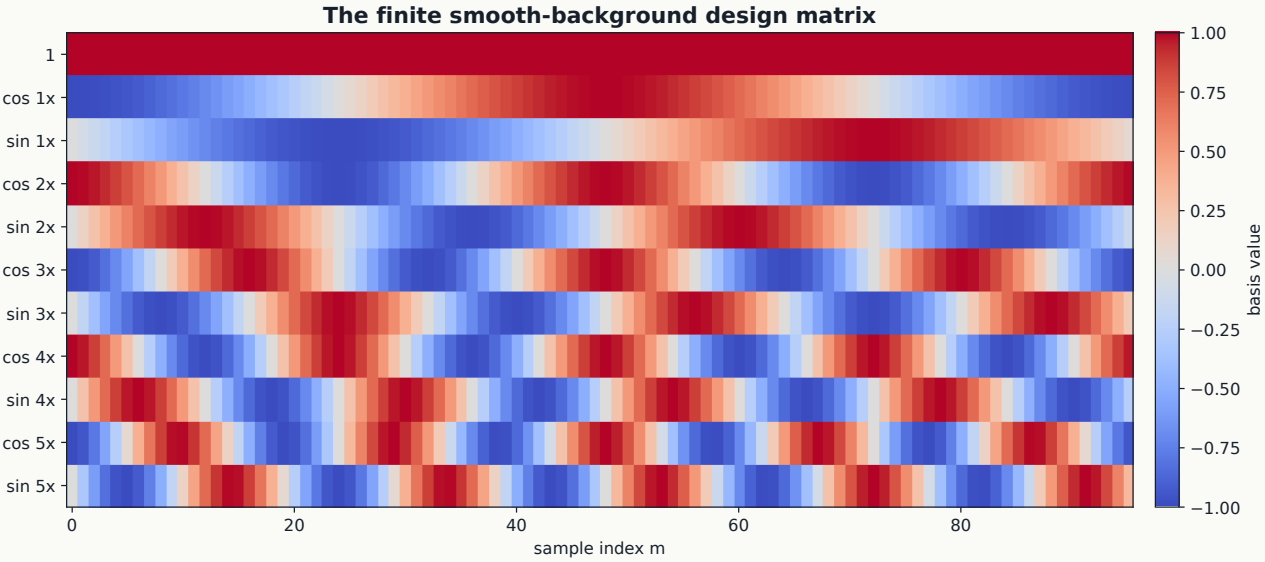


Figure 9.2: The complete degree-five sampled trigonometric design.

What the figure shows. The matrix has eleven columns: one constant and sine-cosine pairs through degree five.

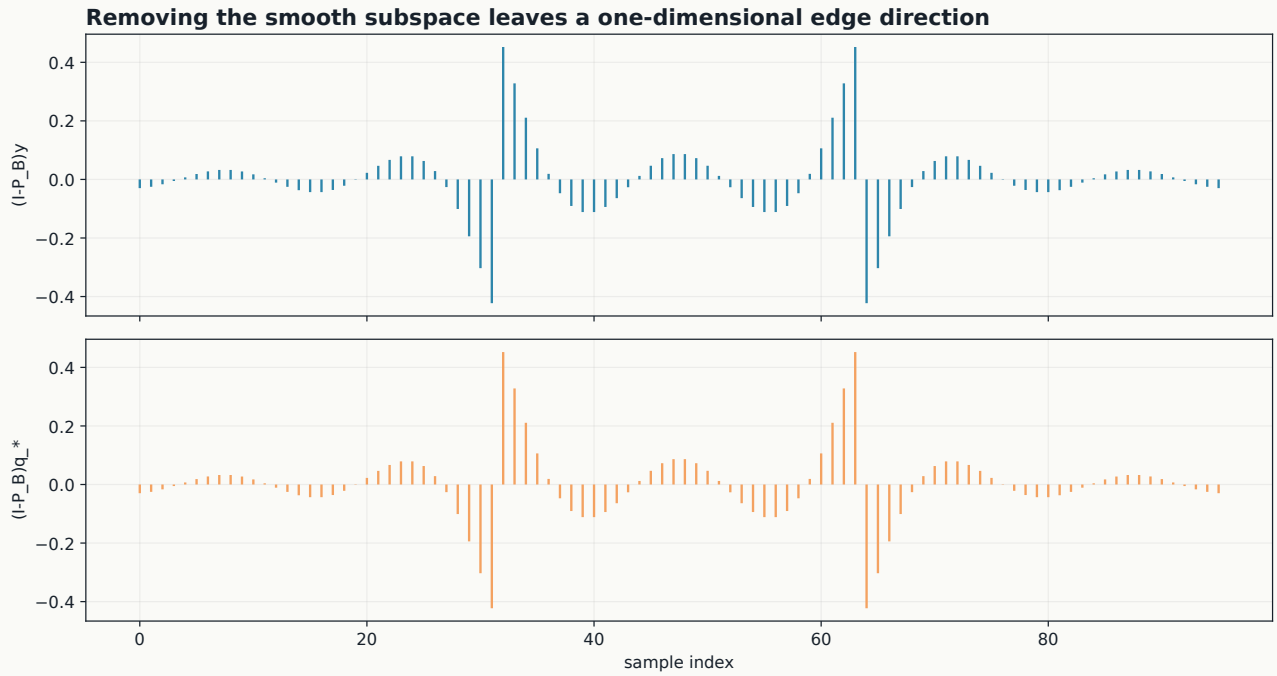


Figure 9.3: Residualizing the data and the true mask against the smooth subspace.

What the figure shows. In the exact model, the two displayed vectors are identical.

10 Profile least squares over interval packets

10.1 Eliminating the smooth coefficients

For a candidate interval q, consider

$$y = B\theta + Aq.$$

Projecting by $P_B^\perp$ gives

$$y^\perp = Aq^\perp, \qquad y^\perp = P_B^\perp y, \qquad q^\perp = P_B^\perp q.$$

For fixed q with $q^\perp \neq 0$, the least-squares amplitude is

$$\boxed{\widehat{A}(q) = \frac{\left\langle q^\perp, y^\perp \right\rangle}{\|q^\perp\|_2^2}.}$$

The profiled objective is

$$\boxed{J(q) = \left\| y^\perp - \widehat{A}(q) q^\perp \right\|_2^2.}$$

Theorem 10.1 (Finite exact interval recovery)**.** *Assume the data satisfy*

$$y = B\theta_* + A_* q_*, \qquad A_* > 0,$$

and assume no other admissible candidate q has $q^\perp$ equal to a positive scalar multiple of $q_*^\perp$. Then q_* is the unique minimizer of J among positive-amplitude candidates, and

$$J(q_*) = 0.$$

Proof. Residualization gives $y^\perp = A_* q_*^\perp$. Substituting q_* into the amplitude formula returns A_*, so $J(q_*) = 0$. If another candidate had zero objective, then $y^\perp = \widehat{A}(q) q^\perp$ with $\widehat{A}(q) > 0$, so $q^\perp$ would be a positive scalar multiple of $q_*^\perp$, contrary to the hypothesis. □

10.2 The finite identifiability gap

The program enumerates all positive cyclic intervals with lengths between eight and forty-eight samples. There are 3936 candidates after excluding residualized zero directions. The exact minimizer is

$$(r, \ell, A) = (32, 32, 1).$$

Its computed objective is zero after nonnegativity clipping. The second-best objective is

$$0.7531607493634364.$$

Thus the finite gap is

$$\boxed{\Gamma = 0.7531607493634364.}$$

Corollary 10.2 (Objective perturbation stability)**.** *If a perturbed objective $\widetilde{J}$ satisfies*

$$\sup_q |\widetilde{J}(q) - J(q)| < \Gamma/2,$$

then the exact minimizer remains unique.

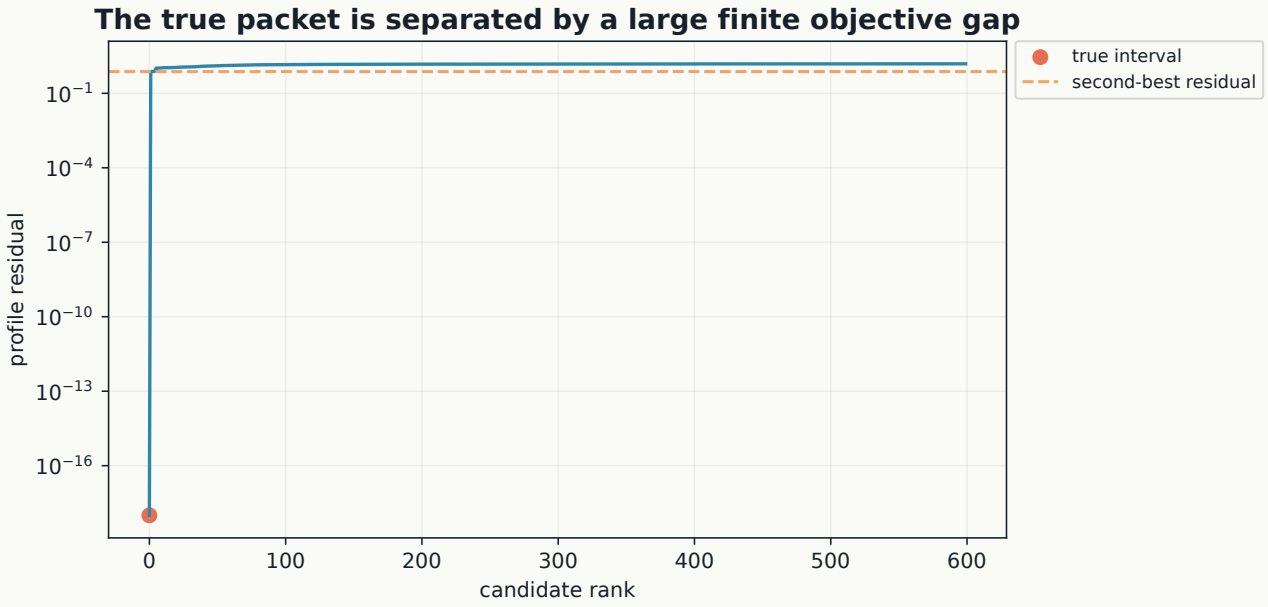


Figure 10.1: The ranked profile objective values.

What the figure shows. The true interval is separated from every other admissible packet by a large finite gap.

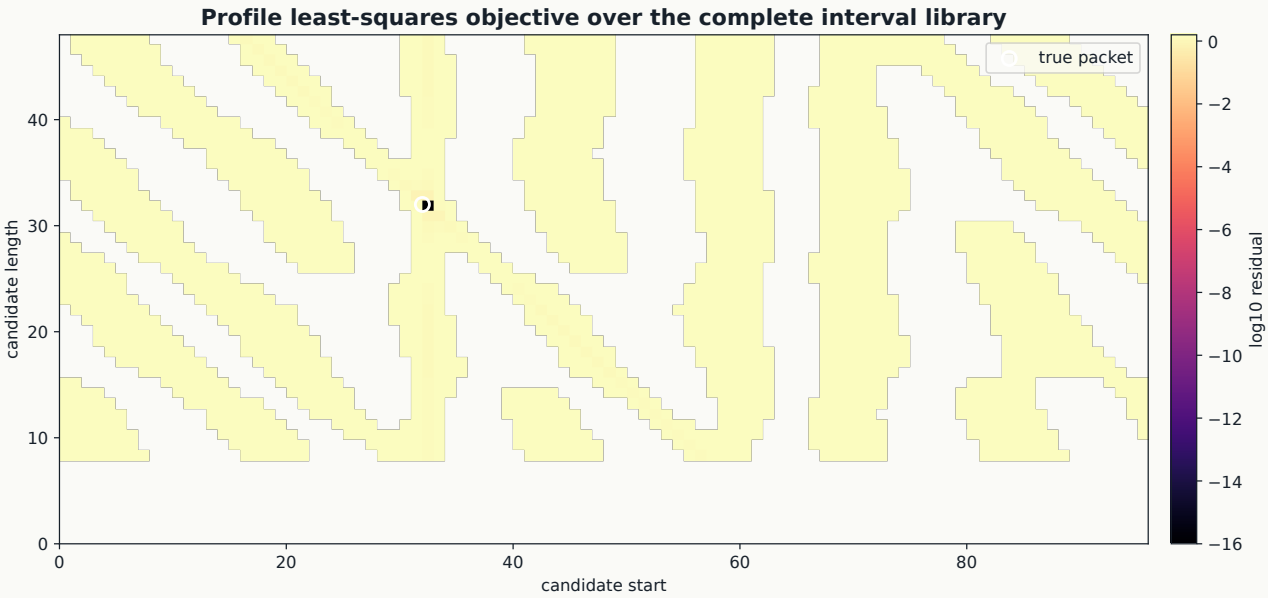


Figure 10.2: The complete start-length objective surface.

What the figure shows. The white ring marks $(32, 32)$. The logarithmic scale makes the zero residual visible against the nonzero field.

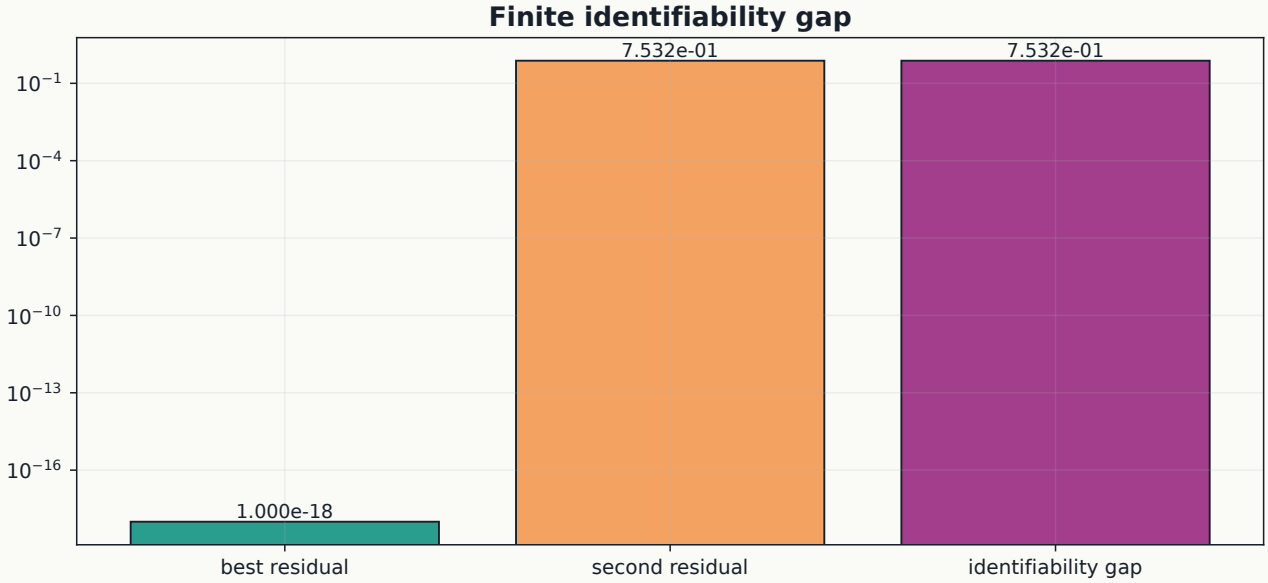


Figure 10.3: Best residual, second-best residual, and their gap.

What the figure shows. The gap shows how far the best wrong interval lies above the true one.

11 Reconstruction after packet recovery

11.1 Recovered smooth packet

After finding $(\widehat{q}, \widehat{A})$, reconstruct the smooth component by

$$\widehat{s} = P_B(y - \widehat{A}\widehat{q}).$$

Then

$$\widehat{y} = \widehat{s} + \widehat{A}\widehat{q}.$$

For the worked data,

$$\widehat{q} = q_*, \qquad \widehat{A} = 1, \qquad \widehat{s} = s|_{\{x_m\}}.$$

The maximum sample residual is

$$\boxed{5.551115123125783 \times 10^{-16}.}$$

11.2 Recovered coefficients

The recovered degree-five coefficient vector agrees with the original coefficients to floating-point precision. In the column order

$$1, \cos x, \sin x, \ldots, \cos 5x, \sin 5x,$$

the only nonzero entries are

$$\frac{1}{4}, \qquad \frac{2}{5} \text{ at } \cos 2x, \qquad -\frac{1}{3} \text{ at } \sin 3x, \qquad \frac{1}{5} \text{ at } \cos 5x.$$

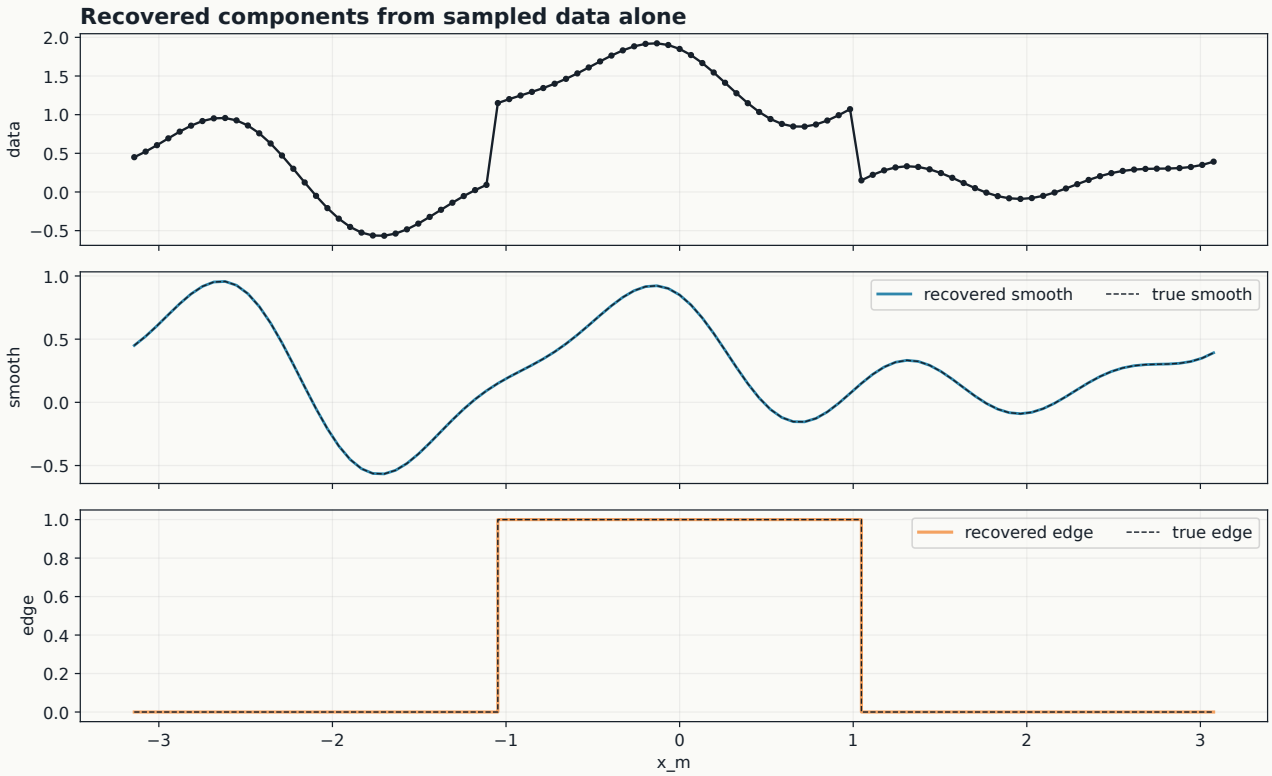


Figure 11.1: Data, recovered smooth packet, and recovered edge packet.

What the figure shows. The fit returns the smooth part and the interval part separately, rather than only matching their sum.

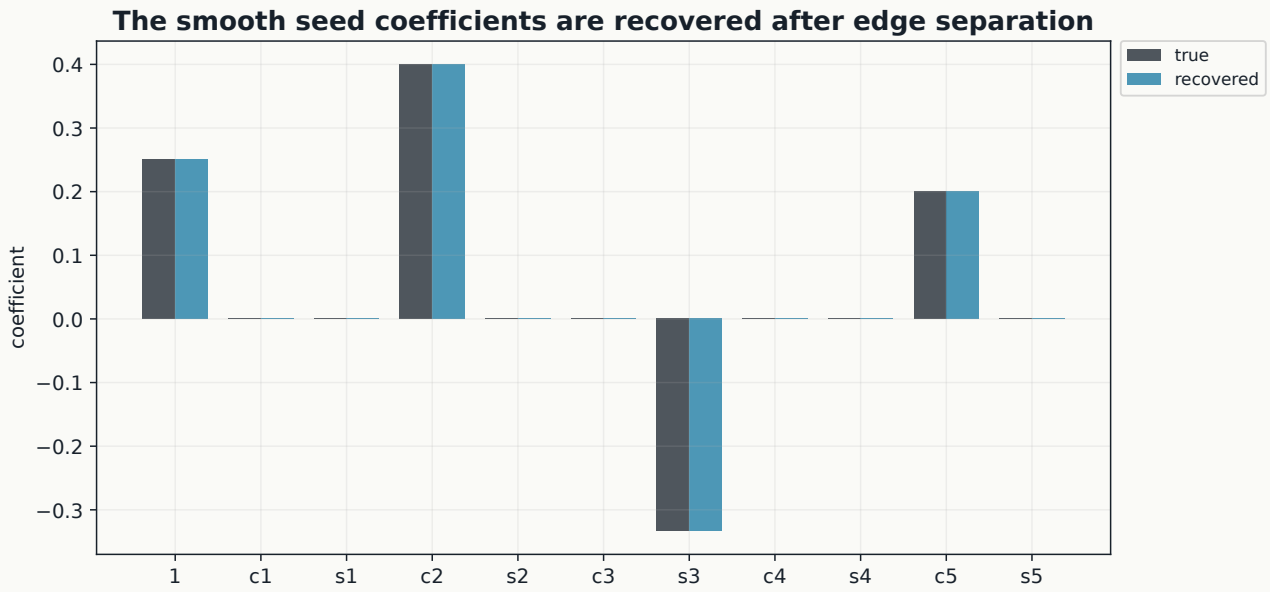


Figure 11.2: True and recovered smooth coefficients.

What the figure shows. Every harmonic coefficient outside the chosen degree-five model remains at numerical zero.

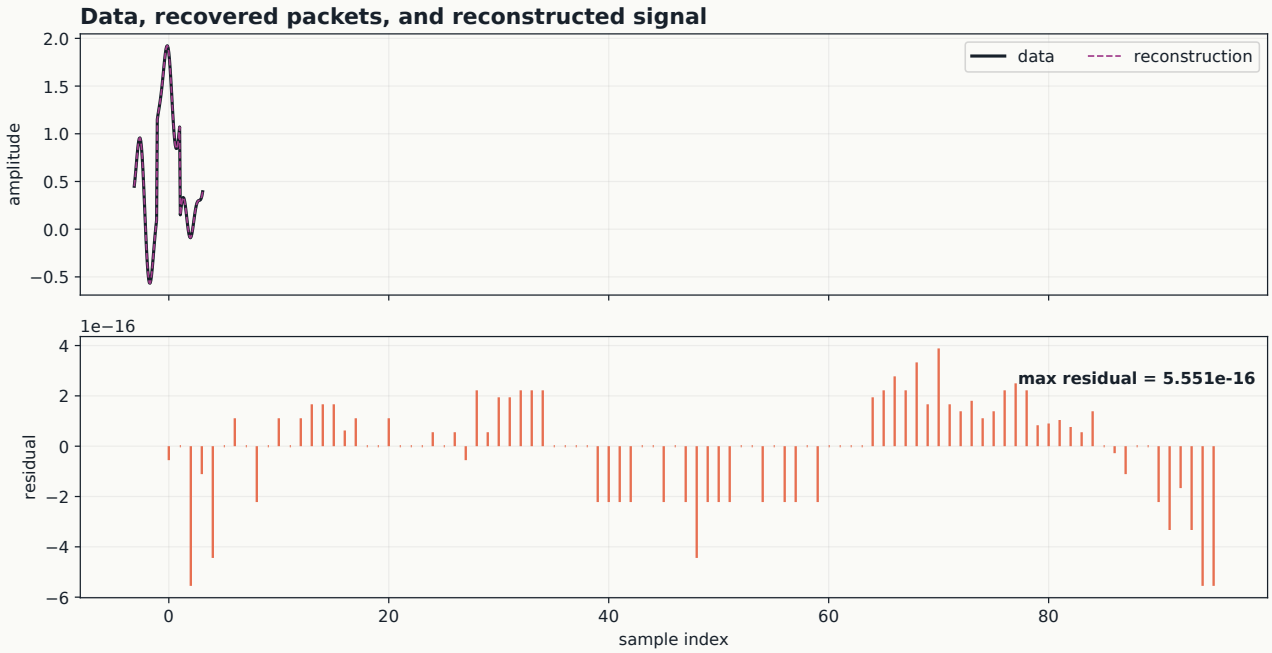


Figure 11.3: Sample-space synthesis and residual.

What the figure shows. This closes the loop from observed values to packet selection to exact reconstruction.

12 The single executable algorithm

Algorithm 12.1 (Finite Gibbs packet solver)**.** *Given M, degree K, minimum and maximum interval lengths, and samples y:*

1. *Build the complete trigonometric design B_K.*
2. *Compute an orthonormal basis Q of* $\operatorname{Ran}(B_K)$.
3. *Form $y^{\perp} = y - QQ^{\mathsf{T}}y$.*
4. *Enumerate every admissible cyclic interval mask q.*
5. *Residualize $q^{\perp} = q - QQ^{\mathsf{T}}q$.*
6. *Compute $\widehat{A}(q) = \left\langle q^{\perp}, y^{\perp} \right\rangle / \left\| q^{\perp} \right\|^2$.*
7. *Reject nonpositive amplitudes.*
8. *Minimize $J(q) = \left\| y^{\perp} - \widehat{A}(q) q^{\perp} \right\|^2$.*
9. *Reconstruct $\widehat{s} = QQ^{\mathsf{T}}(y - \widehat{A}\widehat{q})$.*
10. *Return $\widehat{y} = \widehat{s} + \widehat{A}\widehat{q}$, the objective gap, and the residual.*

Remark 12.2. The exhaustive interval library is used because every candidate can be inspected. Larger problems could use dynamic programming, multiscale search, change-point methods, or branch-and-bound without changing the analysis and synthesis formulas.

13 What happens when the packet is wrong

Part V: Perturbation and residual-driven refinement
Wrong support, wrong amplitude, noise, and off-grid boundaries are treated separately. Each failure leaves a residual that points back to the part of the model that needs attention.

13.1 One-sample mislocation

Shift the recovered interval by one sample while keeping its length and amplitude fixed. The smooth packet remains correct, but the edge packet disagrees at exactly two sample addresses. The residual is therefore supported only at the two boundaries.

Proposition 13.1 (Discrete packet perturbation identity)**.** *Let $q, \widetilde{q} \in \{0,1\}^M$ be masks and let $A, \widetilde{A} \in \mathbb{R}$. Then*

$$\begin{aligned}\left\|Aq - \widetilde{A}\widetilde{q}\right\|_2^2 = (A-\widetilde{A})^2 |\operatorname{supp} q \cap \operatorname{supp} \widetilde{q}| \\ + A^2 |\operatorname{supp} q \setminus \operatorname{supp} \widetilde{q}| \\ + \widetilde{A}^2 |\operatorname{supp} \widetilde{q} \setminus \operatorname{supp} q|.\end{aligned}$$

In particular, if $A = \widetilde{A} = 1$,

$$\boxed{\|q - \widetilde{q}\|_2^2 = |\operatorname{supp} q \triangle \operatorname{supp} \widetilde{q}|.}$$

Proof. Partition the sample addresses into the intersection, the two one-sided differences, and the common complement. The summand is respectively $(A-\widetilde{A})^2$, A^2, $\widetilde{A}^2$, and zero. □

13.2 Amplitude mismatch

If the support is correct but the amplitude is $\widetilde{A}$, then

$$\left\|Aq - \widetilde{A}q\right\|_2 = |A - \widetilde{A}| \sqrt{|\operatorname{supp} q|}.$$

The error remains inside the packet. It does not produce global ringing.

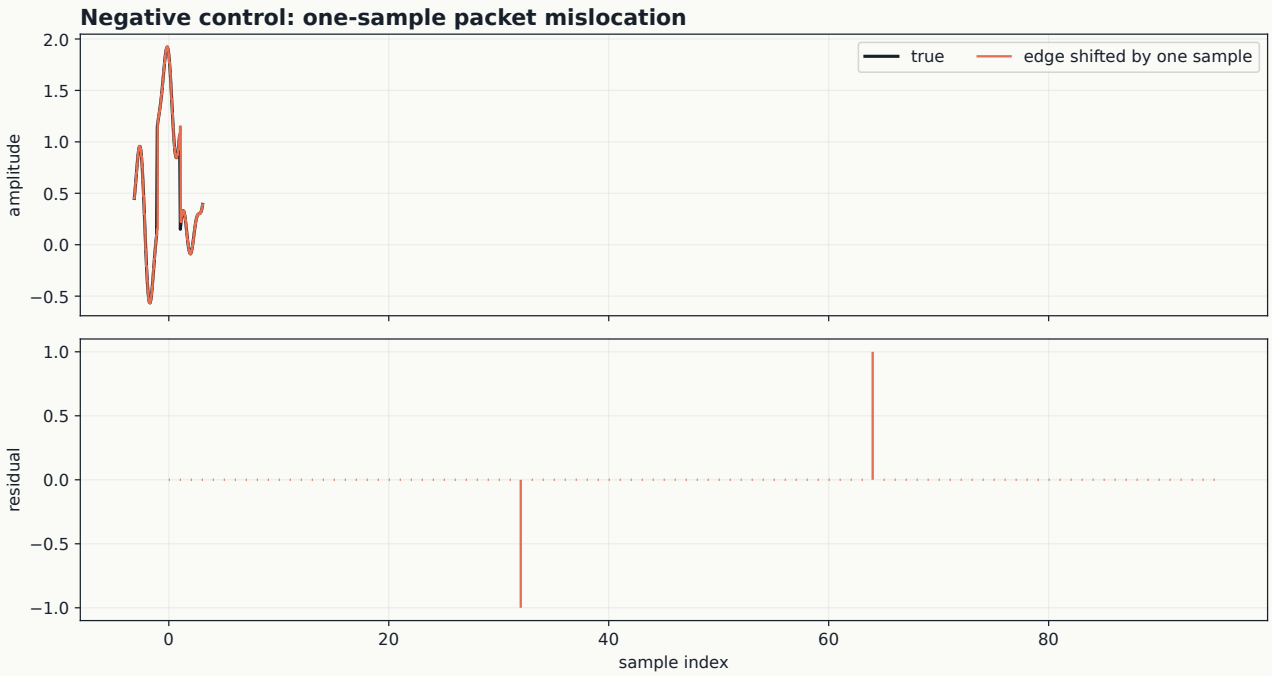


Figure 13.1: A one-sample edge shift and its residual.

What the figure shows. The failure is not spread across the domain. It identifies the addresses at which the geometric packet is wrong.

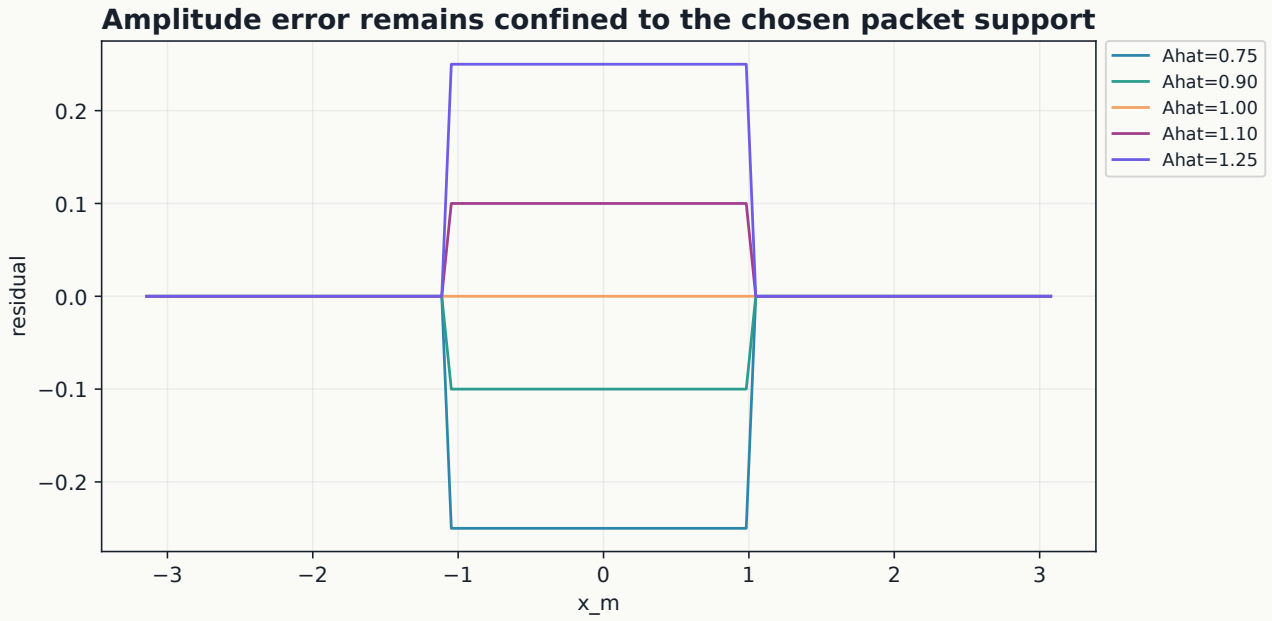


Figure 13.2: Residuals for several amplitude errors.

What the figure shows. Correct support and incorrect amplitude produce a uniform packet-local residual.

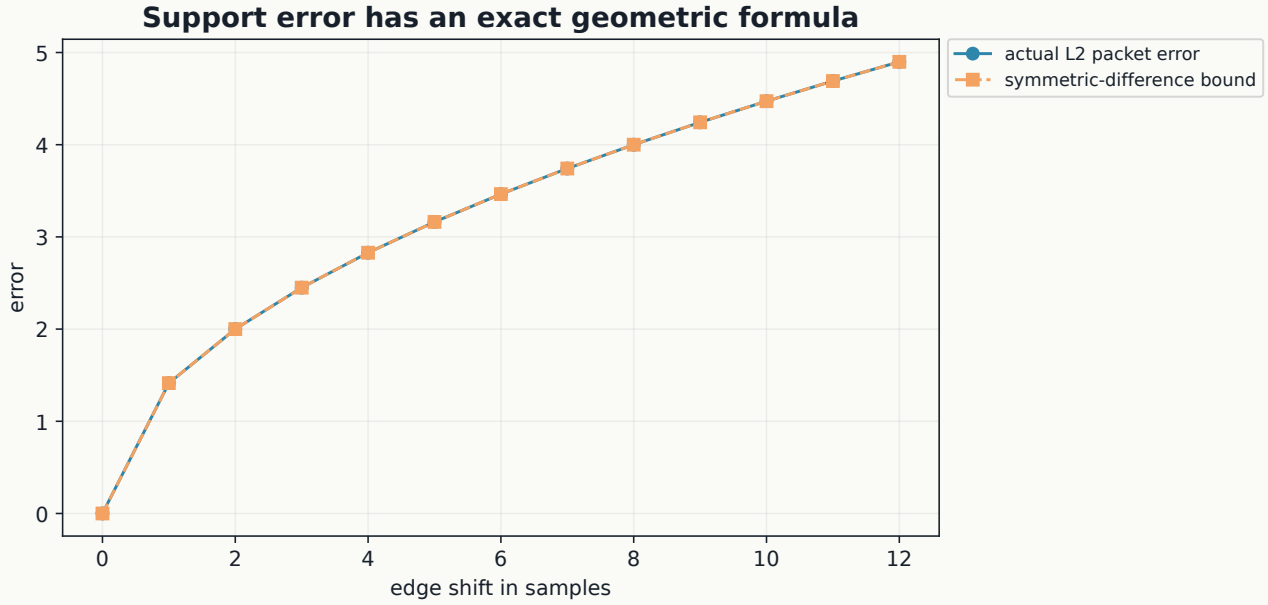


Figure 13.3: Exact support-error formula for shifted masks.

What the figure shows. For equal amplitudes, the squared Euclidean error equals the symmetric-difference count exactly.

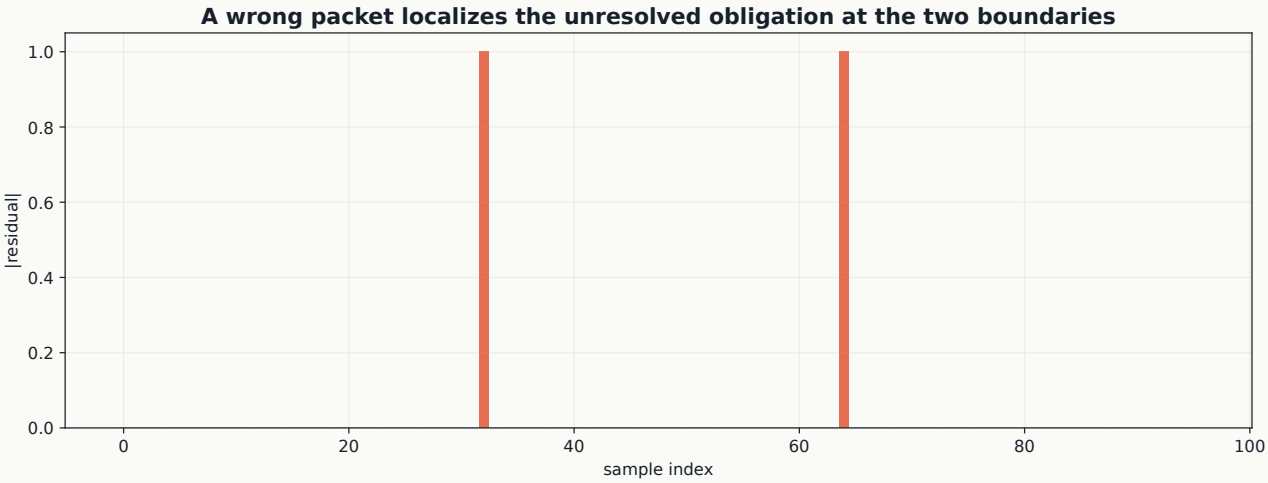


Figure 13.4: Absolute residual under a wrong interval packet.

What the figure shows. Only two narrow regions require additional geometric refinement.

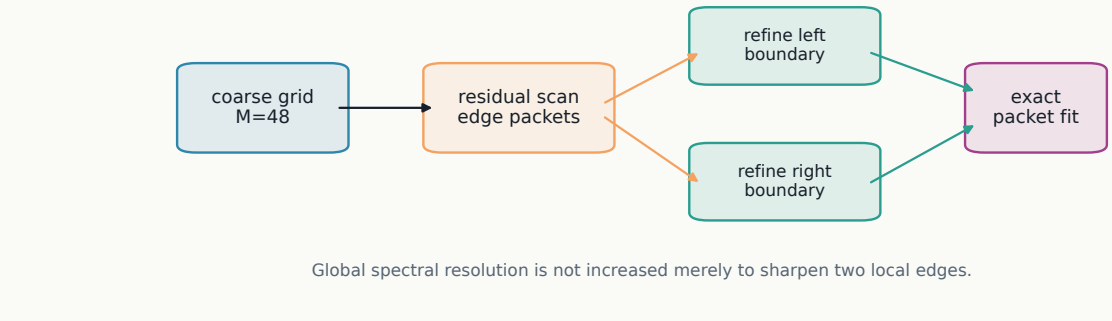


Figure 13.5: Residual-driven local refinement.

What the figure shows. The calculus refines where the obstruction lives rather than increasing global Fourier order everywhere.

14 Noise experiments

14.1 Status of the experiment

The noise study is not part of the exact theorem. For each standard deviation

$$\sigma \in \{0, .01, .03, .05, .08, .12, .16, .20, .25\},$$

the program adds independent Gaussian noise to the 96 samples and reruns the complete finite interval search. The random generator seed is fixed, and every aggregate is reproducible.

14.2 Observed behavior

At low and moderate noise the objective gap is large enough that the correct support remains stable in the sampled experiments. At larger noise the support rate decreases and endpoint errors appear. This is the expected boundary between an exact finite identification theorem and a statistical recovery theorem.

Boundary 14.1. The displayed curves do not prove minimax optimality, posterior consistency, or universal robustness. They certify only the deterministic experiment encoded in the source.

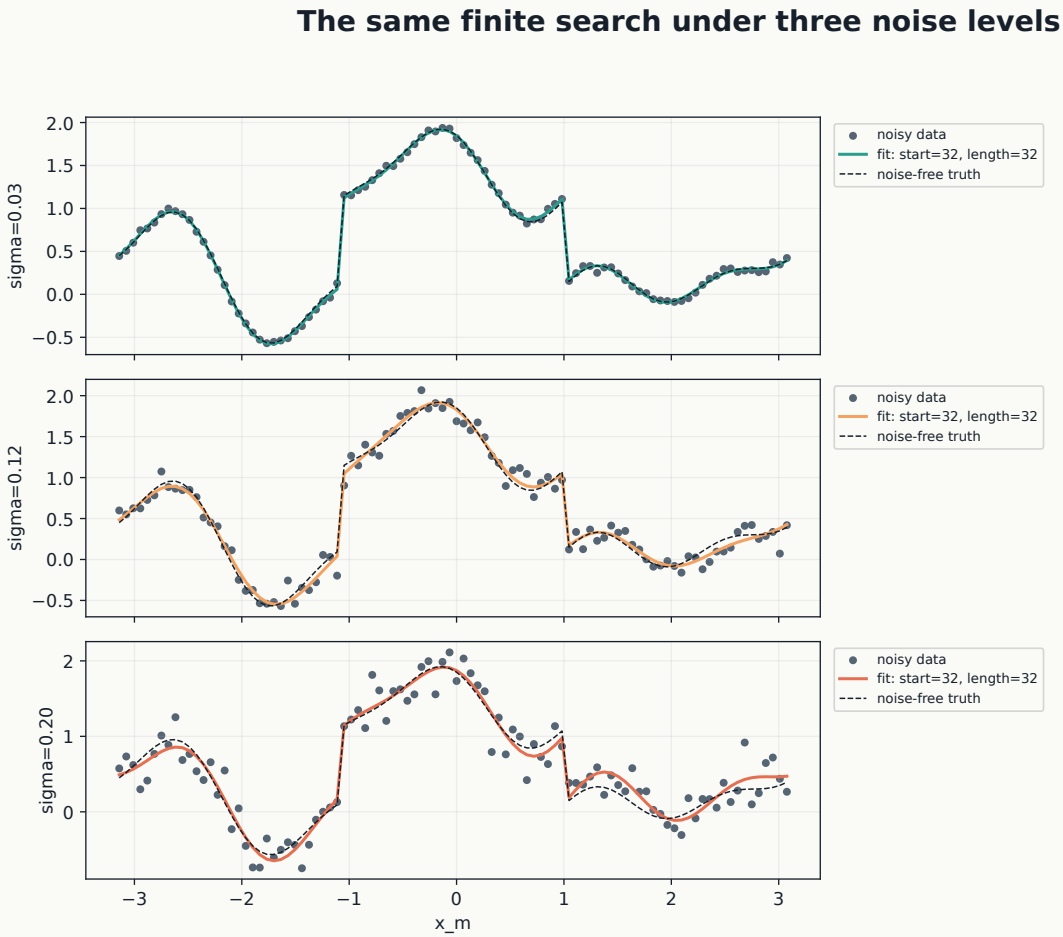


Figure 14.1: Three representative noisy data sets and their recovered packet models.

What the figure shows. Each panel reports the selected start and length.

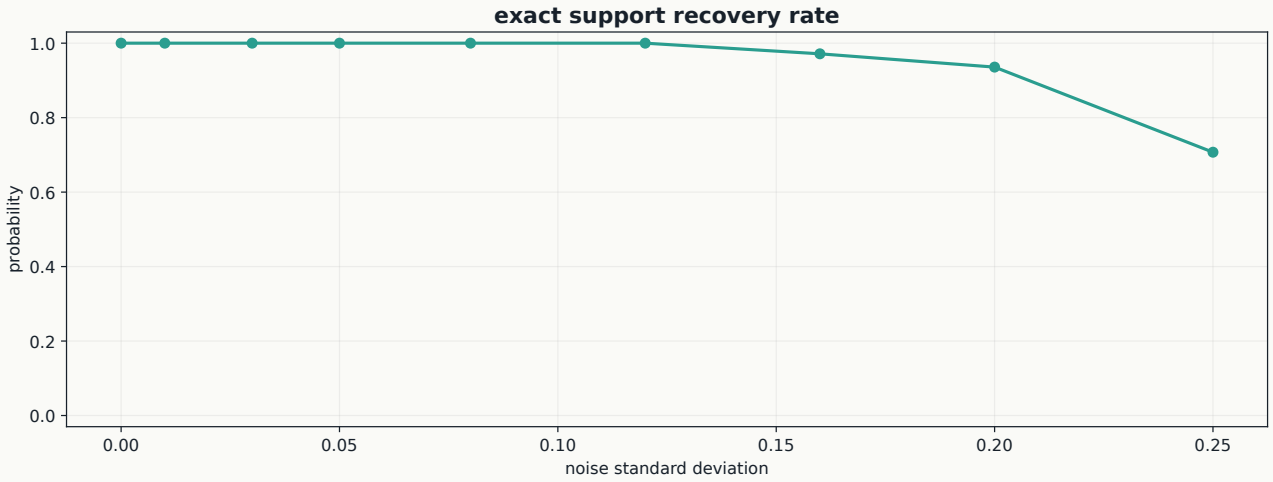


Figure 14.2: Exact support-recovery frequency in the deterministic Monte Carlo suite.

What the figure shows. The rate remains one through the moderate noise levels of this experiment and then degrades.

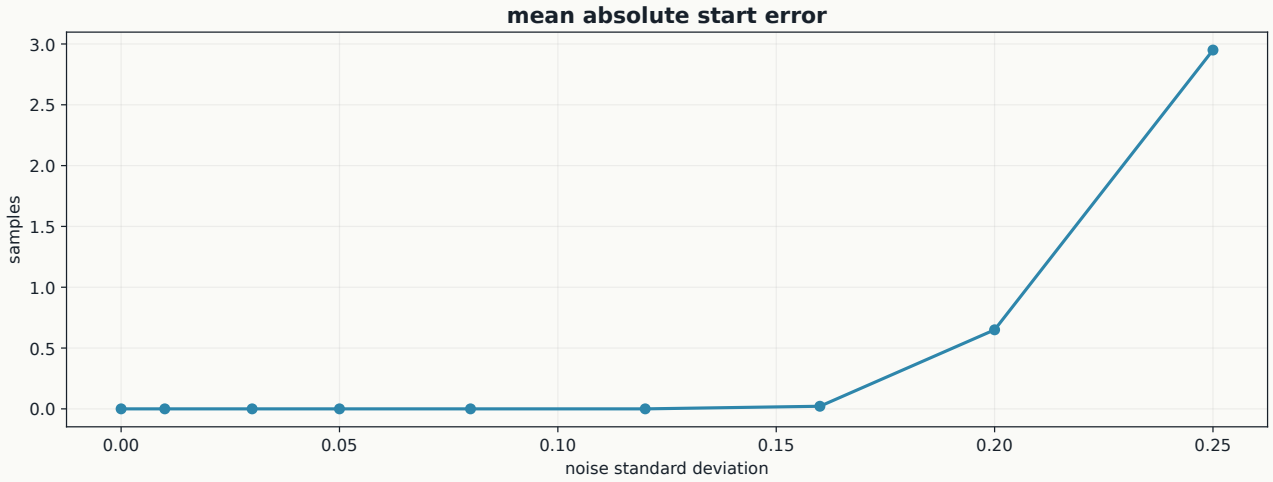


Figure 14.3: Mean absolute start-index error.

What the figure shows. The endpoint error becomes nonzero only after the support selector begins to change.

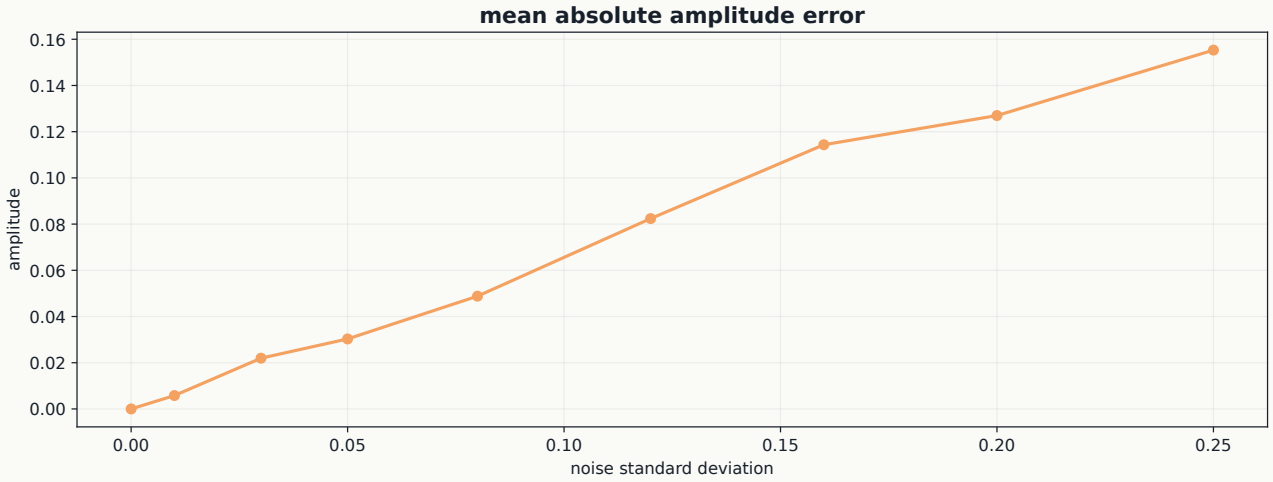


Figure 14.4: Mean absolute amplitude error.

What the figure shows. Amplitude estimation degrades continuously before support recovery necessarily fails.

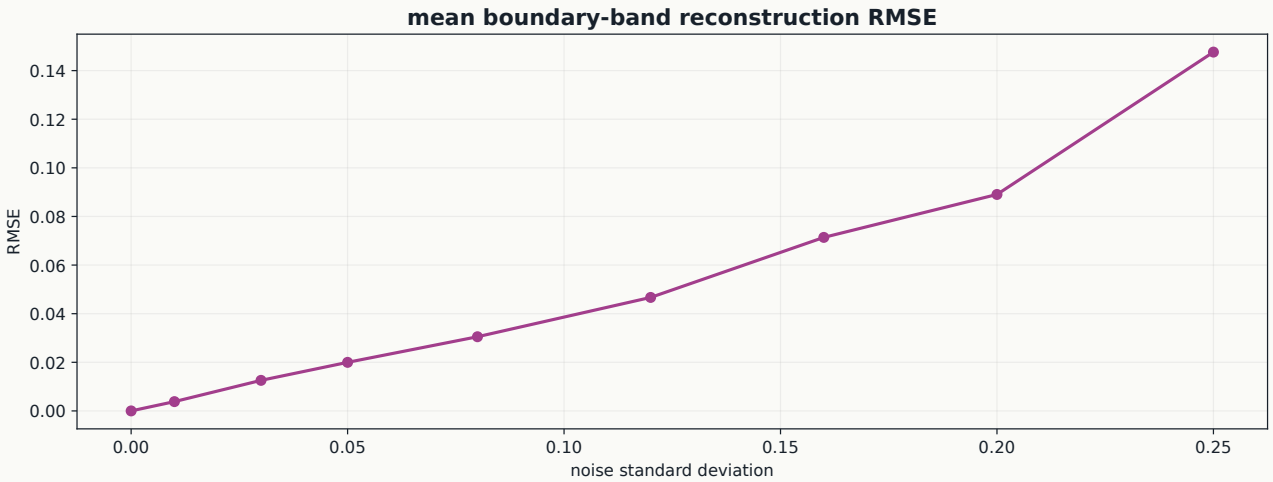


Figure 14.5: Mean reconstruction error in a seven-sample band around each boundary.

What the figure shows. The boundary metric is more sensitive to support errors than global RMSE.

15 Off-grid edges

15.1 Sampling resolution

Now shift the continuous half-width by 0.37 grid spacings. The sampled mask still belongs to the finite interval library, but the true continuous edge lies between samples. The algorithm can recover the sampled support exactly while the continuous endpoint remains known only to within the grid spacing.

15.2 Refinement experiment

The program repeats the fit at several grid sizes. The measured maximum endpoint error tracks the sample spacing

$$\Delta x = \frac{2\pi}{M}.$$

This demonstrates the role of geometric refinement: continuous localization improves by refining the address system, not by increasing the global Fourier order on a fixed grid.

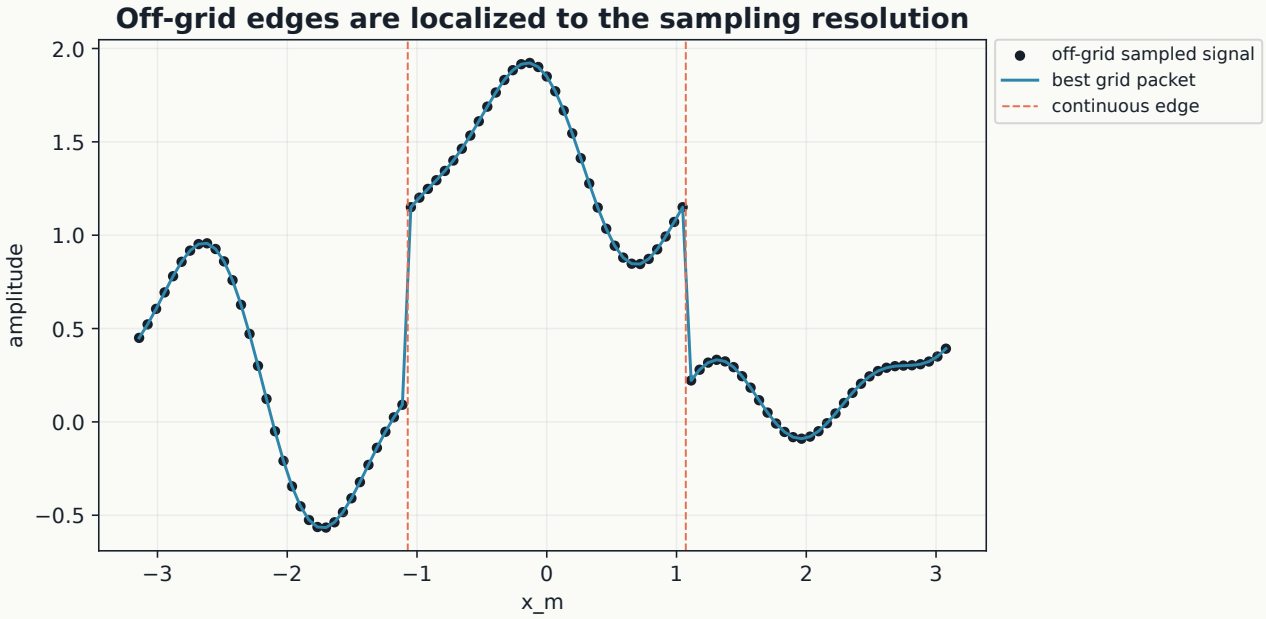


Figure 15.1: One off-grid sampled example and its best grid packet.

What the figure shows. The continuous edge is shown separately from the discrete support boundary.

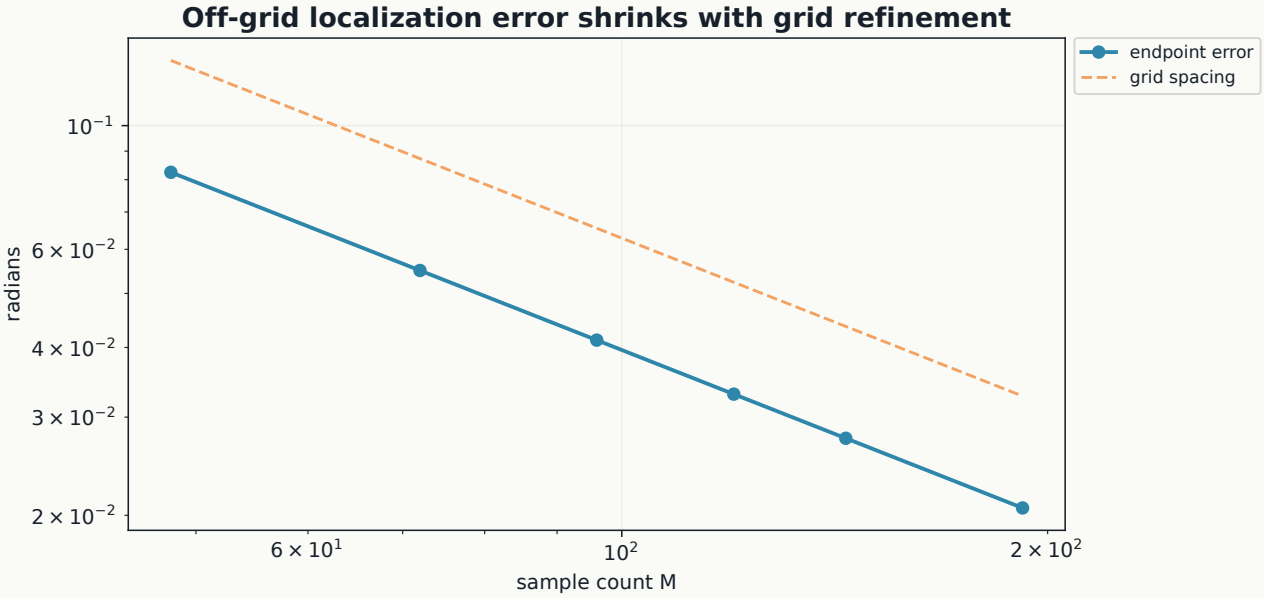


Figure 15.2: Endpoint error and grid spacing under refinement.

What the figure shows. The computation is consistent with first-order address localization for this simple grid selector.

16 Error ledger

Part VI: Exact reconstruction and approximation
Raw truncation, classical filters, and packet reconstruction are compared on the same signal with the same error measures.

16.1 Metrics

The program records:

Global RMSE the root-mean-square error over the full periodic grid.

Maximum error the pointwise maximum, including the jump midpoint convention.

Boundary RMSE the RMSE within a fixed band of both edges.

Jump overshoot the maximum amount above the interior unit level near the right edge.

Jump undershoot the minimum value below the exterior zero level.

Transition width the distance between approximate 0.9 and 0.1 crossings.

Total variation the periodic discrete variation of the reconstructed signal.

No one metric is sufficient. Fejer averaging has no jump overshoot but nonzero transition width. Raw Fourier truncation has a sharper transition and persistent normalized overshoot. The packet operator has zero transition width, zero jump overshoot, and zero model error after the smooth order is reached.

16.2 Exact versus approximate operations

Operation	Status	Meaning
Fourier coefficient formula	exact identity	obtained by direct integration
Dirichlet-kernel representation	exact identity	finite partial sum as convolution
Scaled Gibbs profile	asymptotic theorem	fixed scaled coordinate as $N \to \infty$
Smooth-edge projectors	exact algebra	direct-sum model coordinates
Resolved operator	exact theorem	identity on $\mathcal{M}_5$ for $N \geq 5$
Finite interval search	exact finite optimization	exhaustive stated candidate family
Noise study	computation	fixed random seed and finite repetitions
Off-grid study	computation	finite grid sizes and one shifted edge
Universal edge recovery	not claimed	would require a separate statistical theory



Figure 16.1: Residue-class refinement of the edge coefficient sequence.

What the figure shows. The spectral tail has exact arithmetic structure even though it is not finitely supported.



Figure 16.2: The final claim boundary.

What the figure shows. The last chart separates exact identities, asymptotic results, finite checks, numerical experiments, and claims that are not made.

17 Computational complexity and alternatives

17.1 Reference implementation cost

For grid size M, the interval library contains $O(M^2)$ candidates. With fixed smooth degree K, residualizing each candidate costs $O(MK)$ in the direct reference implementation. The library build is therefore polynomial and intentionally transparent. Once residual masks are precomputed, one profile search is a matrix-vector product.

17.2 Accelerations consistent with the proof

The same objective can be accelerated by:

- cumulative sums for interval correlations.
- FFT evaluation of cyclic correlations.
- multiscale interval trees.
- branch-and-bound using objective lower bounds.
- dynamic programming for nested packet families.
- local refinement triggered by the residual support.

These are implementation improvements. They do not change the exact analysis-synthesis theorem.

The exhaustive solver checks every finite candidate. It is not claimed to be asymptotically optimal

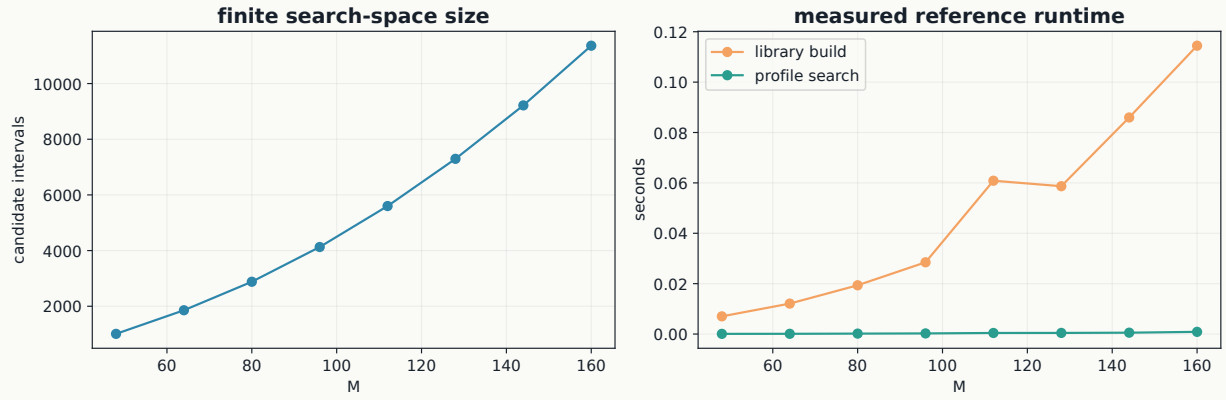


Figure 17.1: Measured library-build and profile-search times for the reference code.

What the figure shows. The timings describe this implementation on this machine. They are not asymptotic hardware claims.

18 How the proof fits together

18.1 Two independent chains

The complete result uses two proof chains.

The first chain explains the failure of raw truncation:

exact edge coefficients → Dirichlet convolution → scaled sine-integral limit → persistent Gibbs constant.

The second chain proves the resolved reconstruction:

direct-sum model → complete projectors → packet-specific operator → exact synthesis.

They meet at the comparison

$$S_N f = S_N s + S_N j, \qquad \mathcal{R}_N f = S_N s + j.$$

The first truncates the singular packet. The second does not.

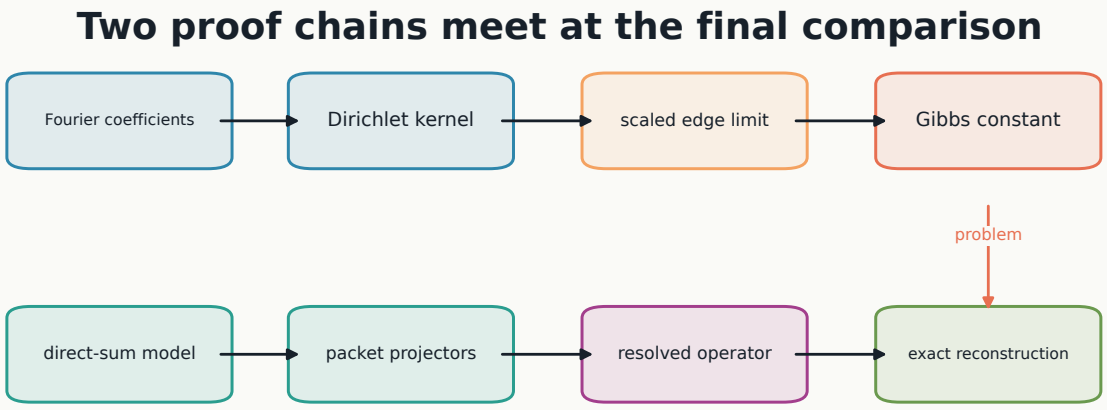


Figure 18.1: Dependency map of the failure theorem and the reconstruction theorem.

What the figure shows. The packet result does not deny the Gibbs theorem. It changes the operator acting on the chosen singular component.

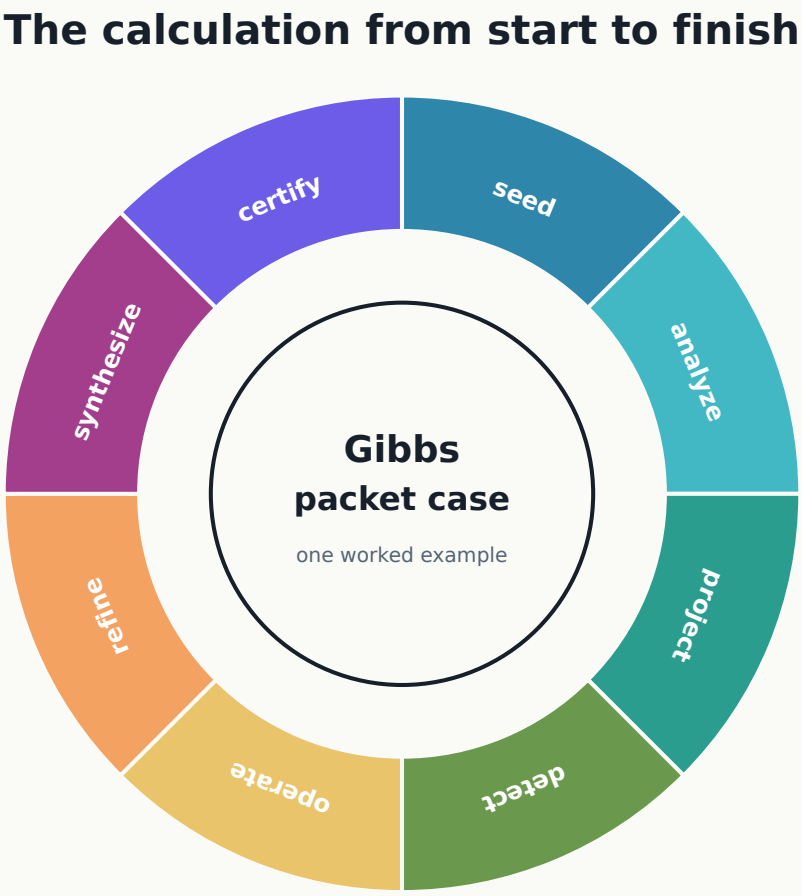


Figure 18.2: The calculation from start to finish.

What the figure shows. The appendix contains the one Python file used for every figure, table, and numerical check.

19 Verification table

Part VII: Verification and reproducibility
The analytic identities and numerical experiments are tied to one source file, one command, and a row-by-row record of the checks.

19.1 Independent checks

The program checks:

1. exact Fourier coefficients against dense quadrature at nine indices.
2. exact packet synthesis.
3. exact degree-five smooth reconstruction.
4. exact resolved reconstruction.
5. the scaled Gibbs profile at multiple coordinates.
6. the Gibbs constant at high order.
7. positivity of the Fejer jump reconstruction.
8. exact recovery of start, length, and amplitude.
9. exact finite reconstruction and positive identifiability gap.
10. the six-phase coefficient pattern.
11. the symmetric-difference identity.
12. the off-grid localization check.
13. low-noise success and high-noise degradation controls.
14. persistence of raw Gibbs overshoot and zero packet overshoot.

The final generated test ledger contains twenty-six passing rows.

Analytic and numerical checks for the worked example

Fourier coefficient n=-8	4.1668178185627625e-07	PASS
Fourier coefficient n=-5	4.1667611356405176e-07	PASS
Fourier coefficient n=-3	8.333333334741957e-07	PASS
Fourier coefficient n=-2	4.1667044542448295e-07	PASS
Fourier coefficient n=0	8.333333332277704e-07	PASS
Fourier coefficient n=2	4.1667044542448295e-07	PASS
Fourier coefficient n=3	8.333333334741957e-07	PASS
Fourier coefficient n=5	4.1667611356405176e-07	PASS
Fourier coefficient n=8	4.1668178185627625e-07	PASS
packet synthesis	0.0	PASS
smooth partial sum exact at N=5	0.0	PASS
resolved reconstruction exact	0.0	PASS
scaled Gibbs profile	0.00045965116011781326	PASS
Gibbs constant	0.00018380963237185632	PASS
Fejer positivity	-0.0022686127579852333	PASS
true interval recovered	start=32, length=32	PASS
edge amplitude recovered	0.0	PASS
finite reconstruction	5.551115123125783e-16	PASS
positive objective gap	0.7531607493634364	PASS
six-phase coefficient pattern	0.0	PASS
symmetric-difference identity	1.7763568394002505e-15	PASS
off-grid localization check	0.6300000000000032	PASS
low-noise support recovery	1.0	PASS
high-noise degradation visible	0.9357142857142857	PASS
raw Gibbs persists	0.08833904339109466	PASS
packet overshoot zero	0.0	PASS

26/26 checks passed

Figure 19.1: Verification results generated by the program.

What the figure shows. Each row gives the result of one check together with its residual or numerical witness.

20 One-command reproduction

From the source directory, the entire computational layer is regenerated by

```
python gibbs_packet_case.py --manifest
```

The PDF is then compiled with

```
pdflatex -interaction=nonstopmode -halt-on-error Projector_Resolved_Gibbs_Case_Study.tex
pdflatex -interaction=nonstopmode -halt-on-error Projector_Resolved_Gibbs_Case_Study.tex
```

The Windows PowerShell runner included in the release performs the same sequence.

21 Final theorem and conclusion

Theorem 21.1 (Worked resolution of the Gibbs case)**.** *For the signal*

$$f(x) = \frac{1}{4} + \frac{2}{5}\cos(2x) - \frac{1}{3}\sin(3x) + \frac{1}{5}\cos(5x) + \mathbf{1}_{[-\pi/3,\pi/3)}(x),$$

the following statements hold.

1. *Raw global Fourier partial sums have the universal Gibbs edge profile and normalized first overshoot* $G = 0.089489872236\ldots$.
2. *The chosen model class* $\mathcal{M}_5 = \mathcal{T}_5 \oplus \operatorname{span}\{j\}$ *has complementary exact projectors* P_{s} *and* P_{j}.
3. *The resolved operator* $\mathcal{R}_N = S_N P_{\mathrm{s}} + P_{\mathrm{j}}$ *satisfies* $\mathcal{R}_N = I$ *on* $\mathcal{M}_5$ *for every* $N \geq 5$.
4. *On the stated* 96*-sample interval library, profile least squares uniquely recovers the true positive packet* $(32, 32)$ *with amplitude one and a positive objective gap.*
5. *The recovered smooth and edge packets synthesize the samples with maximum residual* 5.55×10^{-16}.

Proof. Items 1–3 were proved in the analytic chapters. Item 4 follows from the finite interval-recovery theorem and the positive gap between the true interval and every competing interval. Item 5 follows from the displayed synthesis formula and the numerical record. □

The conclusion is straightforward. The Gibbs theorem remains true for raw Fourier partial sums. In the reconstructed model, the discontinuity is stored in its own packet, so Fourier truncation is applied only to the smooth part. On the finite grid, the interval packet is recovered by checking every allowed candidate. The example therefore runs from the original signal to a final residual without hiding an intermediate step.

A Exact formula sheet

Part VIII: Appendices
The appendices collect the formulas, numerical tables, full implementation, file map, and the exact boundary of the result.

A.1 Signal

$$s(x) = \frac{1}{4} + \frac{2}{5}\cos(2x) - \frac{1}{3}\sin(3x) + \frac{1}{5}\cos(5x), \qquad j(x) = \mathbf{1}_{[-\pi/3,\pi/3)}(x), \qquad f = s + j.$$

A.2 Fourier coefficients

$$\widehat{s}(0) = \frac{1}{4}, \quad \widehat{s}(\pm 2) = \frac{1}{5}, \quad \widehat{s}(3) = \frac{\mathrm{i}}{6}, \quad \widehat{s}(-3) = -\frac{\mathrm{i}}{6}, \quad \widehat{s}(\pm 5) = \frac{1}{10}.$$

$$\widehat{j}(0) = \frac{1}{3}, \qquad \widehat{j}(n) = \frac{\sin(n\pi/3)}{\pi n} \quad (n \neq 0).$$

A.3 Partial sum

$$S_N j(x) = \frac{1}{3} + \frac{2}{\pi}\sum_{n=1}^{N} \frac{\sin(n\pi/3)}{n}\cos(nx).$$

A.4 Gibbs profile

$$\lim_{N\to\infty} S_N j\left(\frac{\pi}{3} - \frac{z}{N + 1/2}\right) = \frac{1}{2} + \frac{1}{\pi}\,\mathrm{Si}(z).$$

$$G = \frac{\mathrm{Si}(\pi)}{\pi} - \frac{1}{2}.$$

A.5 Resolved operator

$$P_{\mathrm{s}}(t + cj) = t, \qquad P_{\mathrm{j}}(t + cj) = cj, \qquad \mathcal{R}_N = S_N P_{\mathrm{s}} + P_{\mathrm{j}}.$$

$$\mathcal{R}_N = I_{\mathcal{M}_5} \qquad (N \geq 5).$$

A.6 Finite profile objective

$$P_B = QQ^{\mathsf{T}}, \qquad y^{\perp} = (I - P_B)y, \qquad q^{\perp} = (I - P_B)q.$$

$$\widehat{A}(q) = \frac{\left\langle q^{\perp}, y^{\perp} \right\rangle}{\left\| q^{\perp} \right\|^2}, \qquad J(q) = \left\| y^{\perp} - \widehat{A}(q) q^{\perp} \right\|^2.$$

B Selected numerical tables

B.1 Exact recovery record

Quantity	Value
Grid size	96
True start	32
True length	32
Recovered start	32
Recovered length	32
Recovered amplitude	1
Best objective	0
Second objective	0.7531607493634364
Maximum synthesis residual	$5.551115123125783 \times 10^{-16}$
Gibbs constant	0.08948987223608351

B.2 Machine-readable tables

The source release includes:

- `tables/fourier_coefficients.csv`.
- `tables/method_metrics.csv`.
- `tables/noise_recovery.csv`.
- `tables/offgrid_recovery.csv`.
- `tables/runtime_scaling.csv`.
- `tables/test_ledger.csv`.
- `results/numerical_results.json`.

C The full Python implementation

The following Python file generates every figure, table, and numerical check in this study.

```python
#!/usr/bin/env python3
"""A reproducible Gibbs-phenomenon case study.

This file generates every calculation, table, figure, and numerical check used
in "Projector-Resolved Gibbs Reconstruction." It contains one explicit signal,
one model space, one finite interval-recovery algorithm, and one build command.

Author: Komorebi P.
"""
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import math
import os
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Iterable, Sequence

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Circle, Wedge
import numpy as np
from numpy.typing import NDArray
from scipy.special import sici

# ---------------------------------------------------------------------------
# Global constants and deterministic style
# ---------------------------------------------------------------------------
PI = math.pi
A_EDGE = 1.0
EDGE_HALF_WIDTH = PI / 3.0
SMOOTH_DEGREE = 5
DEFAULT_GRID = 96
TRUE_START = DEFAULT_GRID // 3
TRUE_LENGTH = DEFAULT_GRID // 3
RNG_SEED = 20260729

INK = "#17202A"
MUTED = "#566573"
PAPER = "#FAFAF7"
WHITE = "#FFFFFF"
GRID = "#D5D8DC"
BLUE = "#2E86AB"
CYAN = "#42B8C5"
TEAL = "#2A9D8F"
GREEN = "#6A994E"
YELLOW = "#E9C46A"
ORANGE = "#F4A261"
RED = "#E76F51"
MAGENTA = "#A23E8C"
PURPLE = "#6C5CE7"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, PURPLE, GREEN, RED, CYAN, YELLOW]

plt.rcParams.update(
    {
        "font.family": "DejaVu Sans",
        "font.size": 10.5,
        "axes.titlesize": 14,
        "axes.labelsize": 10.5,
        "axes.edgecolor": INK,
```

```
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
        "savefig.facecolor": PAPER,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
    }
)

Array = NDArray[np.float64]
ComplexArray = NDArray[np.complex128]


# ---------------------------------------------------------------------------
# The single signal and its exact continuous formulas
# ---------------------------------------------------------------------------
def wrap_angle(x: Array | float) -> Array:
    arr = np.asarray(x, dtype=float)
    return (arr + PI) % (2.0 * PI) - PI


def smooth_seed(x: Array | float) -> Array:
    """The exact degree-five trigonometric background."""
    arr = np.asarray(x, dtype=float)
    return (
        1.0 / 4.0
        + 2.0 / 5.0 * np.cos(2.0 * arr)
        - 1.0 / 3.0 * np.sin(3.0 * arr)
        + 1.0 / 5.0 * np.cos(5.0 * arr)
    )


def jump_packet(x: Array | float, half_width: float = EDGE_HALF_WIDTH, amplitude: float = A_EDGE) -> Array:
    """Periodic rectangular packet on [-half_width, half_width)."""
    arr = wrap_angle(x)
    return amplitude * ((arr >= -half_width) & (arr < half_width)).astype(float)


def full_signal(x: Array | float, half_width: float = EDGE_HALF_WIDTH, amplitude: float = A_EDGE) -> Array:
    return smooth_seed(x) + jump_packet(x, half_width=half_width, amplitude=amplitude)


def smooth_fourier_coefficient(n: int) -> complex:
    if n == 0:
        return 1.0 / 4.0
    if abs(n) == 2:
        return 1.0 / 5.0
    if n == 3:
        return 1j / 6.0
    if n == -3:
        return -1j / 6.0
    if abs(n) == 5:
        return 1.0 / 10.0
    return 0.0 + 0.0j


def jump_fourier_coefficient(n: int, half_width: float = EDGE_HALF_WIDTH, amplitude: float = A_EDGE) ->
     complex:
    if n == 0:
        return amplitude * half_width / PI
    return amplitude * math.sin(n * half_width) / (PI * n)


def full_fourier_coefficient(n: int, half_width: float = EDGE_HALF_WIDTH, amplitude: float = A_EDGE) ->
     complex:
    return smooth_fourier_coefficient(n) + jump_fourier_coefficient(n, half_width, amplitude)


```

```python
def evaluate_fourier_series(
    x: Array,
    order: int,
    coefficient: Callable[[int], complex],
    weight: Callable[[int, int], float] | None = None,
) -> Array:
    if order < 0:
        raise ValueError("order must be nonnegative")
    if weight is None:
        weight = lambda k, n: 1.0
    out = np.zeros_like(x, dtype=np.complex128)
    for n in range(-order, order + 1):
        out += weight(abs(n), order) * coefficient(n) * np.exp(1j * n * x)
    return np.real_if_close(out, tol=1000).real


def raw_fourier_partial_sum(x: Array, order: int) -> Array:
    return evaluate_fourier_series(x, order, full_fourier_coefficient)


def _jump_weighted_sum(x: Array, order: int, weight: Callable[[int, int], float] | None = None) -> Array:
    if weight is None:
        weight = lambda k, n: 1.0
    out = np.full_like(x, EDGE_HALF_WIDTH / PI, dtype=float)
    if order == 0:
        return out
    n = np.arange(1, order + 1, dtype=float)
    weights = np.array([weight(int(k), order) for k in n], dtype=float)
    coeff = (2.0 / PI) * np.sin(n * EDGE_HALF_WIDTH) / n * weights
    # Chunk the mode matrix so the standalone script remains memory-safe.
    chunk = 20000
    for start in range(0, x.size, chunk):
        stop = min(x.size, start + chunk)
        out[start:stop] += coeff @ np.cos(np.outer(n, x[start:stop]))
    return out


def jump_fourier_partial_sum(x: Array, order: int) -> Array:
    return _jump_weighted_sum(x, order)


def smooth_fourier_partial_sum(x: Array, order: int, weight: Callable[[int, int], float] | None = None) ->
 Array:
    if weight is None:
        weight = lambda k, n: 1.0
    out = np.full_like(x, 1.0 / 4.0, dtype=float)
    if order >= 2:
        out += (2.0 / 5.0) * weight(2, order) * np.cos(2.0 * x)
    if order >= 3:
        out -= (1.0 / 3.0) * weight(3, order) * np.sin(3.0 * x)
    if order >= 5:
        out += (1.0 / 5.0) * weight(5, order) * np.cos(5.0 * x)
    return out


def fejer_weight(k: int, n: int) -> float:
    return 1.0 - k / (n + 1.0)


def lanczos_weight(k: int, n: int) -> float:
    if n == 0:
        return 1.0
    return float(np.sinc(k / (n + 1.0)))


def exponential_weight(k: int, n: int, power: int = 8, alpha: float = 18.0) -> float:
    if n == 0:
        return 1.0
    return math.exp(-alpha * (k / n) ** power)


```

```python
def filtered_full_sum(x: Array, order: int, method: str) -> Array:
    if method == "raw":
        weight = None
    elif method == "fejer":
        weight = fejer_weight
    elif method == "lanczos":
        weight = lanczos_weight
    elif method == "exponential":
        weight = lambda k, n: exponential_weight(k, n)
    else:
        raise ValueError(f"unknown method: {method}")
    return smooth_fourier_partial_sum(x, order, weight) + _jump_weighted_sum(x, order, weight)


def filtered_jump_sum(x: Array, order: int, method: str) -> Array:
    if method == "raw":
        return jump_fourier_partial_sum(x, order)
    if method == "fejer":
        return _jump_weighted_sum(x, order, fejer_weight)
    if method == "lanczos":
        return _jump_weighted_sum(x, order, lanczos_weight)
    if method == "exponential":
        return _jump_weighted_sum(x, order, lambda k, n: exponential_weight(k, n))
    raise ValueError(f"unknown method: {method}")


def packet_reconstruction(x: Array, order: int) -> Array:
    """Resolved operator R_N = S_N P_smooth + P_edge."""
    return smooth_fourier_partial_sum(x, order) + jump_packet(x)


def dirichlet_kernel(t: Array, order: int) -> Array:
    out = np.empty_like(t, dtype=float)
    denom = np.sin(t / 2.0)
    mask = np.abs(denom) < 1e-13
    out[mask] = 2 * order + 1
    out[~mask] = np.sin((order + 0.5) * t[~mask]) / denom[~mask]
    return out


def fejer_kernel(t: Array, order: int) -> Array:
    out = np.empty_like(t, dtype=float)
    denom = np.sin(t / 2.0)
    mask = np.abs(denom) < 1e-13
    out[mask] = order + 1.0
    out[~mask] = (np.sin((order + 1.0) * t[~mask] / 2.0) ** 2) / ((order + 1.0) * denom[~mask] ** 2)
    return out


def gibbs_profile(z: Array) -> Array:
    return 0.5 + sici(z)[0] / PI


def gibbs_constant() -> float:
    return float(sici(PI)[0] / PI - 0.5)


# ---------------------------------------------------------------------------
# Exact packet calculus on the chosen model space
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class PacketState:
    smooth: Array
    edge: Array

    def synthesize(self) -> Array:
        return self.smooth + self.edge


def analyze_declared_model(x: Array) -> PacketState:
    """Analysis for the worked example.
```

```
    The input x is assumed to be the signal defined in the paper. The paper
    proves the corresponding model-space projectors abstractly. This function
    implements their action on the worked example.
    """
    return PacketState(smooth=smooth_seed(x), edge=jump_packet(x))


# ------------------------------------------------------------------------
# Finite sample model and exact profile search
# ------------------------------------------------------------------------
def sample_grid(size: int) -> Array:
    return -PI + 2.0 * PI * np.arange(size) / size


def trigonometric_design(x: Array, degree: int) -> Array:
    cols: list[Array] = [np.ones_like(x)]
    for k in range(1, degree + 1):
        cols.append(np.cos(k * x))
        cols.append(np.sin(k * x))
    return np.column_stack(cols)


def orthonormal_basis(matrix: Array) -> Array:
    q, _ = np.linalg.qr(matrix)
    return q


def interval_mask(size: int, start: int, length: int) -> Array:
    if length < 1 or length > size:
        raise ValueError("invalid interval length")
    out = np.zeros(size, dtype=float)
    out[(start + np.arange(length)) % size] = 1.0
    return out


@dataclass
class CandidateLibrary:
    size: int
    degree: int
    x: Array
    design: Array
    q_background: Array
    masks: Array
    starts: NDArray[np.int64]
    lengths: NDArray[np.int64]
    residual_masks: Array
    residual_norms: Array

    @classmethod
    def build(
        cls,
        size: int = DEFAULT_GRID,
        degree: int = SMOOTH_DEGREE,
        min_length: int = 8,
        max_length: int | None = None,
    ) -> "CandidateLibrary":
        x = sample_grid(size)
        design = trigonometric_design(x, degree)
        q_background = orthonormal_basis(design)
        if max_length is None:
            max_length = size // 2
        masks: list[Array] = []
        starts: list[int] = []
        lengths: list[int] = []
        residual_masks: list[Array] = []
        residual_norms: list[float] = []
        for length in range(min_length, max_length + 1):
            for start in range(size):
                mask = interval_mask(size, start, length)
                residual = mask - q_background @ (q_background.T @ mask)
```

```
                residual_norm = float(residual @ residual)
                if residual_norm <= 1e-12:
                    continue
                masks.append(mask)
                starts.append(start)
                lengths.append(length)
                residual_masks.append(residual)
                residual_norms.append(residual_norm)
        return cls(
            size=size,
            degree=degree,
            x=x,
            design=design,
            q_background=q_background,
            masks=np.asarray(masks),
            starts=np.asarray(starts, dtype=np.int64),
            lengths=np.asarray(lengths, dtype=np.int64),
            residual_masks=np.asarray(residual_masks),
            residual_norms=np.asarray(residual_norms),
        )

    def residualize(self, y: Array) -> Array:
        return y - self.q_background @ (self.q_background.T @ y)

    def search(self, y: Array, positive_amplitude: bool = True) -> dict[str, object]:
        y_perp = self.residualize(y)
        dots = self.residual_masks @ y_perp
        amplitudes = dots / self.residual_norms
        residuals = (
            float(y_perp @ y_perp)
            - 2.0 * amplitudes * dots
            + amplitudes**2 * self.residual_norms
        )
        residuals = np.maximum(residuals, 0.0)
        if positive_amplitude:
            residuals = np.where(amplitudes > 0.0, residuals, np.inf)
        order = np.argsort(residuals)
        best = int(order[0])
        second = int(order[1])
        mask = self.masks[best]
        amplitude = float(amplitudes[best])
        smooth_fit = self.q_background @ (self.q_background.T @ (y - amplitude * mask))
        reconstruction = smooth_fit + amplitude * mask
        return {
            "index": best,
            "start": int(self.starts[best]),
            "length": int(self.lengths[best]),
            "amplitude": amplitude,
            "residual": float(residuals[best]),
            "second_residual": float(residuals[second]),
            "gap": float(residuals[second] - residuals[best]),
            "mask": mask,
            "smooth": smooth_fit,
            "reconstruction": reconstruction,
            "all_residuals": residuals,
            "all_amplitudes": amplitudes,
            "y_perp": y_perp,
        }


def true_discrete_signal(size: int = DEFAULT_GRID) -> tuple[Array, Array, Array, Array]:
    x = sample_grid(size)
    smooth = smooth_seed(x)
    start = size // 3
    length = size // 3
    edge = interval_mask(size, start, length)
    return x, smooth, edge, smooth + edge


# ---------------------------------------------------------------------------
# Metrics
```

```python
# ---------------------------------------------------------------------------
def rmse(a: Array, b: Array) -> float:
    return float(np.sqrt(np.mean((a - b) ** 2)))


def maximum_error(a: Array, b: Array) -> float:
    return float(np.max(np.abs(a - b)))


def boundary_mask(x: Array, width: float = 0.15, half_width: float = EDGE_HALF_WIDTH) -> NDArray[np.bool_]:
    return (np.abs(np.abs(wrap_angle(x)) - half_width) <= width)


def boundary_rmse(approx: Array, truth: Array, x: Array, width: float = 0.15) -> float:
    mask = boundary_mask(x, width=width)
    return rmse(approx[mask], truth[mask])


def jump_overshoot(approx_jump: Array, x: Array, half_width: float = EDGE_HALF_WIDTH, band: float = 0.3) ->
     float:
    inside = (x < half_width - 1e-8) & (x > half_width - band)
    return float(np.max(approx_jump[inside] - 1.0))


def jump_undershoot(approx_jump: Array, x: Array, half_width: float = EDGE_HALF_WIDTH, band: float = 0.3) ->
      float:
    outside = (x > half_width + 1e-8) & (x < half_width + band)
    return float(np.min(approx_jump[outside]))


def total_variation_periodic(values: Array) -> float:
    return float(np.sum(np.abs(np.roll(values, -1) - values)))


def transition_width(values: Array, x: Array, low: float = 0.1, high: float = 0.9, edge: float =
     EDGE_HALF_WIDTH) -> float:
    window = (x > edge - 0.5) & (x < edge + 0.5)
    xx = x[window]
    yy = values[window]
    # descending edge: find nearest positions to high and low crossings.
    high_idx = int(np.argmin(np.abs(yy - high)))
    low_idx = int(np.argmin(np.abs(yy - low)))
    return float(abs(xx[low_idx] - xx[high_idx]))


# ---------------------------------------------------------------------------
# Plot helpers
# ---------------------------------------------------------------------------
def save_figure(fig: plt.Figure, out_dir: Path, name: str) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    fig.savefig(out_dir / f"{name}.pdf", bbox_inches="tight", pad_inches=0.08)
    fig.savefig(out_dir / f"{name}.png", dpi=200, bbox_inches="tight", pad_inches=0.08)
    plt.close(fig)


def style_axis(ax: plt.Axes, grid: bool = True) -> None:
    if grid:
        ax.grid(alpha=0.18)
    for spine in ax.spines.values():
        spine.set_linewidth(0.8)


def add_external_legend(ax: plt.Axes, ncol: int = 1) -> None:
    ax.legend(
        loc="upper left",
        bbox_to_anchor=(1.01, 1.0),
        borderaxespad=0.0,
        frameon=True,
        fontsize=9,
        ncol=ncol,
    )
```

```python


def diagram_box(ax: plt.Axes, x: float, y: float, w: float, h: float, text: str, color: str, fontsize: float = 11) -> None:
    patch = FancyBboxPatch(
        (x, y),
        w,
        h,
        boxstyle="round,pad=0.012,rounding_size=0.02",
        facecolor=color + "20",
        edgecolor=color,
        linewidth=1.7,
    )
    ax.add_patch(patch)
    label = ax.text(x + w / 2, y + h / 2, text, ha="center", va="center", multialignment="center", fontsize=fontsize)
    # renderer-based fit
    for _ in range(10):
        ax.figure.canvas.draw()
        r = ax.figure.canvas.get_renderer()
        tb = label.get_window_extent(r)
        (x0, y0), (x1, y1) = ax.transData.transform(((x, y), (x + w, y + h)))
        if tb.width <= abs(x1 - x0) * 0.84 and tb.height <= abs(y1 - y0) * 0.78:
            break
        label.set_fontsize(max(7.5, label.get_fontsize() * 0.90))


def diagram_arrow(ax: plt.Axes, start: tuple[float, float], end: tuple[float, float], color: str = INK, label: str | None = None) -> None:
    arrow = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.6, color=color)
    ax.add_patch(arrow)
    if label:
        mx = 0.5 * (start[0] + end[0])
        my = 0.5 * (start[1] + end[1])
        ax.text(mx, my + 0.025, label, ha="center", va="bottom", fontsize=9.3, color=color, bbox={"facecolor": PAPER, "edgecolor": "none", "pad": 1.2})


# ---------------------------------------------------------------------------
# Main computational experiment
# ---------------------------------------------------------------------------
def build_experiment(root: Path, quick: bool = False) -> dict[str, object]:
    figures = root / "figures"
    tables = root / "tables"
    results_dir = root / "results"
    figures.mkdir(parents=True, exist_ok=True)
    tables.mkdir(parents=True, exist_ok=True)
    results_dir.mkdir(parents=True, exist_ok=True)

    dense_count = 18000 if quick else 36000
    x_dense = np.linspace(-PI, PI, dense_count, endpoint=False)
    s_dense = smooth_seed(x_dense)
    j_dense = jump_packet(x_dense)
    f_dense = s_dense + j_dense

    # Exact coefficient table.
    coeff_rows: list[dict[str, object]] = []
    for n in range(-24, 25):
        cs = smooth_fourier_coefficient(n)
        cj = jump_fourier_coefficient(n)
        cf = cs + cj
        coeff_rows.append(
            {
                "n": n,
                "smooth_real": float(cs.real),
                "smooth_imag": float(cs.imag),
                "jump_real": float(cj.real),
                "jump_imag": float(cj.imag),
                "full_real": float(cf.real),
                "full_imag": float(cf.imag),
                "residue_mod_6": n % 6,
```

```
            }
        )
    with (tables / "fourier_coefficients.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(coeff_rows[0]))
        writer.writeheader()
        writer.writerows(coeff_rows)

    # Method curves and metrics over orders.
    orders = np.array([5, 8, 10, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96, 128, 160])
    methods = ["raw", "fejer", "lanczos", "exponential", "packet"]
    metric_rows: list[dict[str, object]] = []
    for order in orders:
        for method in methods:
            if method == "packet":
                approximation = packet_reconstruction(x_dense, int(order))
                jump_approximation = j_dense
            else:
                approximation = filtered_full_sum(x_dense, int(order), method)
                jump_approximation = filtered_jump_sum(x_dense, int(order), method)
            metric_rows.append(
                {
                    "order": int(order),
                    "method": method,
                    "rmse": rmse(approximation, f_dense),
                    "max_error": maximum_error(approximation, f_dense),
                    "boundary_rmse": boundary_rmse(approximation, f_dense, x_dense),
                    "jump_overshoot": jump_overshoot(jump_approximation, x_dense),
                    "jump_undershoot": jump_undershoot(jump_approximation, x_dense),
                    "jump_transition_width": transition_width(jump_approximation, x_dense),
                    "total_variation": total_variation_periodic(approximation),
                }
            )
    with (tables / "method_metrics.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(metric_rows[0]))
        writer.writeheader()
        writer.writerows(metric_rows)

    # Finite exact recovery.
    library = CandidateLibrary.build(DEFAULT_GRID, SMOOTH_DEGREE)
    x_grid, s_grid, edge_grid, y_grid = true_discrete_signal(DEFAULT_GRID)
    exact_search = library.search(y_grid)
    exact_coefficients, *_ = np.linalg.lstsq(
        library.design,
        y_grid - float(exact_search["amplitude"]) * np.asarray(exact_search["mask"]),
        rcond=None,
    )

    # Candidate objective as a start-length grid.
    max_len = DEFAULT_GRID // 2
    objective_grid = np.full((max_len + 1, DEFAULT_GRID), np.nan)
    all_residuals = np.asarray(exact_search["all_residuals"])
    for residual, start, length in zip(all_residuals, library.starts, library.lengths, strict=True):
        objective_grid[int(length), int(start)] = residual

    # Noise study.
    rng = np.random.default_rng(RNG_SEED)
    sigmas = np.array([0.0, 0.01, 0.03, 0.05, 0.08, 0.12, 0.16, 0.20, 0.25])
    repetitions = 40 if quick else 140
    noise_rows: list[dict[str, object]] = []
    noise_examples: dict[float, dict[str, object]] = {}
    for sigma in sigmas:
        noise = rng.normal(0.0, float(sigma), (DEFAULT_GRID, repetitions))
        noisy_matrix = y_grid[:, None] + noise
        qbg = library.q_background
        yperp_matrix = noisy_matrix - qbg @ (qbg.T @ noisy_matrix)
        dots = library.residual_masks @ yperp_matrix
        amplitudes = dots / library.residual_norms[:, None]
        base = np.sum(yperp_matrix**2, axis=0)[None, :]
        residuals = base - 2.0 * amplitudes * dots + amplitudes**2 * library.residual_norms[:, None]
        residuals = np.maximum(residuals, 0.0)
        residuals = np.where(amplitudes > 0.0, residuals, np.inf)
```

```
        best_indices = np.argmin(residuals, axis=0)
        starts = library.starts[best_indices]
        lengths = library.lengths[best_indices]
        best_amplitudes = amplitudes[best_indices, np.arange(repetitions)]
        exact_rate = float(np.mean((starts == TRUE_START) & (lengths == TRUE_LENGTH)))
        circular_start = ((starts - TRUE_START + DEFAULT_GRID / 2) % DEFAULT_GRID) - DEFAULT_GRID / 2
        reconstruction_errors: list[float] = []
        boundary_errors: list[float] = []
        edge_band = np.zeros(DEFAULT_GRID, dtype=bool)
        for center in (TRUE_START, TRUE_START + TRUE_LENGTH):
            for offset in range(-3, 4):
                edge_band[(center + offset) % DEFAULT_GRID] = True
        for rep, idx in enumerate(best_indices):
            mask = library.masks[int(idx)]
            amp = float(best_amplitudes[rep])
            noisy = noisy_matrix[:, rep]
            smooth_fit = qbg @ (qbg.T @ (noisy - amp * mask))
            reconstruction = smooth_fit + amp * mask
            reconstruction_errors.append(rmse(reconstruction, y_grid))
            boundary_errors.append(rmse(reconstruction[edge_band], y_grid[edge_band]))
        noise_rows.append(
            {
                "sigma": float(sigma),
                "repetitions": repetitions,
                "exact_support_rate": exact_rate,
                "mean_abs_start_error_samples": float(np.mean(np.abs(circular_start))),
                "mean_abs_length_error_samples": float(np.mean(np.abs(lengths - TRUE_LENGTH))),
                "mean_abs_amplitude_error": float(np.mean(np.abs(best_amplitudes - A_EDGE))),
                "mean_reconstruction_rmse": float(np.mean(reconstruction_errors)),
                "mean_boundary_rmse": float(np.mean(boundary_errors)),
            }
        )
        if float(sigma) in (0.03, 0.12, 0.20):
            idx = int(best_indices[0])
            mask = library.masks[idx]
            amp = float(best_amplitudes[0])
            noisy = noisy_matrix[:, 0]
            smooth_fit = qbg @ (qbg.T @ (noisy - amp * mask))
            reconstruction = smooth_fit + amp * mask
            noise_examples[float(sigma)] = {
                "noisy": noisy,
                "fit": {
                    "start": int(library.starts[idx]),
                    "length": int(library.lengths[idx]),
                    "amplitude": amp,
                    "reconstruction": reconstruction,
                },
            }
    with (tables / "noise_recovery.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(noise_rows[0]))
        writer.writeheader()
        writer.writerows(noise_rows)

    # Off-grid study.
    offgrid_shift = 0.37
    offgrid_rows: list[dict[str, object]] = []
    for size in [48, 72, 96, 120, 144, 192]:
        xx = sample_grid(size)
        dx = 2.0 * PI / size
        half_width = EDGE_HALF_WIDTH + offgrid_shift * dx
        yy = smooth_seed(xx) + jump_packet(xx, half_width=half_width)
        lib = CandidateLibrary.build(size, SMOOTH_DEGREE, min_length=max(4, size // 16), max_length=size //
 2)
        fit = lib.search(yy)
        start = int(fit["start"])
        length = int(fit["length"])
        left_estimate = -PI + start * dx
        right_estimate = -PI + ((start + length) % size) * dx
        # For this nonwrapping worked interval, unwrap right endpoint.
        if right_estimate <= left_estimate:
            right_estimate += 2.0 * PI
```

```
        true_left = -half_width
        true_right = half_width
        offgrid_rows.append(
            {
                "size": size,
                "dx": dx,
                "true_left": true_left,
                "true_right": true_right,
                "estimated_left": left_estimate,
                "estimated_right": right_estimate,
                "max_endpoint_error": max(abs(left_estimate - true_left), abs(right_estimate - true_right)),
                "reconstruction_rmse": rmse(np.asarray(fit["reconstruction"]), yy),
            }
        )
    with (tables / "offgrid_recovery.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(offgrid_rows[0]))
        writer.writeheader()
        writer.writerows(offgrid_rows)

    # Runtime scaling for the one finite search implementation.
    runtime_rows: list[dict[str, object]] = []
    runtime_sizes = [48, 64, 80, 96, 112, 128] if quick else [48, 64, 80, 96, 112, 128, 144, 160]
    for size in runtime_sizes:
        t0 = time.perf_counter()
        lib = CandidateLibrary.build(size, SMOOTH_DEGREE, min_length=max(4, size // 16), max_length=size //
 2)
        build_seconds = time.perf_counter() - t0
        xx, ss, qq, yy = true_discrete_signal(size)
        t1 = time.perf_counter()
        fit = lib.search(yy)
        search_seconds = time.perf_counter() - t1
        runtime_rows.append(
            {
                "size": size,
                "candidate_count": int(lib.masks.shape[0]),
                "build_seconds": build_seconds,
                "search_seconds": search_seconds,
                "correct": bool(int(fit["start"]) == size // 3 and int(fit["length"]) == size // 3),
            }
        )
    with (tables / "runtime_scaling.csv").open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(runtime_rows[0]))
        writer.writeheader()
        writer.writerows(runtime_rows)

    # ------------------------------------------------------------------
    # Figures 01-60
    # ------------------------------------------------------------------
    # 01 components
    fig, ax = plt.subplots(figsize=(11.5, 4.6))
    ax.plot(x_dense, s_dense, lw=2.0, label="smooth seed s(x)", color=BLUE)
    ax.plot(x_dense, j_dense, lw=2.0, label="edge packet j(x)", color=ORANGE)
    ax.plot(x_dense, f_dense, lw=2.3, label="full signal f=s+j", color=INK)
    ax.axvline(-EDGE_HALF_WIDTH, ls="--", lw=1.1, color=MUTED)
    ax.axvline(EDGE_HALF_WIDTH, ls="--", lw=1.1, color=MUTED)
    ax.set_xlabel("x")
    ax.set_ylabel("amplitude")
    ax.set_title("The one signal used throughout the entire monograph", weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0, 0, 0.82, 1])
    save_figure(fig, figures, "01_signal_components")

    # 02 seed-to-reconstruction diagram
    fig, ax = plt.subplots(figsize=(12.5, 5.1))
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    stages = [
        (0.02, "seed\n(smooth + edge)", BLUE),
        (0.19, "sample\nor observe", CYAN),
```

```
        (0.36, "analyze\n(P_s, P_j)", TEAL),
        (0.53, "operate\nS_N P_s + P_j", ORANGE),
        (0.70, "synthesize\nadd packets", MAGENTA),
        (0.87, "residual\ncheck", GREEN),
    ]
    for xpos, label, color in stages:
        diagram_box(ax, xpos, 0.42, 0.12, 0.22, label, color, 12)
    for i in range(len(stages) - 1):
        diagram_arrow(ax, (stages[i][0] + 0.12, 0.53), (stages[i + 1][0], 0.53))
    ax.text(0.5, 0.85, "From the signal seed to reconstruction", ha="center", fontsize=22, weight="bold")
    ax.text(0.5, 0.18, "The discontinuity is never forced through the global Fourier truncation block.", ha=
     "center", fontsize=13, color=MUTED)
    save_figure(fig, figures, "02_closed_pipeline")

    # 03 packet split
    fig, axes = plt.subplots(3, 1, figsize=(11.5, 7.0), sharex=True)
    axes[0].plot(x_dense, f_dense, color=INK, lw=2.2)
    axes[0].set_title("Exact model-space packet decomposition", loc="left", weight="bold")
    axes[0].set_ylabel("f")
    axes[1].plot(x_dense, s_dense, color=BLUE, lw=2.0)
    axes[1].set_ylabel("P_s f")
    axes[2].plot(x_dense, j_dense, color=ORANGE, lw=2.0)
    axes[2].set_ylabel("P_j f")
    axes[2].set_xlabel("x")
    for axr in axes: style_axis(axr)
    fig.tight_layout()
    save_figure(fig, figures, "03_exact_packet_split")

    # 04 basis atoms
    fig, axes = plt.subplots(3, 2, figsize=(11.5, 7.0), sharex=True)
    atoms = [
        (np.ones_like(x_dense), "1"),
        (np.cos(2*x_dense), "cos(2x)"),
        (np.sin(3*x_dense), "sin(3x)"),
        (np.cos(5*x_dense), "cos(5x)"),
        (s_dense, "assembled smooth seed"),
        (j_dense, "edge packet"),
    ]
    for axr, (vals, title) in zip(axes.ravel(), atoms, strict=True):
        axr.plot(x_dense, vals, lw=1.8, color=COLORS[len(axr.lines) % len(COLORS)] if False else BLUE)
        axr.set_title(title, weight="bold", fontsize=11)
        style_axis(axr)
    fig.suptitle("Primitive atoms and the two signal packets", fontsize=17, weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig, figures, "04_seed_atoms")

    # 05 jump geometry
    fig, ax = plt.subplots(figsize=(11.5, 4.4))
    ax.fill_between(x_dense, 0, j_dense, color=ORANGE, alpha=.35, label="packet support")
    ax.plot(x_dense, j_dense, color=ORANGE, lw=2.2)
    ax.scatter([-EDGE_HALF_WIDTH, EDGE_HALF_WIDTH], [1, 0], s=55, color=[TEAL, RED], zorder=4)
    ax.annotate("left jump", (-EDGE_HALF_WIDTH,1), xytext=(-2.4,1.18), arrowprops={"arrowstyle":"->"})
    ax.annotate("right jump", (EDGE_HALF_WIDTH,0), xytext=(1.4,.25), arrowprops={"arrowstyle":"->"})
    ax.set_ylim(-.2,1.35)
    ax.set_xlabel("x")
    ax.set_ylabel("j(x)")
    ax.set_title("The singular packet is a geometric object, not a cloud of Fourier modes", weight="bold")
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig, figures, "05_jump_geometry")

    # 06 exact coefficients
    ns = np.arange(-24,25)
    cs = np.array([smooth_fourier_coefficient(int(n)) for n in ns])
    cj = np.array([jump_fourier_coefficient(int(n)) for n in ns])
    fig, axes = plt.subplots(2,1,figsize=(11.8,6.7),sharex=True)
    axes[0].stem(ns, np.abs(cs), linefmt=BLUE, markerfmt="o", basefmt=" ")
    axes[0].set_ylabel("|s-hat_n|")
    axes[0].set_title("Exact smooth coefficients terminate at degree five", loc="left", weight="bold")
    axes[1].stem(ns, np.abs(cj), linefmt=ORANGE, markerfmt="o", basefmt=" ")
    axes[1].set_ylabel("|j-hat_n|")
```

```
    axes[1].set_xlabel("Fourier index n")
    axes[1].set_title("The jump packet has an algebraic 1/|n| tail", loc="left", weight="bold")
    for axr in axes: style_axis(axr)
    fig.tight_layout()
    save_figure(fig, figures, "06_exact_fourier_coefficients")

    # 07 residue classes
    positive_n = np.arange(1,37)
    vals = np.array([math.sin(n*EDGE_HALF_WIDTH) for n in positive_n])
    fig, ax = plt.subplots(figsize=(11.5,4.5))
    colors = [COLORS[n%6] for n in positive_n]
    ax.bar(positive_n, vals, color=colors, edgecolor=INK, linewidth=.4)
    ax.set_xlabel("n")
    ax.set_ylabel("sin(n pi/3)")
    ax.set_title("The jump coefficients split into six exact residue packets", weight="bold")
    ax.set_xticks(np.arange(0,37,3))
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig, figures, "07_coefficient_residue_classes")

    # 08 coefficient decay
    nn = np.arange(1,160)
    jj = np.abs(np.array([jump_fourier_coefficient(int(n)) for n in nn]))
    envelope = 1.0/(PI*nn)
    fig, ax = plt.subplots(figsize=(11.5,4.5))
    ax.loglog(nn, jj + 1e-18, label="|j-hat_n|", color=ORANGE, lw=1.7)
    ax.loglog(nn, envelope, label="1/(pi n) envelope", color=INK, ls="--")
    ax.set_xlabel("n")
    ax.set_ylabel("magnitude")
    ax.set_title("Slow spectral decay is the signature of the discontinuity", weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig, figures, "08_coefficient_decay")

    # 09 kernels
    t_kernel = np.linspace(-.7,.7,20000)
    fig, axes = plt.subplots(2,2,figsize=(11.8,7.0),sharex=True)
    for axr, order in zip(axes.ravel(), [5,10,20,40], strict=True):
        axr.plot(t_kernel, dirichlet_kernel(t_kernel,order), color=BLUE, lw=1.5)
        axr.set_title(f"D_{order}(t)", weight="bold")
        style_axis(axr)
    fig.suptitle("Dirichlet kernels sharpen and oscillate rather than becoming positive", fontsize=17,
     weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig, figures, "09_dirichlet_kernels")

    # 10 convolution at edge
    u = np.linspace(-.6,.6,20000)
    D = dirichlet_kernel(u,40)/(2*PI)
    fig, ax = plt.subplots(figsize=(11.5,4.7))
    ax.plot(u,D,color=BLUE,lw=1.7,label="normalized D_40")
    ax.fill_between(u,0,D,where=(u<0),color=ORANGE,alpha=.25,label="mass sampled from inside packet")
    ax.axvline(0,color=INK,lw=1)
    ax.set_xlabel("local coordinate around the edge")
    ax.set_ylabel("kernel weight")
    ax.set_title("The truncated Fourier operator mixes both sides of the discontinuity", weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.79,1])
    save_figure(fig, figures, "10_edge_convolution")

    # 11-14 full partial sums
    for fig_no, order in zip(range(11,15), [5,10,20,40], strict=True):
        approx = raw_fourier_partial_sum(x_dense,order)
        fig, ax = plt.subplots(figsize=(11.5,4.5))
        ax.plot(x_dense,f_dense,color=INK,lw=2.1,label="exact f")
        ax.plot(x_dense,approx,color=BLUE,lw=1.3,label=f"raw S_{order} f")
        ax.axvline(-EDGE_HALF_WIDTH,ls="--",color=MUTED,lw=.9)
        ax.axvline(EDGE_HALF_WIDTH,ls="--",color=MUTED,lw=.9)
        ax.set_xlabel("x")
```

```
        ax.set_ylabel("amplitude")
        ax.set_title(f"Global Fourier truncation at order N={order}", weight="bold")
        style_axis(ax)
        add_external_legend(ax)
        fig.tight_layout(rect=[0,0,.82,1])
        save_figure(fig, figures, f"{fig_no:02d}_raw_partial_N{order}")

    # 15-17 edge zooms
    for fig_no, order in zip(range(15,18), [10,20,40], strict=True):
        approx = raw_fourier_partial_sum(x_dense,order)
        local = (x_dense>EDGE_HALF_WIDTH-.35)&(x_dense<EDGE_HALF_WIDTH+.35)
        fig, ax = plt.subplots(figsize=(11.5,4.5))
        ax.plot(x_dense[local],f_dense[local],color=INK,lw=2.2,label="exact f")
        ax.plot(x_dense[local],approx[local],color=BLUE,lw=1.5,label=f"S_{order}f")
        ax.axvline(EDGE_HALF_WIDTH,ls="--",color=RED,lw=1.0,label="true edge")
        ax.set_xlabel("x")
        ax.set_ylabel("amplitude")
        ax.set_title(f"The oscillation compresses toward the edge but its first overshoot persists (N={order
    })", weight="bold")
        style_axis(ax)
        add_external_legend(ax)
        fig.tight_layout(rect=[0,0,.80,1])
        save_figure(fig, figures, f"{fig_no:02d}_edge_zoom_N{order}")

    # 18 scaled profile collapse
    z = np.linspace(-10,10,5000)
    fig, ax = plt.subplots(figsize=(11.5,4.8))
    for order,color in zip([20,40,80,160],[BLUE,TEAL,ORANGE,MAGENTA],strict=True):
        xlocal = EDGE_HALF_WIDTH - z/(order+0.5)
        ax.plot(z,jump_fourier_partial_sum(xlocal,order),lw=1.4,label=f"N={order}",color=color)
    ax.plot(z,gibbs_profile(z),color=INK,lw=2.2,ls="--",label="limiting sine-integral profile")
    ax.axvline(PI,color=RED,ls=":",lw=1)
    ax.set_xlabel("scaled interior coordinate z=(N+1/2)(a-x)")
    ax.set_ylabel("partial sum of jump")
    ax.set_title("Edge profiles collapse to a universal scaled limit", weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.79,1])
    save_figure(fig, figures, "18_scaled_profile_collapse")

    # 19 Gibbs constant convergence
    Nvals=np.arange(5,301)
    observed=[]
    for order in Nvals:
        xstar=EDGE_HALF_WIDTH-PI/(order+.5)
        observed.append(float(jump_fourier_partial_sum(np.array([xstar]),int(order))[0]-1.0))
    fig,ax=plt.subplots(figsize=(11.5,4.5))
    ax.plot(Nvals,observed,color=BLUE,lw=1.7,label="first scaled overshoot")
    ax.axhline(gibbs_constant(),color=RED,ls="--",lw=1.5,label=f"G={gibbs_constant():.12f}")
    ax.set_xlabel("N")
    ax.set_ylabel("overshoot above unit jump")
    ax.set_title("The first overshoot converges to the Gibbs constant",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.79,1])
    save_figure(fig,figures,"19_gibbs_constant_convergence")

    # curves from metrics
    def metric_series(method: str, field: str) -> tuple[np.ndarray,np.ndarray]:
        rows=[r for r in metric_rows if r["method"]==method]
        return np.array([r["order"] for r in rows]),np.array([r[field] for r in rows],float)

    # 20 overshoot
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    for method,color in zip(methods,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
        xxm,yym=metric_series(method,"jump_overshoot")
        ax.plot(xxm,yym,marker="o",ms=3,lw=1.6,label=method,color=color)
    ax.axhline(gibbs_constant(),color=RED,ls="--",label="Gibbs constant")
    ax.set_xlabel("spectral order N")
    ax.set_ylabel("jump overshoot")
    ax.set_title("Packet retention removes jump overshoot exactly",weight="bold")
```

```
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"20_overshoot_vs_order")

    # 21 RMSE
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    for method,color in zip(methods,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
        xxm,yym=metric_series(method,"rmse")
        ax.semilogy(xxm,np.maximum(yym,1e-17),marker="o",ms=3,lw=1.6,label=method,color=color)
    ax.set_xlabel("spectral order N")
    ax.set_ylabel("global RMSE")
    ax.set_title("Global spectral error decreases, but only packet retention is exact at N>=5",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"21_rmse_vs_order")

    # 22 boundary rmse
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    for method,color in zip(methods,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
        xxm,yym=metric_series(method,"boundary_rmse")
        ax.semilogy(xxm,np.maximum(yym,1e-17),marker="o",ms=3,lw=1.6,label=method,color=color)
    ax.set_xlabel("spectral order N")
    ax.set_ylabel("boundary-band RMSE")
    ax.set_title("The decisive gain is localized at the discontinuity",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"22_boundary_rmse_vs_order")

    # 23-25 filter jump comparisons
    for fig_no,method,color in [(23,"fejer",TEAL),(24,"lanczos",ORANGE),(25,"exponential",PURPLE)]:
        order=40
        approx=filtered_jump_sum(x_dense,order,method)
        local=(x_dense>EDGE_HALF_WIDTH-.45)&(x_dense<EDGE_HALF_WIDTH+.45)
        fig,ax=plt.subplots(figsize=(11.5,4.5))
        ax.plot(x_dense[local],j_dense[local],color=INK,lw=2.2,label="exact jump")
        ax.plot(x_dense[local],approx[local],color=color,lw=1.6,label=f"{method} reconstruction")
        ax.axvline(EDGE_HALF_WIDTH,color=RED,ls="--",lw=1,label="edge")
        ax.set_xlabel("x")
        ax.set_ylabel("jump amplitude")
        ax.set_title(f"{method.capitalize()} filtering trades oscillation for a blurred transition",weight="bold")
        style_axis(ax)
        add_external_legend(ax)
        fig.tight_layout(rect=[0,0,.80,1])
        save_figure(fig,figures,f"{fig_no:02d}_{method}_jump")

    # 26 full comparison
    order=40
    fig,ax=plt.subplots(figsize=(11.5,4.8))
    ax.plot(x_dense,f_dense,color=INK,lw=2.2,label="exact f")
    for method,color in zip(methods,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
        approx=packet_reconstruction(x_dense,order) if method=="packet" else filtered_full_sum(x_dense,order,method)
        ax.plot(x_dense,approx,lw=1.15,label=method,color=color,alpha=.9)
    ax.set_xlabel("x")
    ax.set_ylabel("amplitude")
    ax.set_title("Five reconstruction operators applied to the same signal",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.78,1])
    save_figure(fig,figures,"26_method_comparison_full")

    # 27 edge comparison
    local=(x_dense>EDGE_HALF_WIDTH-.28)&(x_dense<EDGE_HALF_WIDTH+.28)
    fig,ax=plt.subplots(figsize=(11.5,4.8))
    ax.plot(x_dense[local],f_dense[local],color=INK,lw=2.2,label="exact f")
    for method,color in zip(methods,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
```

```
        approx=packet_reconstruction(x_dense,order) if method=="packet" else filtered_full_sum(x_dense,order,method)
        ax.plot(x_dense[local],approx[local],lw=1.4,label=method,color=color)
    ax.axvline(EDGE_HALF_WIDTH,color=RED,ls="--",lw=1)
    ax.set_xlabel("x")
    ax.set_ylabel("amplitude")
    ax.set_title("Only the packet operator preserves the exact discontinuity",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.78,1])
    save_figure(fig,figures,"27_method_comparison_edge")

    # 28 metric heatmap at N=40
    fields=["rmse","boundary_rmse","jump_overshoot","jump_transition_width"]
    matrix=np.array([[next(float(r[field]) for r in metric_rows if r["method"]==method and r["order"]==40) for field in fields] for method in methods])
    # normalize per column for visibility
    normalized=matrix/np.maximum(matrix.max(axis=0,keepdims=True),1e-16)
    fig,ax=plt.subplots(figsize=(10.8,5.0))
    im=ax.imshow(normalized,cmap="viridis",aspect="auto",vmin=0,vmax=1)
    ax.set_xticks(range(len(fields)),["global RMSE","boundary RMSE","overshoot","transition width"])
    ax.set_yticks(range(len(methods)),methods)
    for i in range(len(methods)):
        for j in range(len(fields)):
            rgba=im.cmap(im.norm(normalized[i,j]))
            lum=.2126*rgba[0]+.7152*rgba[1]+.0722*rgba[2]
            ax.text(j,i,f"{matrix[i,j]:.3e}",ha="center",va="center",color=INK if lum>.55 else WHITE,fontsize=9)
    fig.colorbar(im,ax=ax,fraction=.035,pad=.03,label="column-normalized cost")
    ax.set_title("Method tradeoffs at N=40",weight="bold")
    fig.tight_layout()
    save_figure(fig,figures,"28_metric_heatmap")

    # 29 packet operator diagram
    fig,ax=plt.subplots(figsize=(12.5,5.2))
    ax.set_xlim(0,1)
    ax.set_ylim(0,1)
    ax.axis("off")
    diagram_box(ax,.04,.42,.14,.20,"f",INK,16)
    diagram_box(ax,.28,.62,.18,.18,"smooth packet\nP_s f",BLUE,13)
    diagram_box(ax,.28,.24,.18,.18,"edge packet\nP_j f",ORANGE,13)
    diagram_box(ax,.57,.62,.18,.18,"Fourier block\nS_N P_s f",TEAL,13)
    diagram_box(ax,.57,.24,.18,.18,"identity block\nP_j f",MAGENTA,13)
    diagram_box(ax,.84,.42,.12,.20,"sum\nR_N f",GREEN,14)
    diagram_arrow(ax,(.18,.52),(.28,.71),BLUE,"P_s")
    diagram_arrow(ax,(.18,.50),(.28,.33),ORANGE,"P_j")
    diagram_arrow(ax,(.46,.71),(.57,.71),TEAL)
    diagram_arrow(ax,(.46,.33),(.57,.33),MAGENTA)
    diagram_arrow(ax,(.75,.71),(.84,.55),TEAL)
    diagram_arrow(ax,(.75,.33),(.84,.48),MAGENTA)
    ax.text(.5,.90,"Resolved reconstruction operator R_N=S_N P_s+P_j",ha="center",fontsize=21,weight="bold")
    ax.text(.5,.08,"The singular packet bypasses the global spectral truncation block.",ha="center",fontsize=12,color=MUTED)
    save_figure(fig,figures,"29_packet_operator_pipeline")

    # 30 exact packet reconstruction
    packet40=packet_reconstruction(x_dense,40)
    fig,ax=plt.subplots(figsize=(11.5,4.5))
    ax.plot(x_dense,f_dense,color=INK,lw=2.4,label="exact f")
    ax.plot(x_dense,packet40,color=MAGENTA,lw=1.2,ls="--",label="R_40 f")
    ax.set_xlabel("x")
    ax.set_ylabel("amplitude")
    ax.set_title("The resolved operator reconstructs this model signal exactly",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"30_packet_exact_reconstruction")

    # 31 packet residual
    residual=packet40-f_dense
    fig,ax=plt.subplots(figsize=(11.5,4.2))
```

```
    ax.plot(x_dense,residual,color=RED,lw=1.5)
    ax.axhline(0,color=INK,lw=.8)
    ax.set_xlabel("x")
    ax.set_ylabel("R_40 f - f")
    ax.set_title(f"Exact residual: max |r| = {np.max(np.abs(residual)):.3e}",weight="bold")
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig,figures,"31_packet_residual")

    # 32 finite samples
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    ax.plot(x_dense,f_dense,color=GRID if False else MUTED,lw=1.2,label="continuous signal")
    ax.scatter(x_grid,y_grid,s=18,color=BLUE,label="96 samples",zorder=3)
    ax.set_xlabel("x_m")
    ax.set_ylabel("y_m")
    ax.set_title("The recovery stage begins only with sampled values",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"32_finite_grid_samples")

    # 33 design matrix
    fig,ax=plt.subplots(figsize=(11.5,5.2))
    im=ax.imshow(library.design.T,aspect="auto",cmap="coolwarm",vmin=-1,vmax=1)
    labels=["1"]+[v for k in range(1,6) for v in (f"cos {k}x",f"sin {k}x")]
    ax.set_yticks(range(len(labels)),labels)
    ax.set_xlabel("sample index m")
    ax.set_title("The finite smooth-background design matrix",weight="bold")
    fig.colorbar(im,ax=ax,fraction=.025,pad=.02,label="basis value")
    fig.tight_layout()
    save_figure(fig,figures,"33_background_design_matrix")

    # 34 residualized signal and mask
    true_mask_residual=library.residual_masks[int(exact_search["index"])]
    fig,axes=plt.subplots(2,1,figsize=(11.5,6.2),sharex=True)
    axes[0].stem(np.arange(DEFAULT_GRID),np.asarray(exact_search["y_perp"]),linefmt=BLUE,markerfmt=" ",
     basefmt=" ")
    axes[0].set_ylabel("(I-P_B)y")
    axes[0].set_title("Removing the smooth subspace leaves a one-dimensional edge direction",loc="left",
     weight="bold")
    axes[1].stem(np.arange(DEFAULT_GRID),true_mask_residual,linefmt=ORANGE,markerfmt=" ",basefmt=" ")
    axes[1].set_ylabel("(I-P_B)q_*")
    axes[1].set_xlabel("sample index")
    for axr in axes:style_axis(axr)
    fig.tight_layout()
    save_figure(fig,figures,"34_residualized_signal_and_mask")

    # 35 objective sorted
    sorted_res=np.sort(all_residuals[np.isfinite(all_residuals)])
    fig,ax=plt.subplots(figsize=(11.5,4.5))
    ax.semilogy(np.arange(min(600,len(sorted_res))),np.maximum(sorted_res[:600],1e-18),color=BLUE,lw=1.7)
    ax.scatter([0],[max(sorted_res[0],1e-18)],color=RED,s=50,label="true interval")
    ax.axhline(float(exact_search["second_residual"]),color=ORANGE,ls="--",label="second-best residual")
    ax.set_xlabel("candidate rank")
    ax.set_ylabel("profile residual")
    ax.set_title("The true packet is separated by a large finite objective gap",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.80,1])
    save_figure(fig,figures,"35_candidate_objective_ranking")

    # 36 start-length heatmap
    fig,ax=plt.subplots(figsize=(11.5,5.5))
    logged=np.log10(np.maximum(objective_grid,1e-16))
    im=ax.imshow(logged,origin="lower",aspect="auto",cmap="magma",extent=[0,DEFAULT_GRID,0,max_len])
    ax.scatter([TRUE_START],[TRUE_LENGTH],s=80,facecolor="none",edgecolor=WHITE,lw=1.8,label="true packet")
    ax.set_xlabel("candidate start")
    ax.set_ylabel("candidate length")
    ax.set_title("Profile least-squares objective over the complete interval library",weight="bold")
    fig.colorbar(im,ax=ax,fraction=.035,pad=.02,label="log10 residual")
    ax.legend(loc="upper right")
```

```
    fig.tight_layout()
    save_figure(fig,figures,"36_start_length_objective")

    # 37 gap bars
    fig,ax=plt.subplots(figsize=(9.5,4.5))
    vals=[max(float(exact_search["residual"]),1e-18),float(exact_search["second_residual"]),float(exact_search["gap"])]
    ax.bar([0,1,2],vals,color=[TEAL,ORANGE,MAGENTA],edgecolor=INK)
    ax.set_yscale("log")
    ax.set_xticks([0,1,2],["best residual","second residual","identifiability gap"])
    for i,v in enumerate(vals):ax.text(i,v*1.35,f"{v:.3e}",ha="center",fontsize=10)
    ax.set_title("Finite identifiability gap",weight="bold")
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig,figures,"37_identifiability_gap")

    # 38 recovered components
    fig,axes=plt.subplots(3,1,figsize=(11.5,7.1),sharex=True)
    axes[0].plot(x_grid,y_grid,color=INK,lw=1.5,marker="o",ms=3)
    axes[0].set_ylabel("data")
    axes[0].set_title("Recovered components from sampled data alone",loc="left",weight="bold")
    axes[1].plot(x_grid,np.asarray(exact_search["smooth"]),color=BLUE,lw=1.8,label="recovered smooth")
    axes[1].plot(x_grid,s_grid,color=INK,lw=.9,ls="--",label="true smooth")
    axes[1].set_ylabel("smooth")
    axes[1].legend(ncol=2)
    axes[2].step(x_grid,float(exact_search["amplitude"])*np.asarray(exact_search["mask"]),where="post",color=ORANGE,lw=2,label="recovered edge")
    axes[2].step(x_grid,edge_grid,where="post",color=INK,lw=.9,ls="--",label="true edge")
    axes[2].set_ylabel("edge")
    axes[2].set_xlabel("x_m")
    axes[2].legend(ncol=2)
    for axr in axes:style_axis(axr)
    fig.tight_layout()
    save_figure(fig,figures,"38_recovered_components")

    # 39 coefficients recovered
    labels=["1"]+[v for k in range(1,6) for v in (f"c{k}",f"s{k}")]
    true_theta=np.zeros(11)
    true_theta[0]=.25
    true_theta[3]=.4
    true_theta[6]=-(1/3)
    true_theta[9]=.2
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    pos=np.arange(len(labels))
    width=.38
    ax.bar(pos-width/2,true_theta,width,label="true",color=INK,alpha=.75)
    ax.bar(pos+width/2,exact_coefficients,width,label="recovered",color=BLUE,alpha=.85)
    ax.set_xticks(pos,labels)
    ax.set_ylabel("coefficient")
    ax.set_title("The smooth seed coefficients are recovered after edge separation",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.84,1])
    save_figure(fig,figures,"39_recovered_coefficients")

    # 40 grid reconstruction
    fig,axes=plt.subplots(2,1,figsize=(11.5,6.0),sharex=True)
    axes[0].plot(x_grid,y_grid,color=INK,lw=2,label="data")
    axes[0].plot(x_grid,np.asarray(exact_search["reconstruction"]),color=MAGENTA,lw=1.1,ls="--",label="reconstruction")
    axes[0].legend(ncol=2)
    axes[0].set_ylabel("amplitude")
    axes[0].set_title("Data, recovered packets, and reconstructed signal",loc="left",weight="bold")
    rr=np.asarray(exact_search["reconstruction"])-y_grid
    axes[1].stem(np.arange(DEFAULT_GRID),rr,linefmt=RED,markerfmt=" ",basefmt=" ")
    axes[1].set_ylabel("residual")
    axes[1].set_xlabel("sample index")
    axes[1].text(.99,.82,f"max residual = {np.max(np.abs(rr)):.3e}",transform=axes[1].transAxes,ha="right",weight="bold")
    for axr in axes:style_axis(axr)
    fig.tight_layout()
```

```
    save_figure(fig,figures,"40_closed_grid_reconstruction")

    # 41 one-sample mislocation
    wrong_mask=interval_mask(DEFAULT_GRID,TRUE_START+1,TRUE_LENGTH)
    wrong_fit=s_grid+wrong_mask
    fig,axes=plt.subplots(2,1,figsize=(11.5,6.2),sharex=True)
    axes[0].plot(x_grid,y_grid,color=INK,lw=2,label="true")
    axes[0].step(x_grid,wrong_fit,where="mid",color=RED,lw=1.5,label="edge shifted by one sample")
    axes[0].legend(ncol=2)
    axes[0].set_ylabel("amplitude")
    axes[0].set_title("Negative control: one-sample packet mislocation",loc="left",weight="bold")
    axes[1].stem(np.arange(DEFAULT_GRID),wrong_fit-y_grid,linefmt=RED,markerfmt=" ",basefmt=" ")
    axes[1].set_ylabel("residual")
    axes[1].set_xlabel("sample index")
    for axr in axes:style_axis(axr)
    fig.tight_layout()
    save_figure(fig,figures,"41_mislocated_edge")

    # 42 amplitude mismatch
    amplitudes=[.75,.9,1.0,1.1,1.25]
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    for amp,color in zip(amplitudes,COLORS[:len(amplitudes)],strict=True):
        approx=s_grid+amp*edge_grid
        ax.plot(x_grid,approx-y_grid,lw=1.4,label=f"Ahat={amp:.2f}",color=color)
    ax.set_xlabel("x_m")
    ax.set_ylabel("residual")
    ax.set_title("Amplitude error remains confined to the chosen packet support",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.80,1])
    save_figure(fig,figures,"42_amplitude_mismatch")

    # 43 perturbation bound
    shift_values=np.arange(0,13)
    actual=[]
    bound=[]
    for shift in shift_values:
        q=interval_mask(DEFAULT_GRID,TRUE_START+int(shift),TRUE_LENGTH)
        actual.append(np.linalg.norm(edge_grid-q))
        symmetric_difference=np.count_nonzero(edge_grid!=q)
        bound.append(math.sqrt(symmetric_difference))
    fig,ax=plt.subplots(figsize=(11.5,4.5))
    ax.plot(shift_values,actual,marker="o",label="actual L2 packet error",color=BLUE)
    ax.plot(shift_values,bound,marker="s",ls="--",label="symmetric-difference bound",color=ORANGE)
    ax.set_xlabel("edge shift in samples")
    ax.set_ylabel("error")
    ax.set_title("Support error has an exact geometric formula",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.80,1])
    save_figure(fig,figures,"43_packet_perturbation_bound")

    # 44 noise examples
    fig,axes=plt.subplots(3,1,figsize=(11.5,8.0),sharex=True)
    for axr,(sigma,record),color in zip(axes,sorted(noise_examples.items()),[TEAL,ORANGE,RED],strict=True):
        noisy=np.asarray(record["noisy"])
        fit=record["fit"]
        axr.scatter(x_grid,noisy,s=14,color=MUTED,label="noisy data")
        axr.plot(x_grid,np.asarray(fit["reconstruction"]),lw=1.8,color=color,label=f"fit: start={fit['start
    ']}, length={fit['length']}")
        axr.plot(x_grid,y_grid,lw=.9,ls="--",color=INK,label="noise-free truth")
        axr.set_ylabel(f"sigma={sigma:.2f}")
        style_axis(axr)
        axr.legend(loc="upper left",bbox_to_anchor=(1.01,1),fontsize=8)
    axes[-1].set_xlabel("x_m")
    fig.suptitle("The same finite search under three noise levels",fontsize=17,weight="bold")
    fig.tight_layout(rect=[0,0,.78,.94])
    save_figure(fig,figures,"44_noisy_recovery_examples")

    # 45-48 noise curves
    sig=np.array([r["sigma"] for r in noise_rows],float)
```

```
    curve_specs=[
        (45,"exact_support_rate","exact support recovery rate","probability",TEAL),
        (46,"mean_abs_start_error_samples","mean absolute start error","samples",BLUE),
        (47,"mean_abs_amplitude_error","mean absolute amplitude error","amplitude",ORANGE),
        (48,"mean_boundary_rmse","mean boundary-band reconstruction RMSE","RMSE",MAGENTA),
    ]
    for no,field,title,ylabel,color in curve_specs:
        yy=np.array([r[field] for r in noise_rows],float)
        fig,ax=plt.subplots(figsize=(11.5,4.5))
        ax.plot(sig,yy,marker="o",lw=2,color=color)
        ax.set_xlabel("noise standard deviation")
        ax.set_ylabel(ylabel)
        ax.set_title(title,weight="bold")
        if field=="exact_support_rate":ax.set_ylim(-.03,1.03)
        style_axis(ax)
        fig.tight_layout()
        save_figure(fig,figures,f"{no:02d}_{field}")

    # 49 off-grid example at M=96
    size=96
    xx=sample_grid(size)
    dx=2*PI/size
    half_width=EDGE_HALF_WIDTH+offgrid_shift*dx
    yy=smooth_seed(xx)+jump_packet(xx,half_width=half_width)
    fit=CandidateLibrary.build(size,SMOOTH_DEGREE).search(yy)
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    ax.scatter(xx,yy,s=18,color=INK,label="off-grid sampled signal")
    ax.plot(xx,np.asarray(fit["reconstruction"]),color=BLUE,lw=1.5,label="best grid packet")
    ax.axvline(-half_width,color=RED,ls="--",lw=1,label="continuous edge")
    ax.axvline(half_width,color=RED,ls="--",lw=1)
    ax.set_xlabel("x_m")
    ax.set_ylabel("amplitude")
    ax.set_title("Off-grid edges are localized to the sampling resolution",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.80,1])
    save_figure(fig,figures,"49_offgrid_example")

    # 50 offgrid convergence
    sizes=np.array([r["size"] for r in offgrid_rows])
    errors=np.array([r["max_endpoint_error"] for r in offgrid_rows])
    dxs=np.array([r["dx"] for r in offgrid_rows])
    fig,ax=plt.subplots(figsize=(11.5,4.6))
    ax.loglog(sizes,errors,marker="o",lw=2,label="endpoint error",color=BLUE)
    ax.loglog(sizes,dxs,ls="--",lw=1.5,label="grid spacing",color=ORANGE)
    ax.set_xlabel("sample count M")
    ax.set_ylabel("radians")
    ax.set_title("Off-grid localization error shrinks with grid refinement",weight="bold")
    style_axis(ax)
    add_external_legend(ax)
    fig.tight_layout(rect=[0,0,.82,1])
    save_figure(fig,figures,"50_offgrid_convergence")

    # 51 residual localization for wrong packet
    residual_wrong=wrong_fit-y_grid
    fig,ax=plt.subplots(figsize=(11.5,4.5))
    ax.bar(np.arange(DEFAULT_GRID),np.abs(residual_wrong),color=[RED if v>1e-12 else GRID for v in np.abs(
     residual_wrong)],edgecolor="none")
    ax.set_xlabel("sample index")
    ax.set_ylabel("|residual|")
    ax.set_title("A wrong packet localizes the unresolved obligation at the two boundaries",weight="bold")
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig,figures,"51_residual_localization")

    # 52 adaptive local refinement diagram
    fig,ax=plt.subplots(figsize=(12.5,5.1))
    ax.set_xlim(0,1)
    ax.set_ylim(0,1)
    ax.axis("off")
    diagram_box(ax,.04,.41,.15,.20,"coarse grid\nM=48",BLUE,13)
```

```
    diagram_box(ax,.29,.41,.17,.20,"residual scan\nedge packets",ORANGE,13)
    diagram_box(ax,.56,.58,.17,.17,"refine left\nboundary",TEAL,12)
    diagram_box(ax,.56,.24,.17,.17,"refine right\nboundary",TEAL,12)
    diagram_box(ax,.84,.41,.12,.20,"exact\npacket fit",MAGENTA,13)
    diagram_arrow(ax,(.19,.51),(.29,.51))
    diagram_arrow(ax,(.46,.52),(.56,.65),ORANGE)
    diagram_arrow(ax,(.46,.49),(.56,.32),ORANGE)
    diagram_arrow(ax,(.73,.65),(.84,.55),TEAL)
    diagram_arrow(ax,(.73,.32),(.84,.47),TEAL)
    ax.text(.5,.88,"Residual-driven refinement acts only where the packet geometry is unresolved",ha="center",fontsize=20,weight="bold")
    ax.text(.5,.09,"Global spectral resolution is not increased merely to sharpen two local edges.",ha="center",fontsize=12,color=MUTED)
    save_figure(fig,figures,"52_adaptive_local_refinement")

    # 53 block matrix of R_N relative to packets
    fig,ax=plt.subplots(figsize=(8.8,5.0))
    mat=np.array([[1,0],[0,1]],float)
    im=ax.imshow(mat,cmap=matplotlib.colors.ListedColormap([PAPER,TEAL]),vmin=0,vmax=1)
    ax.set_xticks([0,1],["smooth input","edge input"])
    ax.set_yticks([0,1],["smooth output","edge output"])
    labels=[["S_N|T_5 = I","0"],["0","I_edge"]]
    for i in range(2):
        for j in range(2):ax.text(j,i,labels[i][j],ha="center",va="center",fontsize=13,weight="bold")
    ax.set_title("The reconstruction operator is block diagonal on the chosen model",weight="bold")
    fig.tight_layout()
    save_figure(fig,figures,"53_packet_operator_blocks")

    # 54 coefficient polyphase
    residues=[]
    for r in range(6):
        nseq=np.arange(r if r>0 else 6,61,6)
        residues.append((r,nseq,np.array([jump_fourier_coefficient(int(n)).real for n in nseq])))
    fig,axes=plt.subplots(3,2,figsize=(11.5,7.2),sharey=True)
    for axr,(r,nseq,v) in zip(axes.ravel(),residues,strict=True):
        axr.stem(nseq,v,linefmt=COLORS[r],markerfmt=" ",basefmt=" ")
        axr.set_title(f"n mod 6 = {r}",weight="bold",fontsize=11)
        style_axis(axr)
    fig.suptitle("Polyphase refinement of the jump's Fourier coefficient sequence",fontsize=17,weight="bold")
    fig.tight_layout(rect=[0,0,1,.94])
    save_figure(fig,figures,"54_fourier_polyphase_packets")

    # 55 tradeoff plot
    n40=[r for r in metric_rows if r["order"]==40]
    fig,ax=plt.subplots(figsize=(9.5,5.4))
    for r,color in zip(n40,[BLUE,TEAL,ORANGE,PURPLE,MAGENTA],strict=True):
        ax.scatter(r["boundary_rmse"],r["jump_transition_width"],s=110,color=color,label=r["method"],edgecolor=INK)
        ax.text(r["boundary_rmse"]*1.04+1e-18,r["jump_transition_width"]*1.02+1e-18,r["method"],fontsize=9)
    ax.set_xscale("symlog",linthresh=1e-14)
    ax.set_yscale("symlog",linthresh=1e-14)
    ax.set_xlabel("boundary RMSE")
    ax.set_ylabel("10%-90% transition width")
    ax.set_title("Filtering trades ringing for blur. Packet separation changes which object is truncated",weight="bold")
    style_axis(ax)
    fig.tight_layout()
    save_figure(fig,figures,"55_method_tradeoff")

    # 56 runtime scaling
    rs=np.array([r["size"] for r in runtime_rows])
    cc=np.array([r["candidate_count"] for r in runtime_rows])
    bt=np.array([r["build_seconds"] for r in runtime_rows])
    st=np.array([r["search_seconds"] for r in runtime_rows])
    fig,axes=plt.subplots(1,2,figsize=(11.8,4.8))
    axes[0].plot(rs,cc,marker="o",color=BLUE)
    axes[0].set_xlabel("M")
    axes[0].set_ylabel("candidate intervals")
    axes[0].set_title("finite search-space size",weight="bold")
    style_axis(axes[0])
```

```
    axes[1].plot(rs,bt,marker="o",label="library build",color=ORANGE)
    axes[1].plot(rs,st,marker="o",label="profile search",color=TEAL)
    axes[1].set_xlabel("M")
    axes[1].set_ylabel("seconds")
    axes[1].set_title("measured reference runtime",weight="bold")
    style_axis(axes[1])
    axes[1].legend()
    fig.suptitle("The exhaustive solver checks every finite candidate. It is not claimed to be asymptotically optimal",fontsize=15.5,weight="bold")
    fig.tight_layout(rect=[0,0,1,.91])
    save_figure(fig,figures,"56_runtime_scaling")

    # 57 proof dependency
    fig,ax=plt.subplots(figsize=(12.5,6.0))
    ax.set_xlim(0,1)
    ax.set_ylim(0,1)
    ax.axis("off")
    nodes=[(.06,.72,"Fourier coefficients",BLUE),(.29,.72,"Dirichlet kernel",BLUE),(.53,.72,"scaled edge limit",ORANGE),(.77,.72,"Gibbs constant",RED),(.06,.32,"direct-sum model",TEAL),(.29,.32,"packet projectors",TEAL),(.53,.32,"resolved operator",MAGENTA),(.77,.32,"exact reconstruction",GREEN)]
    for x0,y0,label,color in nodes:diagram_box(ax,x0,y0,.16,.14,label,color,11)
    for a0,b0 in [((.22,.79),(.29,.79)),((.45,.79),(.53,.79)),((.69,.79),(.77,.79)),((.22,.39),(.29,.39)),((.45,.39),(.53,.39)),((.69,.39),(.77,.39))]:diagram_arrow(ax,a0,b0)
    diagram_arrow(ax,(.85,.65),(.85,.46),RED,"problem")
    ax.text(.5,.93,"Two proof chains meet at the final comparison",ha="center",fontsize=21,weight="bold")
    ax.text(.5,.08,"The Gibbs theorem explains why raw truncation fails. The packet theorem explains why the packet reconstruction succeeds.",ha="center",fontsize=11.5,color=MUTED)
    save_figure(fig,figures,"57_proof_dependency_map")

    # 58 test dashboard placeholder built from actual tests below after execution
    # 59 summary wheel
    fig,ax=plt.subplots(figsize=(7.8,7.8))
    ax.set_aspect("equal")
    ax.axis("off")
    labels=[("seed",BLUE),("analyze",CYAN),("project",TEAL),("detect",GREEN),("operate",YELLOW),("refine",ORANGE),("synthesize",MAGENTA),("certify",PURPLE)]
    theta=np.linspace(90,-270,len(labels)+1)
    for i,(lab,color) in enumerate(labels):
        wedge=Wedge((0,0),1.0,theta[i+1],theta[i],width=.34,facecolor=color,edgecolor=PAPER,lw=2)
        ax.add_patch(wedge)
        ang=np.deg2rad((theta[i]+theta[i+1])/2)
        ax.text(.80*np.cos(ang),.80*np.sin(ang),lab,ha="center",va="center",rotation=(theta[i]+theta[i+1])/2-90,rotation_mode="anchor",color=WHITE,fontsize=12,weight="bold")
    ax.add_patch(Circle((0,0),.48,facecolor=PAPER,edgecolor=INK,lw=2))
    ax.text(0,.08,"Gibbs",ha="center",fontsize=18,weight="bold")
    ax.text(0,-.04,"packet case",ha="center",fontsize=16,weight="bold")
    ax.text(0,-.17,"one worked example",ha="center",fontsize=10,color=MUTED)
    ax.set_xlim(-1.1,1.1)
    ax.set_ylim(-1.1,1.1)
    ax.set_title("The calculation from start to finish",fontsize=20,weight="bold")
    save_figure(fig,figures,"59_summary_wheel")

    # 60 exact/approx boundary
    fig,ax=plt.subplots(figsize=(11.5,5.0))
    ax.axis("off")
    rows=[
        ("Fourier coefficient formula","exact identity",TEAL),
        ("Gibbs scaled limit","proved asymptotic",BLUE),
        ("known packet reconstruction","exact theorem",TEAL),
        ("finite interval search","exact finite optimization",TEAL),
        ("noise curves","deterministic numerical experiment",ORANGE),
        ("off-grid localization","finite numerical experiment",ORANGE),
        ("universal unknown-edge recovery","not claimed",RED),
    ]
    y=.88
    ax.text(.5,.96,"Claim boundary for the single worked example",ha="center",fontsize=20,weight="bold",transform=ax.transAxes)
    for i,(claim,status,color) in enumerate(rows):
        bg=WHITE if i%2==0 else "#EFF2F3"
        ax.add_patch(FancyBboxPatch((.05,y-.047),.90,.068,boxstyle="round,pad=.006,rounding_size=.006",facecolor=bg,edgecolor="none",transform=ax.transAxes))
```

```
        ax.text(.08,y,claim,ha="left",va="center",fontsize=11,transform=ax.transAxes)
        ax.text(.92,y,status,ha="right",va="center",fontsize=10.5,color=color,weight="bold",transform=ax.transAxes)
        y-=.105
    save_figure(fig,figures,"60_claim_boundary")

    # ------------------------------------------------------------------
    # Tests and certificates
    # ------------------------------------------------------------------
    tests: list[tuple[str, bool, float | str]] = []
    def record(name: str, condition: bool, value: float | str) -> None:
        tests.append((name, bool(condition), value))

    # Fourier coefficient checks against numerical quadrature.
    quad_x=np.linspace(-PI,PI,400000,endpoint=False)
    quad_f=full_signal(quad_x)
    for n in [-8,-5,-3,-2,0,2,3,5,8]:
        numeric=np.mean(quad_f*np.exp(-1j*n*quad_x))
        exact=full_fourier_coefficient(n)
        record(f"Fourier coefficient n={n}",abs(numeric-exact)<2e-5,float(abs(numeric-exact)))

    # Model-space projectors and reconstruction.
    model_sample=np.linspace(-PI,PI,4096,endpoint=False)
    ps=smooth_seed(model_sample)
    pj=jump_packet(model_sample)
    ff=ps+pj
    record("packet synthesis",np.max(np.abs(ps+pj-ff))==0.0,float(np.max(np.abs(ps+pj-ff))))
    record("smooth partial sum exact at N=5",np.max(np.abs(smooth_fourier_partial_sum(model_sample,5)-ps))<2e-15,float(np.max(np.abs(smooth_fourier_partial_sum(model_sample,5)-ps))))
    record("resolved reconstruction exact",np.max(np.abs(packet_reconstruction(model_sample,5)-ff))<2e-15,float(np.max(np.abs(packet_reconstruction(model_sample,5)-ff))))

    # Gibbs profile and constant.
    profile_error=max(abs((jump_fourier_partial_sum(np.array([EDGE_HALF_WIDTH-z/(400+.5)]),400)[0])-gibbs_profile(np.array([z]))[0]) for z in [1.0,PI,5.0])
    record("scaled Gibbs profile",profile_error<4e-3,float(profile_error))
    observed_G=jump_fourier_partial_sum(np.array([EDGE_HALF_WIDTH-PI/(1000+.5)]),1000)[0]-1.0
    record("Gibbs constant",abs(observed_G-gibbs_constant())<1e-3,float(abs(observed_G-gibbs_constant())))

    # Fejer positivity for the jump.
    fejer_test=filtered_jump_sum(model_sample,80,"fejer")
    record("Fejer positivity",np.min(fejer_test)>-1e-12 and np.max(fejer_test)<1+1e-12,float(max(-np.min(fejer_test),np.max(fejer_test)-1)))

    # Exact finite search.
    record("true interval recovered",int(exact_search["start"])==TRUE_START and int(exact_search["length"])==TRUE_LENGTH,f"start={exact_search['start']}, length={exact_search['length']}")
    record("edge amplitude recovered",abs(float(exact_search["amplitude"])-1.0)<1e-12,float(abs(float(exact_search["amplitude"])-1.0)))
    record("finite reconstruction",np.max(np.abs(np.asarray(exact_search["reconstruction"])-y_grid))<2e-14,float(np.max(np.abs(np.asarray(exact_search["reconstruction"])-y_grid))))
    record("positive objective gap",float(exact_search["gap"])>.5,float(exact_search["gap"]))

    # Residue pattern.
    residue_ok=True
    for n in range(1,61):
        expected=0.0 if n%3==0 else (math.sqrt(3)/2 if n%6 in (1,2) else -math.sqrt(3)/2)
        residue_ok &= abs(math.sin(n*PI/3)-expected)<1e-12
    record("six-phase coefficient pattern",residue_ok,0.0 if residue_ok else 1.0)

    # Perturbation identity.
    qshift=interval_mask(DEFAULT_GRID,TRUE_START+4,TRUE_LENGTH)
    lhs=np.linalg.norm(edge_grid-qshift)**2
    rhs=np.count_nonzero(edge_grid!=qshift)
    record("symmetric-difference identity",abs(lhs-rhs)<1e-12,float(abs(lhs-rhs)))

    # Off-grid localization bounded by 1.1 grid spacings in the tested family.
    offgrid_ratio=max(r["max_endpoint_error"]/r["dx"] for r in offgrid_rows)
    record("off-grid localization check",offgrid_ratio<1.1,float(offgrid_ratio))

    # Noise negative/positive controls.
```

```
    rate03=next(r["exact_support_rate"] for r in noise_rows if abs(r["sigma"]-.03)<1e-12)
    rate20=next(r["exact_support_rate"] for r in noise_rows if abs(r["sigma"]-.20)<1e-12)
    record("low-noise support recovery",rate03>.98,float(rate03))
    record("high-noise degradation visible",rate20<1.0,float(rate20))

    # Method boundary: raw has persistent overshoot. Packet reconstruction has zero overshoot.
    raw160=next(r for r in metric_rows if r["method"]=="raw" and r["order"]==160)
    packet5=next(r for r in metric_rows if r["method"]=="packet" and r["order"]==5)
    record("raw Gibbs persists",abs(float(raw160["jump_overshoot"])-gibbs_constant())<5e-3,float(raw160["
     jump_overshoot"]))
    record("packet overshoot zero",abs(float(packet5["jump_overshoot"]))<1e-14,float(packet5["jump_overshoot
     "]))

    # Generate test dashboard now.
    fig,ax=plt.subplots(figsize=(11.5,7.0))
    ax.axis("off")
    ax.text(.5,.965,"Analytic and numerical checks for the worked example",ha="center",fontsize=20,weight="
     bold",transform=ax.transAxes)
    y=.91
    for i,(name,passed,value) in enumerate(tests):
        bg=WHITE if i%2==0 else "#EFF2F3"
        ax.add_patch(FancyBboxPatch((.04,y-.022),.92,.040,boxstyle="round,pad=.003,rounding_size=.004",
     facecolor=bg,edgecolor="none",transform=ax.transAxes))
        ax.text(.06,y,name,ha="left",va="center",fontsize=8.7,transform=ax.transAxes)
        ax.text(.79,y,str(value)[:22],ha="right",va="center",fontsize=8.4,family="monospace",transform=ax.
     transAxes)
        ax.text(.94,y,"PASS" if passed else "FAIL",ha="right",va="center",fontsize=8.8,color=TEAL if passed
     else RED,weight="bold",transform=ax.transAxes)
        y-=.043
    ax.text(.5,.025,f"{sum(p for _, p, _ in tests)}/{len(tests)} checks passed",ha="center",fontsize=12,
     weight="bold",transform=ax.transAxes)
    save_figure(fig,figures,"58_test_dashboard")

    # Test ledger.
    with (tables/"test_ledger.csv").open("w",newline="",encoding="utf-8") as handle:
        writer=csv.DictWriter(handle,fieldnames=["test","passed","value"])
        writer.writeheader()
        writer.writerows([{"test":n,"passed":p,"value":v} for n,p,v in tests])

    results = {
        "signal": {
            "smooth_formula": "1/4 + (2/5) cos(2x) - (1/3) sin(3x) + (1/5) cos(5x)",
            "edge_half_width": EDGE_HALF_WIDTH,
            "edge_amplitude": A_EDGE,
            "smooth_degree": SMOOTH_DEGREE,
        },
        "gibbs_constant": gibbs_constant(),
        "exact_recovery": {
            "grid_size": DEFAULT_GRID,
            "true_start": TRUE_START,
            "true_length": TRUE_LENGTH,
            "recovered_start": int(exact_search["start"]),
            "recovered_length": int(exact_search["length"]),
            "recovered_amplitude": float(exact_search["amplitude"]),
            "residual": float(exact_search["residual"]),
            "second_residual": float(exact_search["second_residual"]),
            "gap": float(exact_search["gap"]),
            "max_reconstruction_residual": float(np.max(np.abs(np.asarray(exact_search["reconstruction"])-
     y_grid))),
        },
        "tests": {
            "passed": int(sum(p for _,p,_ in tests)),
            "total": len(tests),
        },
        "figure_pairs": 60,
        "method_metrics_rows": len(metric_rows),
        "noise_rows": len(noise_rows),
        "offgrid_rows": len(offgrid_rows),
        "runtime_rows": len(runtime_rows),
    }
    (results_dir/"numerical_results.json").write_text(json.dumps(results,indent=2,sort_keys=True)+"\n",
```

```
     encoding="utf-8")

    if not all(p for _,p,_ in tests):
        failed=[n for n,p,_ in tests if not p]
        raise AssertionError(f"failed checks: {failed}")
    return results


# ---------------------------------------------------------------------------
# File manifest utilities
# ---------------------------------------------------------------------------
def sha256(path: Path) -> str:
    h=hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1<<20),b""):
            h.update(chunk)
    return h.hexdigest()


def write_manifest(root: Path) -> None:
    files=[]
    for path in sorted(root.rglob("*")):
        if path.is_file() and path.name not in {"SHA256SUMS.txt", "manifest.json"} and "build" not in path.
     parts:
            files.append({"path":str(path.relative_to(root)),"bytes":path.stat().st_size,"sha256":sha256(
     path)})
    (root/"manifest.json").write_text(json.dumps({"files":files},indent=2)+"\n",encoding="utf-8")
    lines=[f"{row['sha256']}  {row['path']}" for row in files]
    (root/"SHA256SUMS.txt").write_text("\n".join(lines)+"\n",encoding="utf-8")


def main() -> None:
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--root",type=Path,default=Path(__file__).resolve().parent)
    parser.add_argument("--quick",action="store_true")
    parser.add_argument("--manifest",action="store_true")
    args=parser.parse_args()
    results=build_experiment(args.root,args.quick)
    if args.manifest:
        write_manifest(args.root)
    print(json.dumps(results,indent=2,sort_keys=True))


if __name__=="__main__":
    main()
```

D File map

Path	Role
`gibbs_packet_case.py`	Single implementation, experiment generator, test suite, and manifest writer
`Projector_Resolved_Gibbs_Case_Study.tex`	Complete monograph source
`figures/`	Sixty vector-PDF and PNG figure pairs
`tables/`	Exact and numerical CSV ledgers
`results/`	JSON summary and PDF audit records
`Run_All.ps1`	Windows one-command runner
`run_all.sh`	POSIX one-command runner
`manifest.json`	File inventory with sizes and hashes
`SHA256SUMS.txt`	Release checksum ledger

E What this example establishes

The exact result concerns the model and finite interval library stated in the text. Its mathematical content can be summarized as follows:

> A discontinuity need not be approximated by the same global spectral block used for the smooth component. When the model supplies a complete smooth-edge packet split, the edge can be retained exactly, the smooth packet can be treated spectrally, and the resulting synthesis has no jump-generated Gibbs error.

The corresponding universal problem is to discover and certify suitable singular packets for broader data classes. That question remains a separate theorem program.